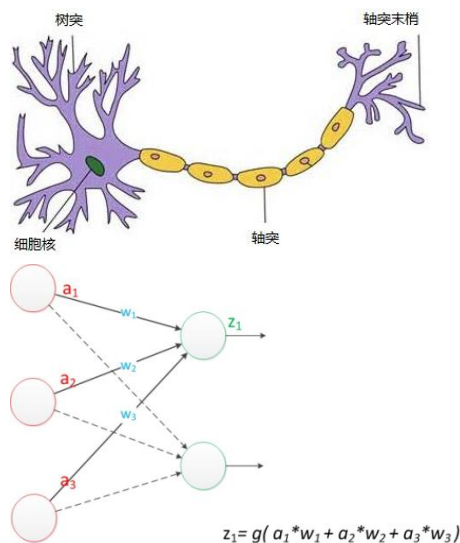


神经网络的历史

Brief History of Neural Networks

- The First wave
 - 1943 McCulloch and Pitts proposed the **McCulloch-Pitts neuron model**.
 - 1958 Rosenblatt introduced the simple single layer networks now called **Perceptrons**.
 - 1969 Minsky and Papert's book Perceptrons demonstrated the limitation of single layer perceptrons, and almost the whole field went into hibernation.

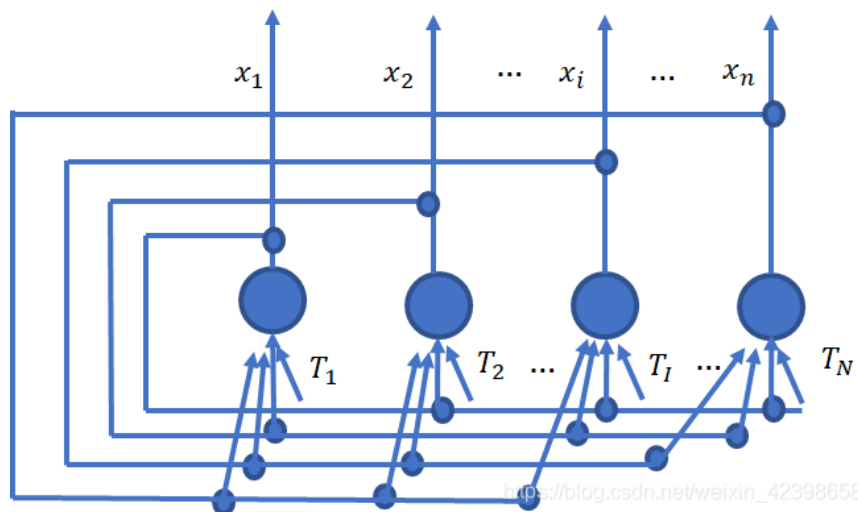


Rosenblatt and perceptron

神经网络的历史

Brief History of Neural Networks

- The Second wave
 - 1982 Energy Based Model (EBM), the hopfield network
 - 1984 the traveling salesman problem (abbr. TSP), one of the NP-complete combinatorial optimization problems, can be solved by Hopfield neural network



Hopfield, John J. "Neural Networks and Physical Systems with Emergent Collective Computational Abilities." Proceedings of the National Academy of Sciences 79.8 (1982): 2554-2558.

神经网络的历史

Brief History of Neural Networks

- The Second wave
 - 1986 The **Back-Propagation learning algorithm** for Multi-Layer Perceptrons was rediscovered and the whole field took off again.



David Rumelhart



Geoffery Hinton

Learning representations by back-propagating errors

David E. Rumelhart*, Geoffrey E. Hinton†
& Ronald J. Williams*

* Institute for Cognitive Science, C-015, University of California,
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,
Pittsburgh, Philadelphia 15213, USA

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure¹.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons, there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input, x_j , to unit j is a linear function of the outputs, y_i , of the units that are connected to j and of the weights, w_{ji} , on these connections

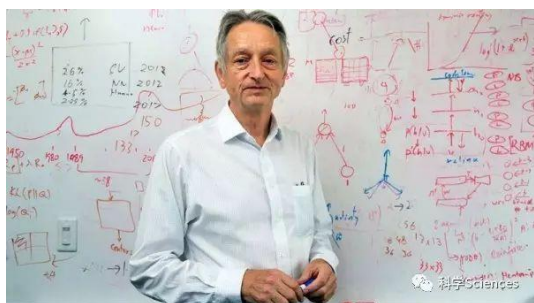
$$x_j = \sum_i y_i w_{ji} \quad (1)$$

Units can be given biases by introducing an extra input to each

神经网络的历史

Brief History of Neural Networks

- The Third wave
 - 2006 **Deep (neural networks) Learning** gains popularity and
 - 2012 made significant break-through in many applications.
 - AlexNet—ImageNet champion model



Geoffery Hinton



Microsoft DNN research



Google Brain

Breaking News of AI in 2016

- AlphaGo wins Lee Sedol (4-1)



<https://deepmind.com/research/alphago/>

Rank	Name	Rank	Flag	Elo
1	Ke Jie	↑	🇨🇳	3628
2	AlphaGo		🇬🇧	3598
3	Park Junghwan	↑	🇰🇷	3585
4	Tuo Liang	↑	🇨🇳	3535
5	Mi Yuting	↑	🇨🇳	3534
6	Iyama Yuta	↑	🇯🇵	3525
7	Shi Yue	↑	🇨🇳	3522
8	Lee Sedol	↑	🇰🇷	3521
9	Zhou Ruiyang	↑	🇨🇳	3517
10	Shin Jinseo	↑	🇰🇷	3503
11	Chen Yaoye	↑	🇨🇳	3495
12	Lian Xiao	↑	🇨🇳	3493
13	Tan Xiao	↑	🇨🇳	3489
14	Kim Jiseok	↑	🇰🇷	3489
15	Choi Cheolhan	↑	🇰🇷	3482
16	Park Yeonghan	↑	🇰🇷	3482
17	Gu Zihao	↑	🇨🇳	3468
18	Fan Yunruo	↑	🇨🇳	3468
19	Huang Yunsong	↑	🇨🇳	3467
20	Li Qincheng	↑	🇨🇳	3465
21	Tang Weixing	↑	🇨🇳	3461
22	Lee Donghoon	↑	🇰🇷	3460
23	Lee Yeongkyu	↑	🇰🇷	3459
24	Fan Tingyu	↑	🇨🇳	3459
25	Tang Mengcheng	↑	🇨🇳	3447
26	Kang Dongyan	↑	🇰🇷	3442
27	Yang Xi	↑	🇨🇳	3439
28	Veon Seongjin	↑	🇰🇷	3439
29	Yang Dingxin	↑	🇨🇳	3439
30	Gu Li	↑	🇨🇳	3436

<https://www.goratings.org/>

Deep Learning Attracts Lots of Attention

I believe you have seen lots of exciting results before !



无人驾驶



机器人



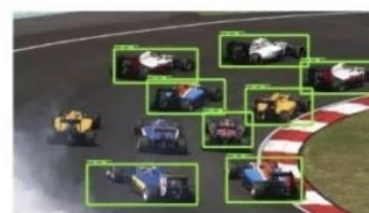
游戏



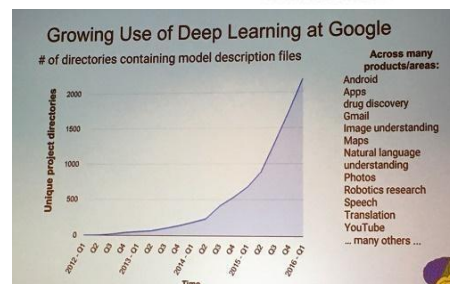
自然语言处理



智能推荐



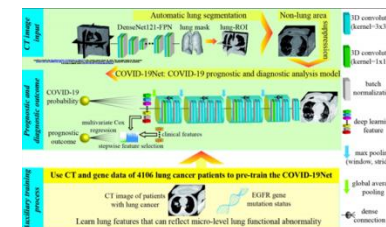
计算机视觉



Deep learning trends at Google.



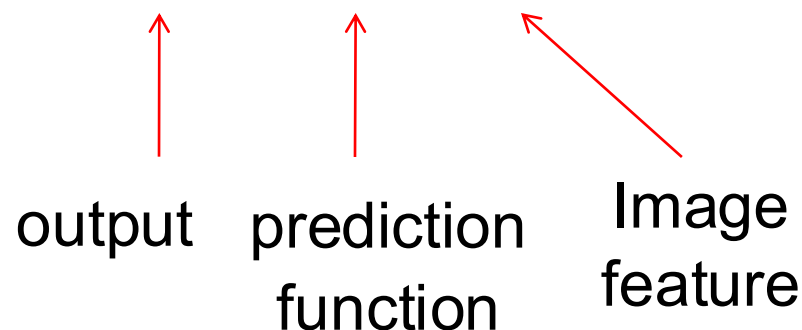
金融风控



医学影像

深度学习和传统的机器学习相比有什么不同？

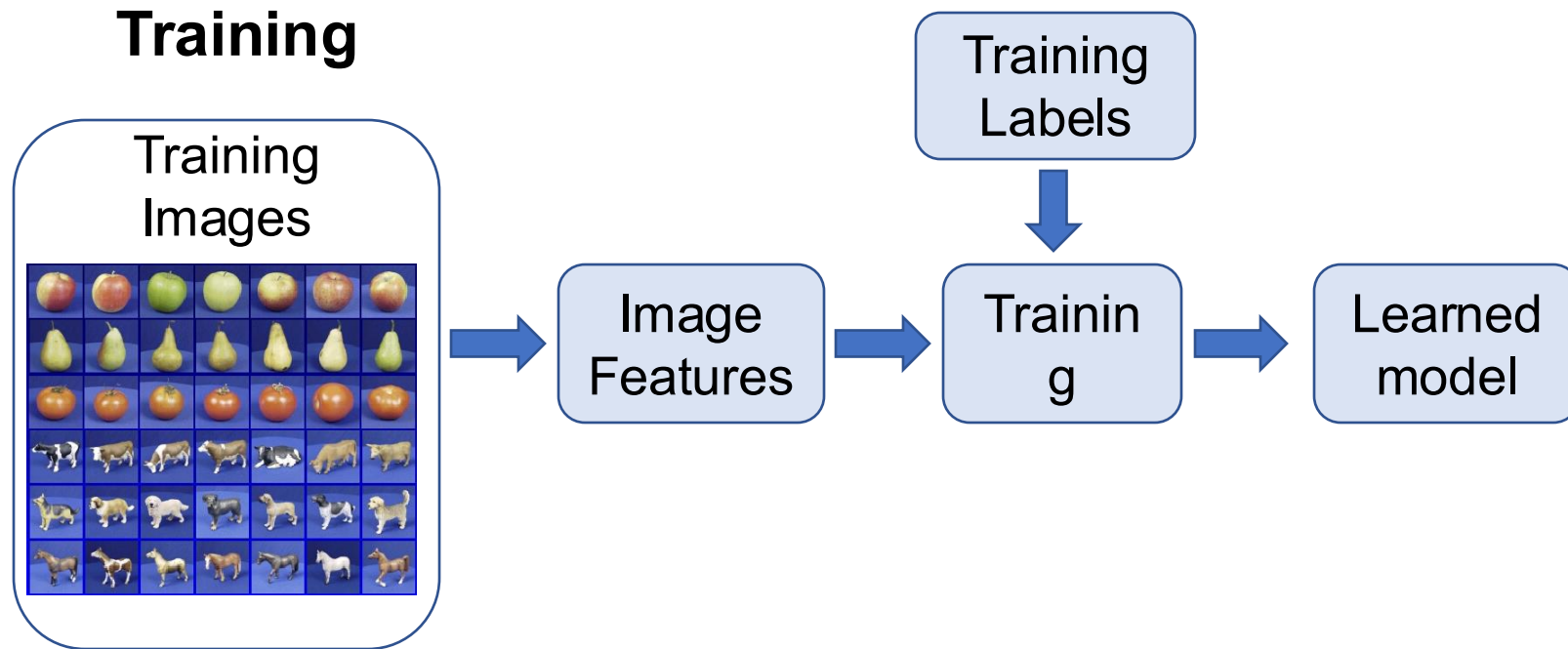
Machine Learning Framework

$$y = f(\mathbf{x})$$


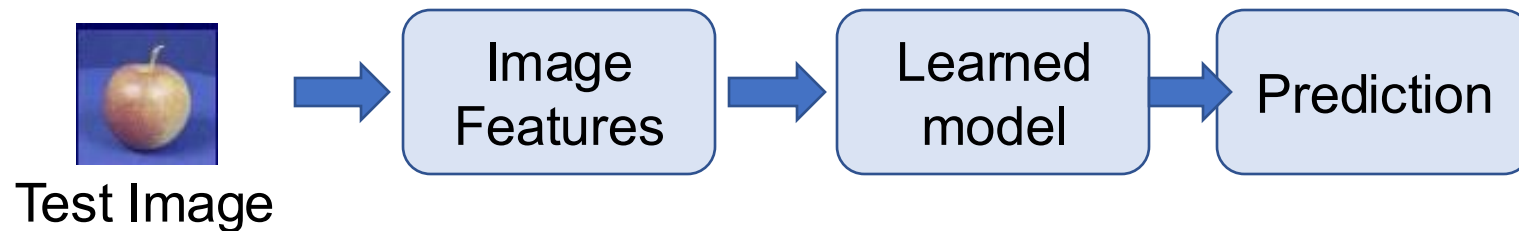
- **Training:** given a *training set* of labeled examples $\{(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_N, y_N)\}$, estimate the prediction function f by minimizing the prediction error on the training set
- **Testing:** apply f to a never before seen *test example* $\mathbf{x}$ and output the predicted value $y = f(\mathbf{x})$

Machine Learning Framework

Training



Testing



Machine Learning Framework

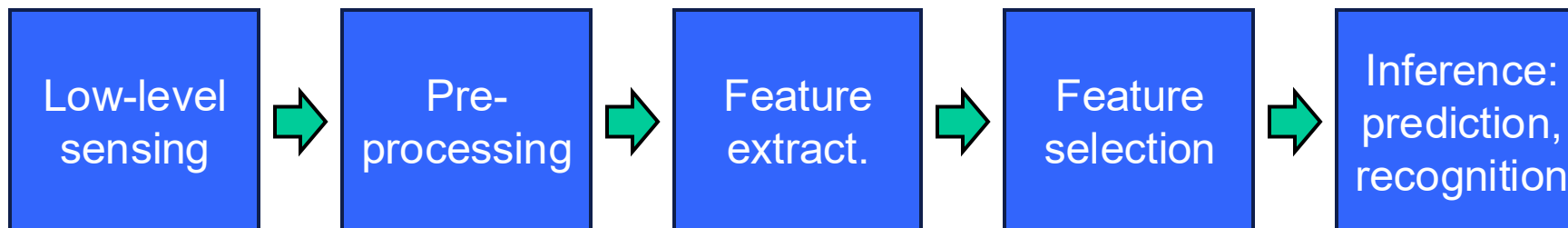
- Apply a prediction function to a feature representation of the image to get the desired output:

$$f(\text{apple image}) = \text{"apple"}$$

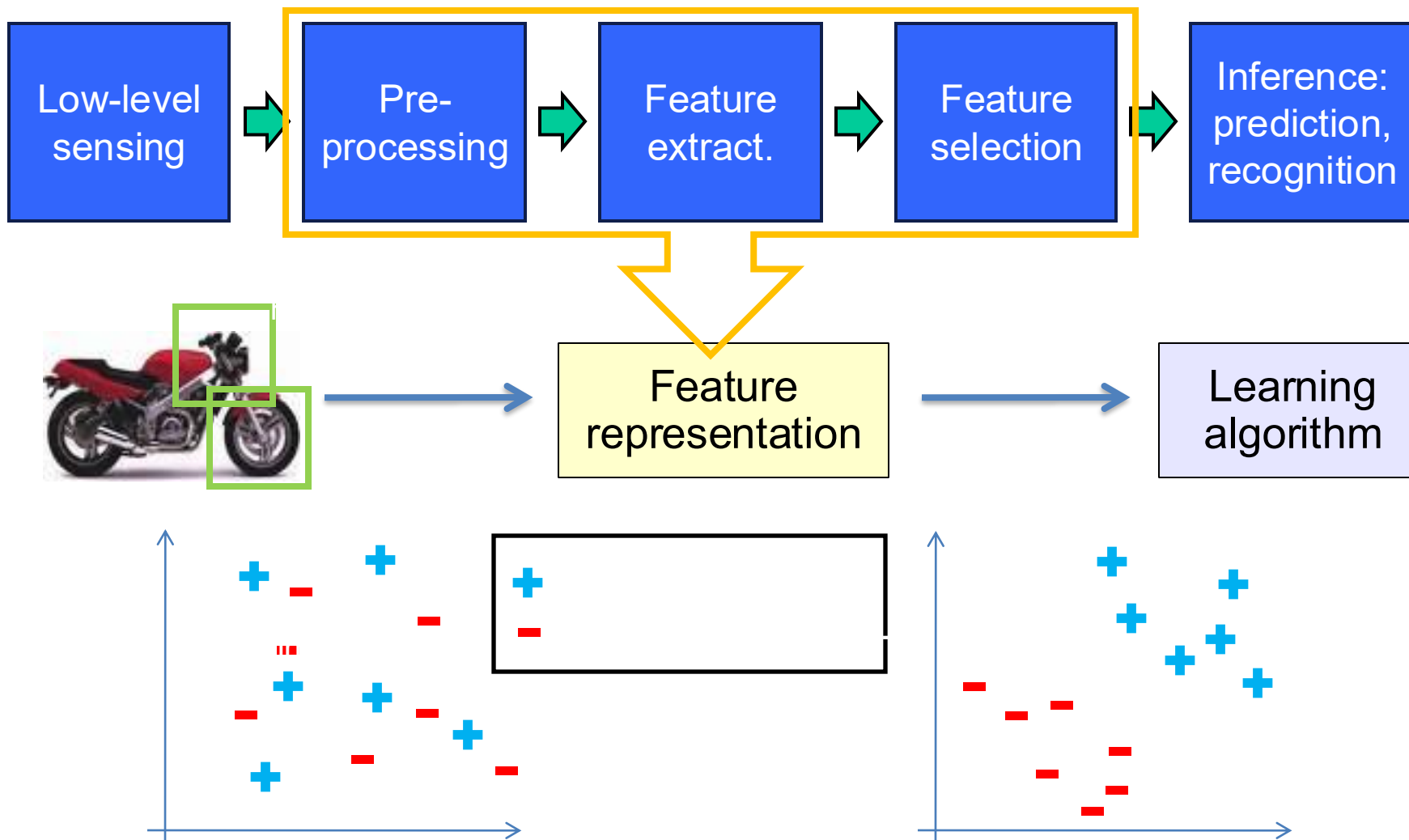
$$f(\text{tomato image}) = \text{"tomato"}$$

$$f(\text{cow image}) = \text{"cow"}$$

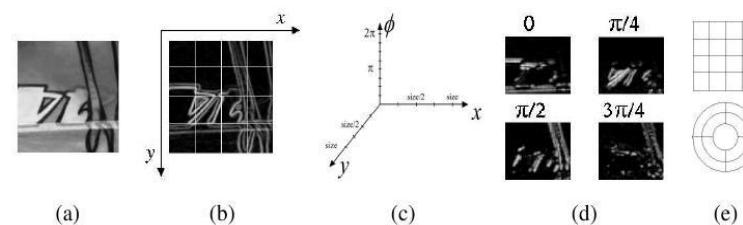
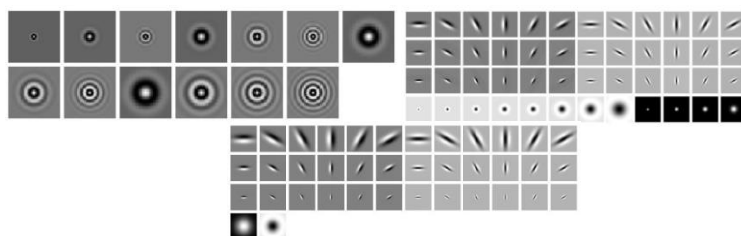
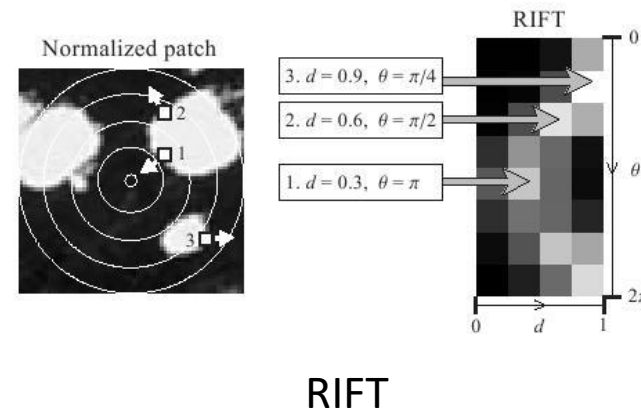
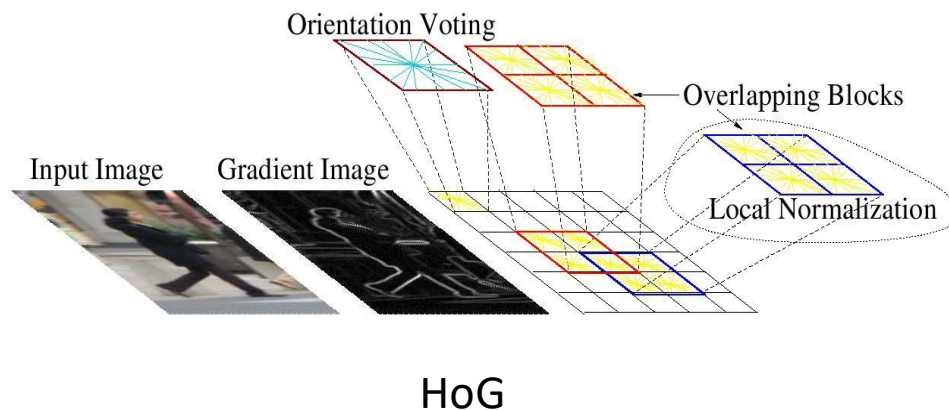
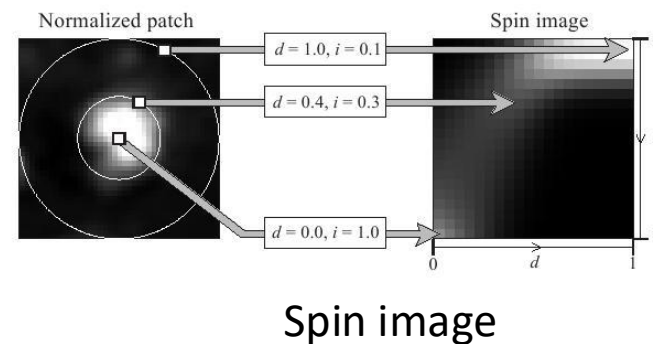
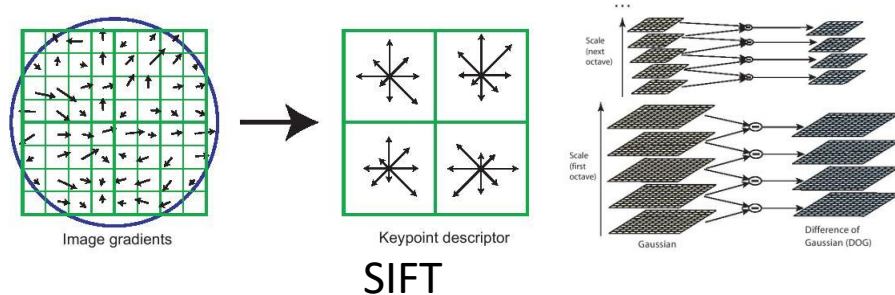
General Machine Learning Process



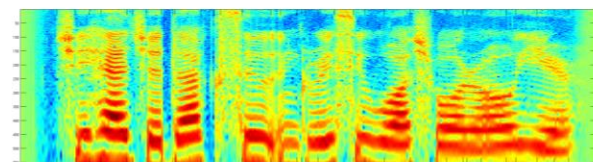
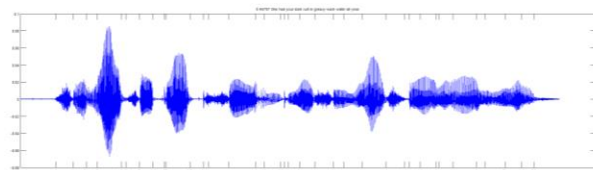
Feature Representations



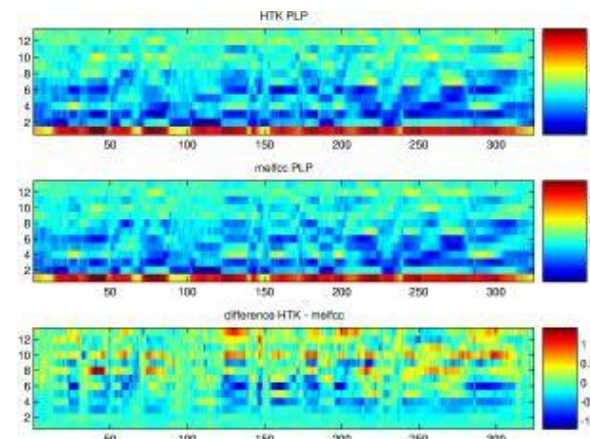
Computer Vision Features



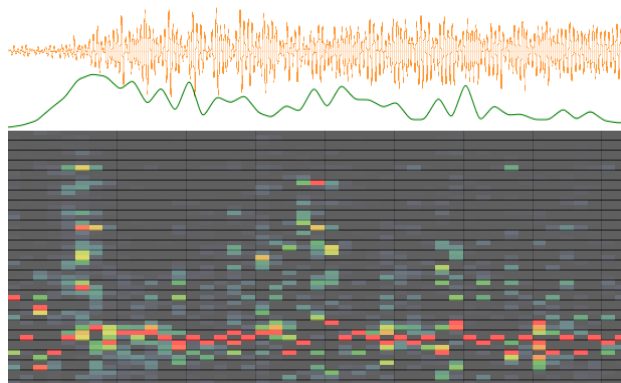
Audio Features



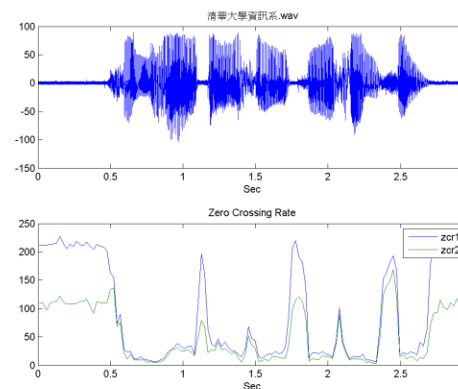
Spectrogram



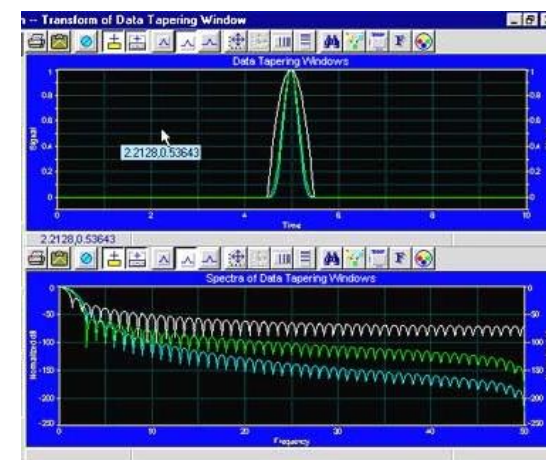
MFCC



Flux



ZCR



Rolloff

Feature Engineering

- **Manually designed features** dominate the applications of image and video understanding (actually , and many other applications) in the past
 - Rely on human domain knowledge much more than data
 - Feature design is separate from training the classifier
 - Developing effective features for new applications is slow
- **The performance of a machine learning system heavily depends on feature representations!**

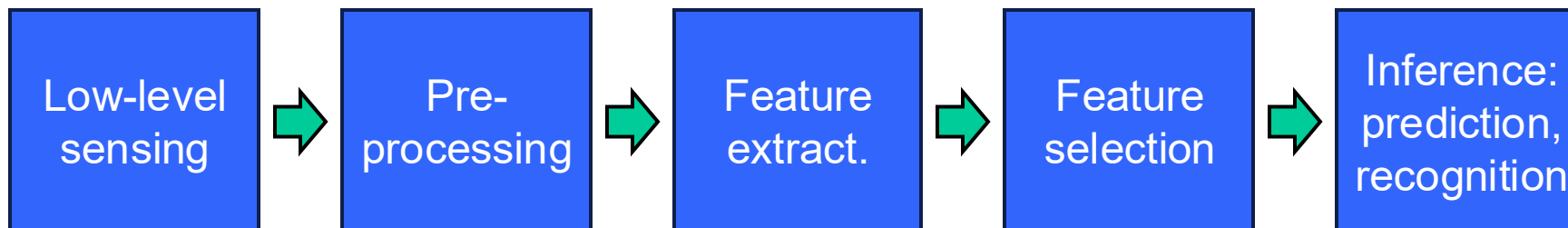
Deep Learning Means Feature Learning!

- Deep learning methods are representation-learning methods with **multiple levels of representation**, obtained by composing simple but **non-linear modules** that each transform the representation at one level (starting with the raw input) into a representation at a higher, slightly more abstract level.
- Mostly implemented via neural networks

[LeCun, Bengio and Hinton. Deep Learning. Nature 2015.]

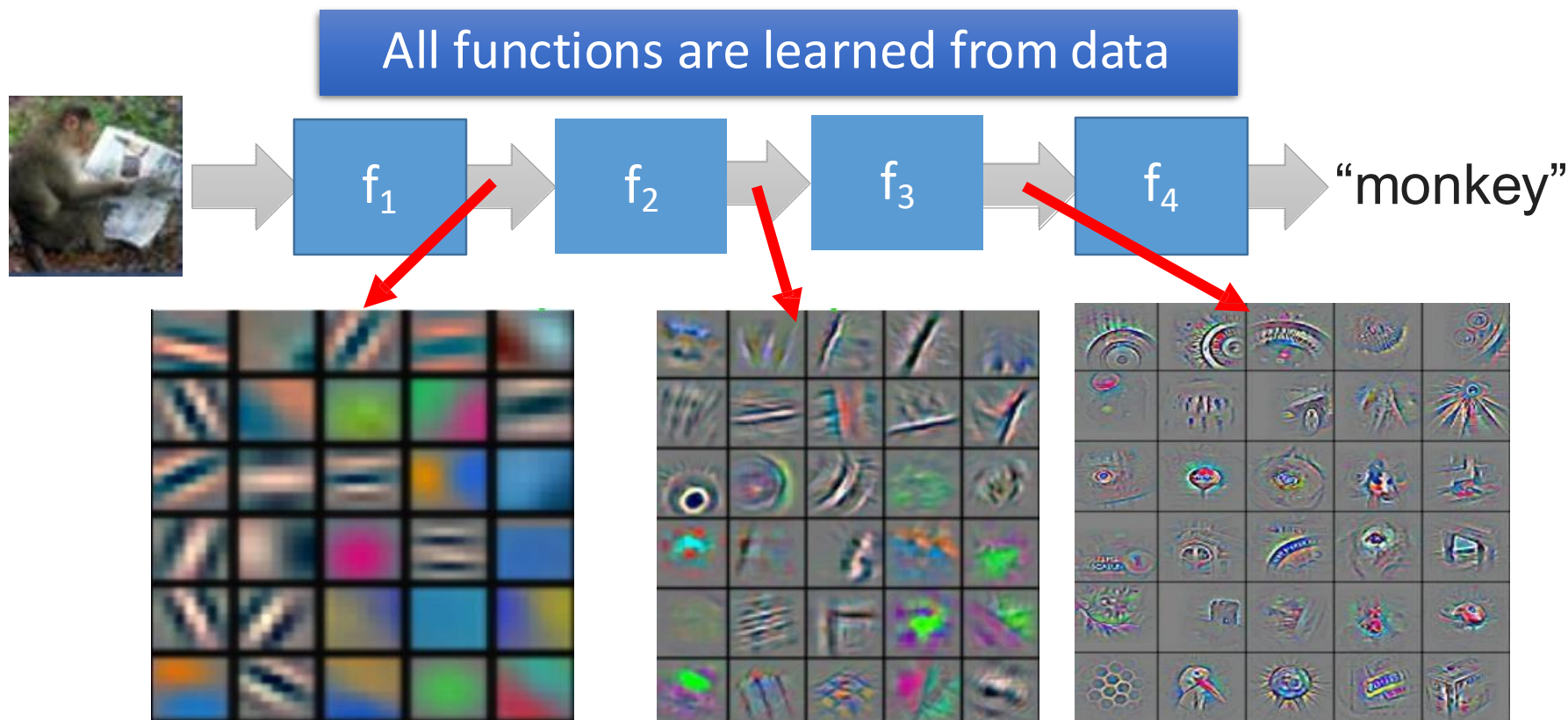


General Machine Learning Process



End-to-end Learning- Image Recognition

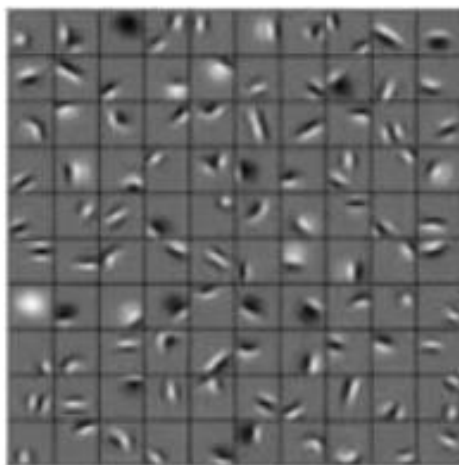
- Deep Learning



Zeiler, Matthew D., and Rob Fergus. "Visualizing and Understanding Convolutional Networks." In Proceedings of the European Conference on Computer Vision, pp. 818-833, 2014

End-to-end Learning- Image Recognition

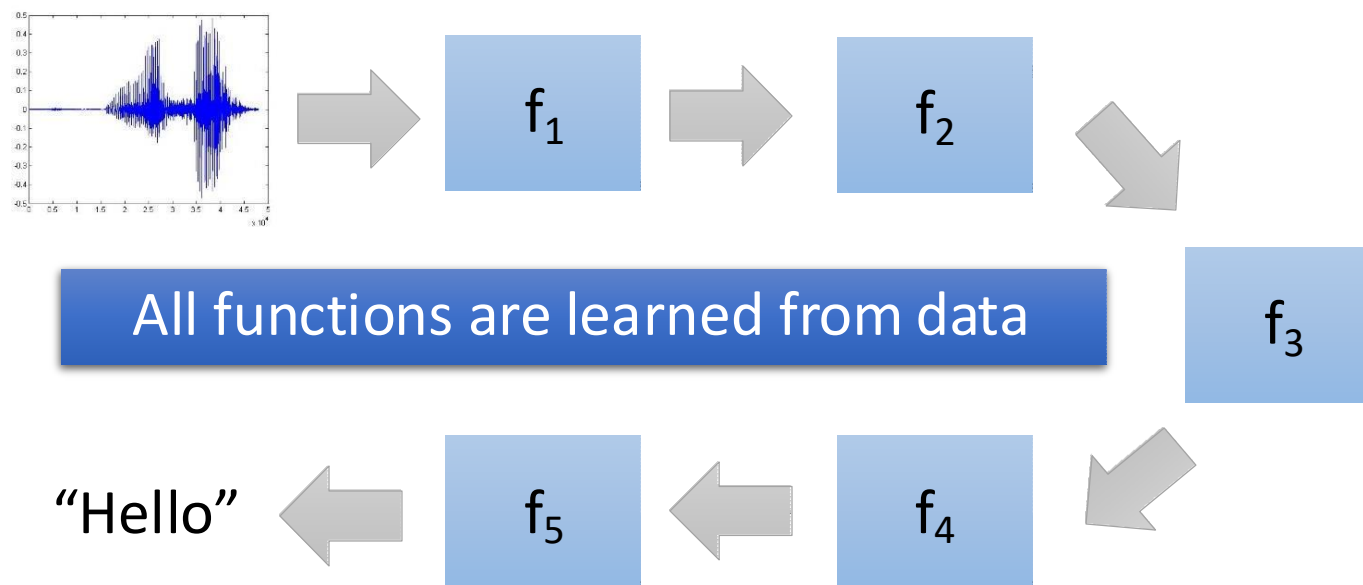
- Deep Learning



by Andrew Ng

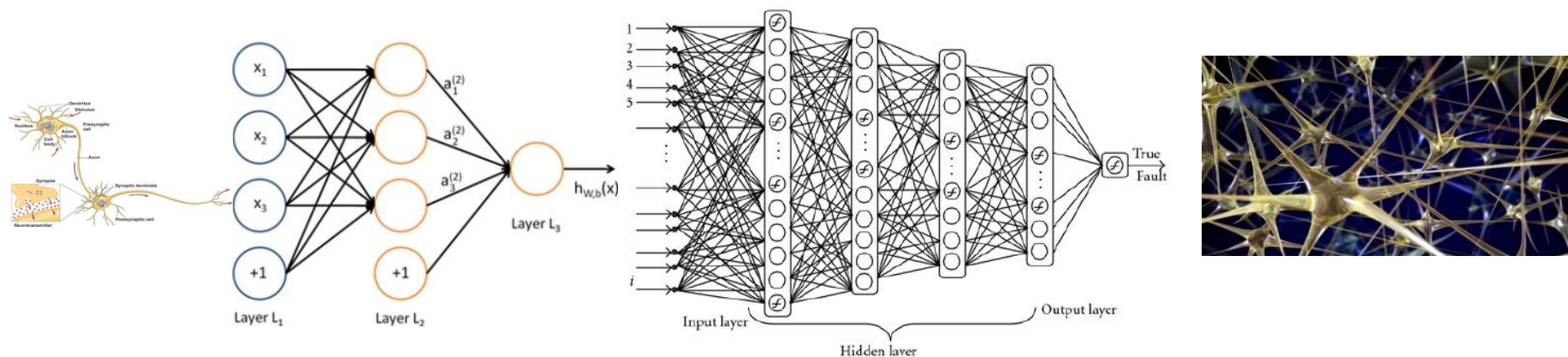
End-to-end Learning - Speech Recognition

- Deep Learning

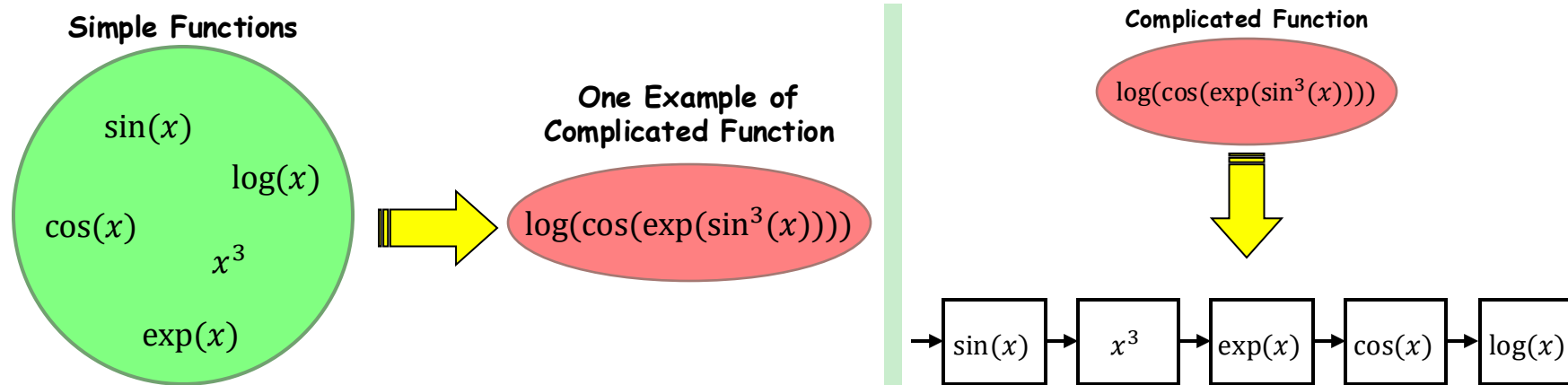


Less engineering labor, but machine learns more

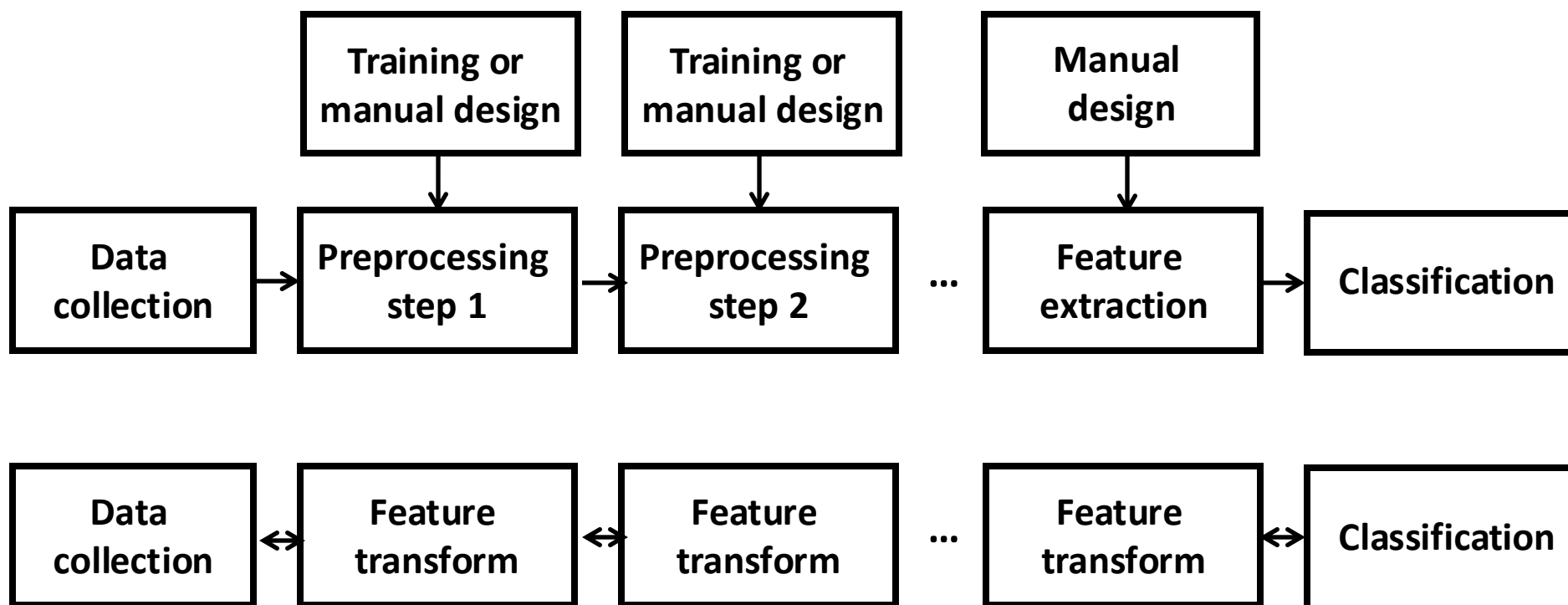
A Higher & More Abstract Level Learning



Multi-layer perceptron with many hidden layers



Joint Learning vs Separate Learning

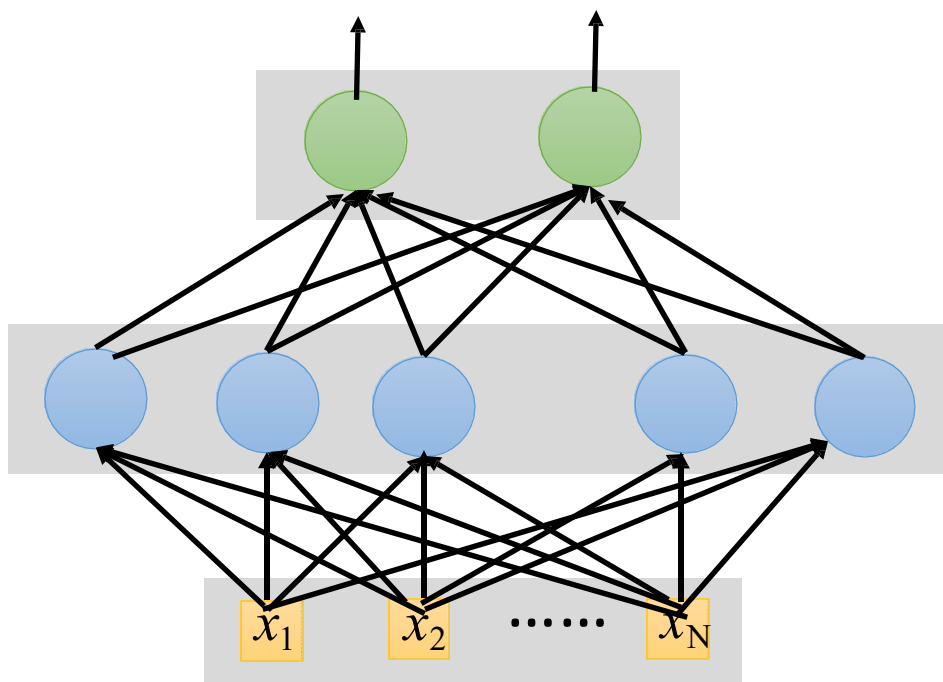


Deep learning's power comes from jointly learning feature (multiple non-linear transformations) and classifiers !

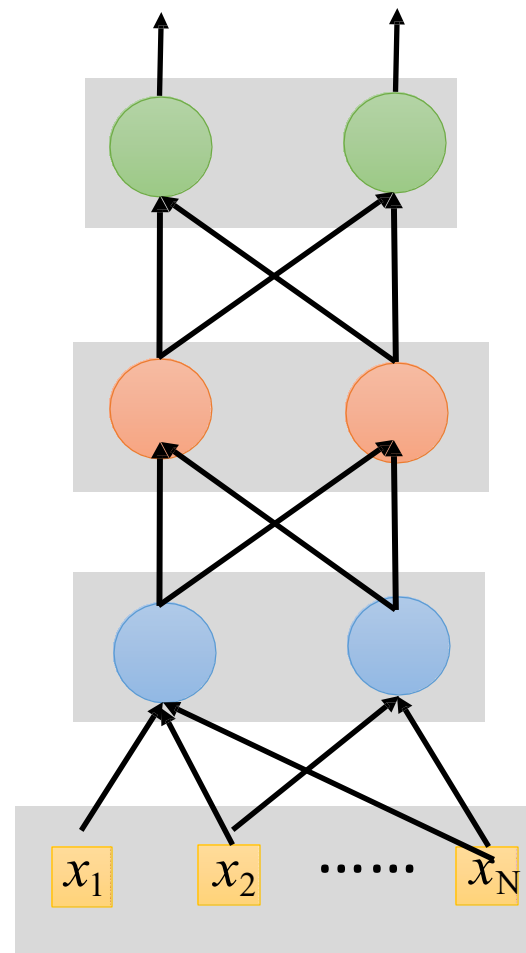
为什么是“深度学习” 而不是“宽度学习”？

Fat + Short v.s. Thin + Tall

The same number
of parameters

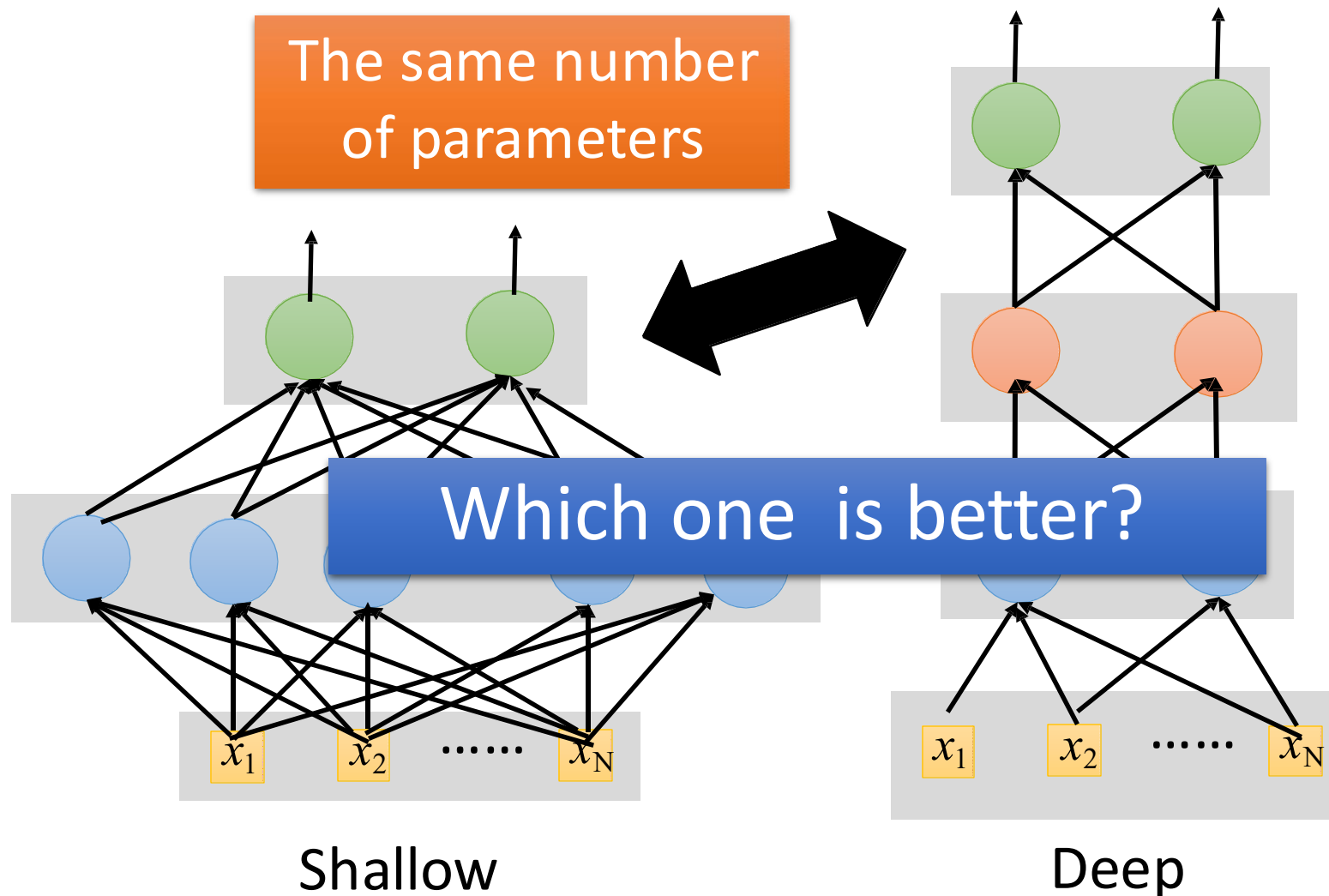


Shallow



Deep

Fat + Short v.s. Thin + Tall



Deeper is Better?

Layer X Size	Word Error Rate (%)
1 X 2k	24.2
2 X 2k	20.4
3 X 2k	18.4
4 X 2k	17.8
5 X 2k	17.2
7 X 2k	17.1

Not surprised, more parameters, better performance

Seide, Frank, Gang Li, and Dong Yu. "Conversational Speech Transcription Using Context-Dependent Deep Neural Networks." *Interspeech*. 2011.

Fat + Short v.s. Thin + Tall

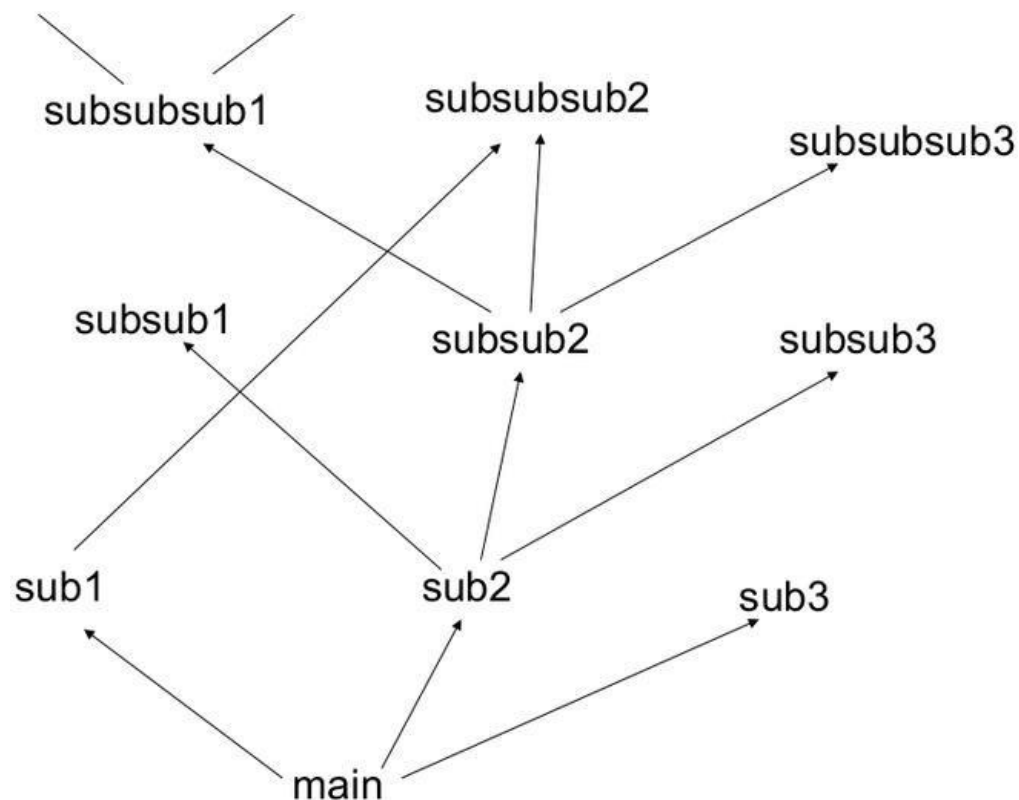
Layer X Size	Word Error Rate (%)	Layer X Size	Word Error Rate (%)
1 X 2k	24.2		
2 X 2k	20.4		
3 X 2k	18.4		
4 X 2k	17.8		
5 X 2k	17.2	1 X 3772	22.5
7 X 2k	17.1	1 X 4634	22.6
		1 X 16k	22.1

Seide, Frank, Gang Li, and Dong Yu. "Conversational Speech Transcription Using Context-Dependent Deep Neural Networks." *Interspeech*. 2011.

Modularization

- Deep → Modularization

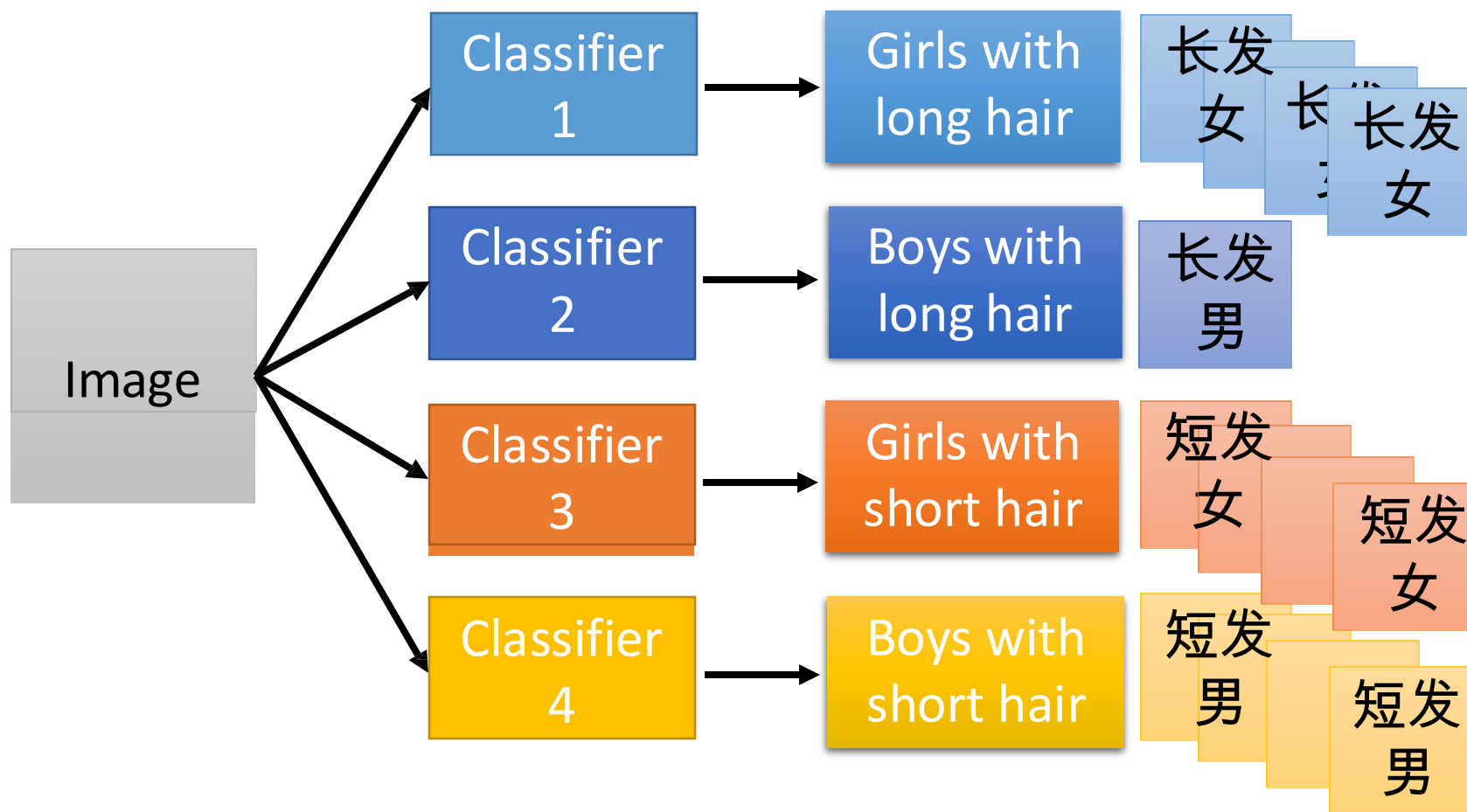
Don't put
everything in your
main function.



<http://rinuboney.github.io/2015/10/18/theoretical-motivations-deep-learning.html>

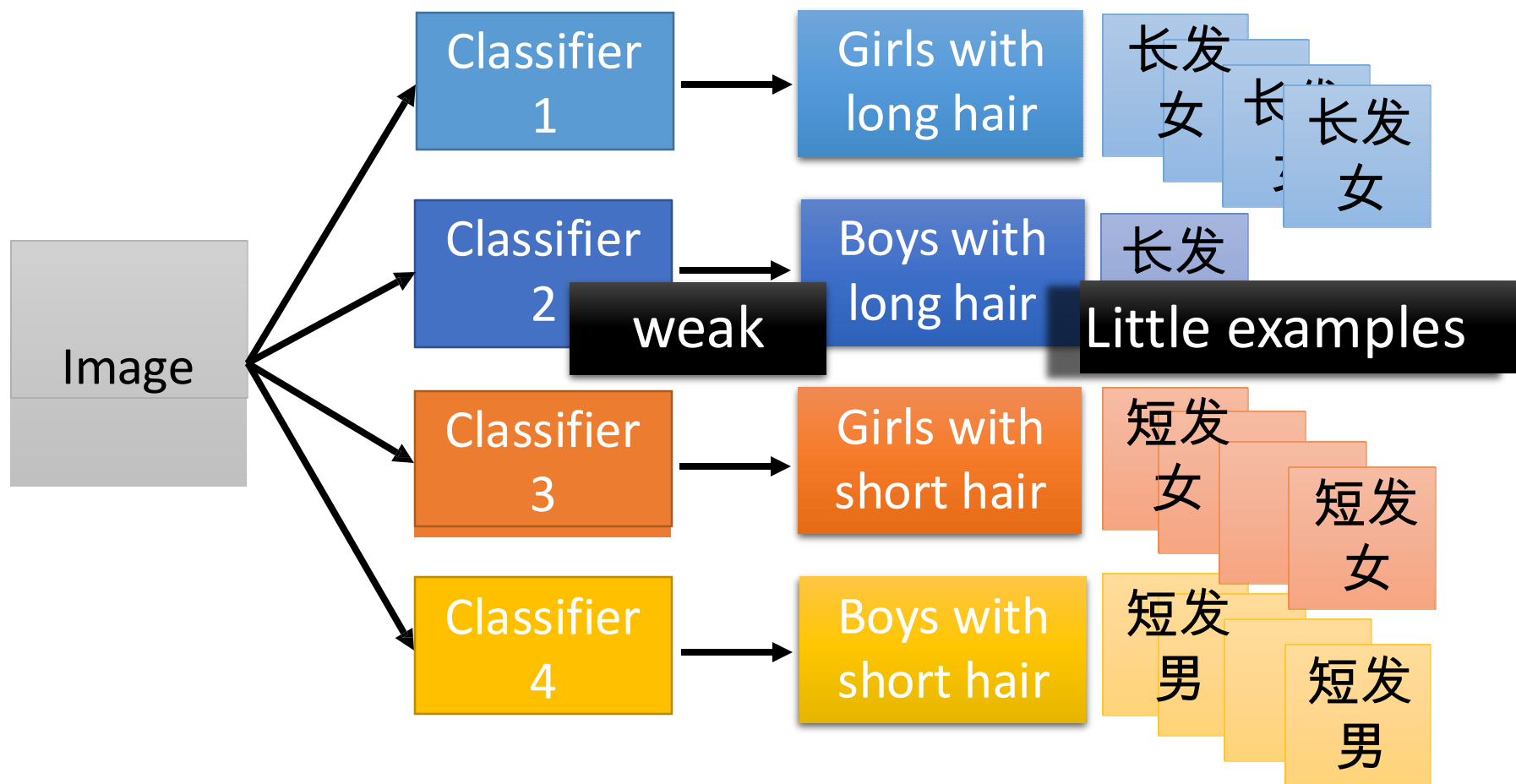
Modularization

- Deep → Modularization



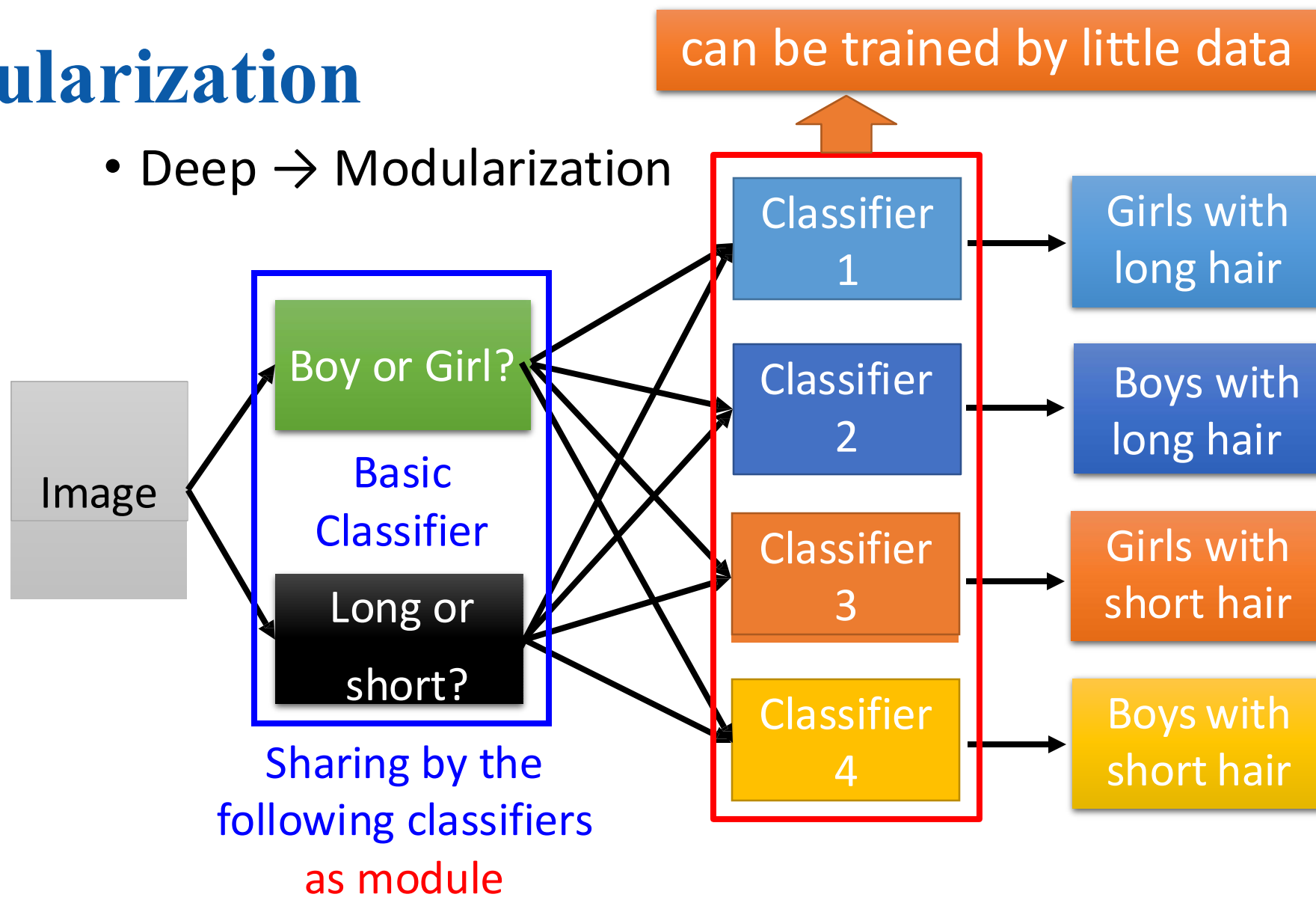
Modularization

- Deep $\rightarrow$ Modularization



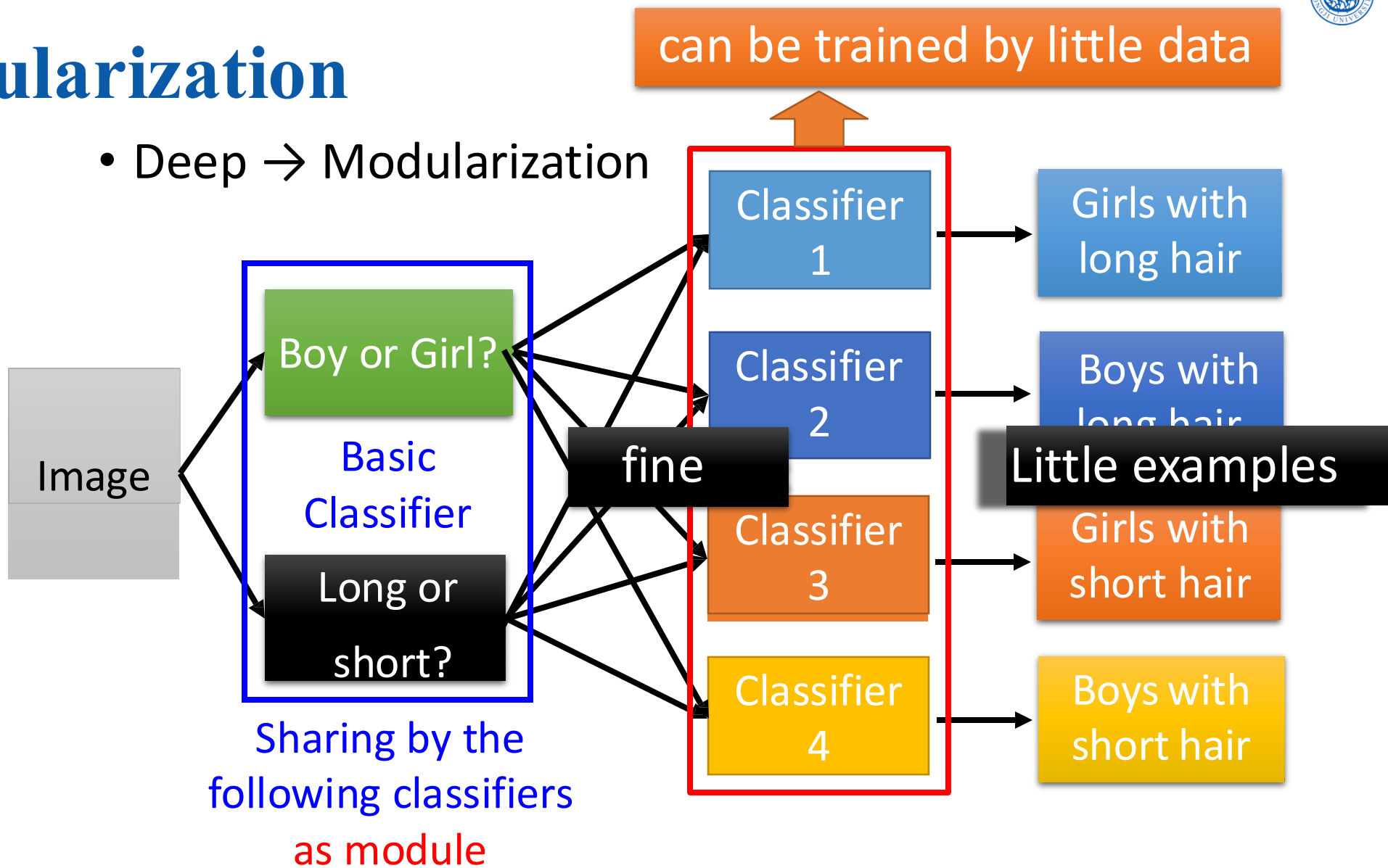
Modularization

- Deep → Modularization



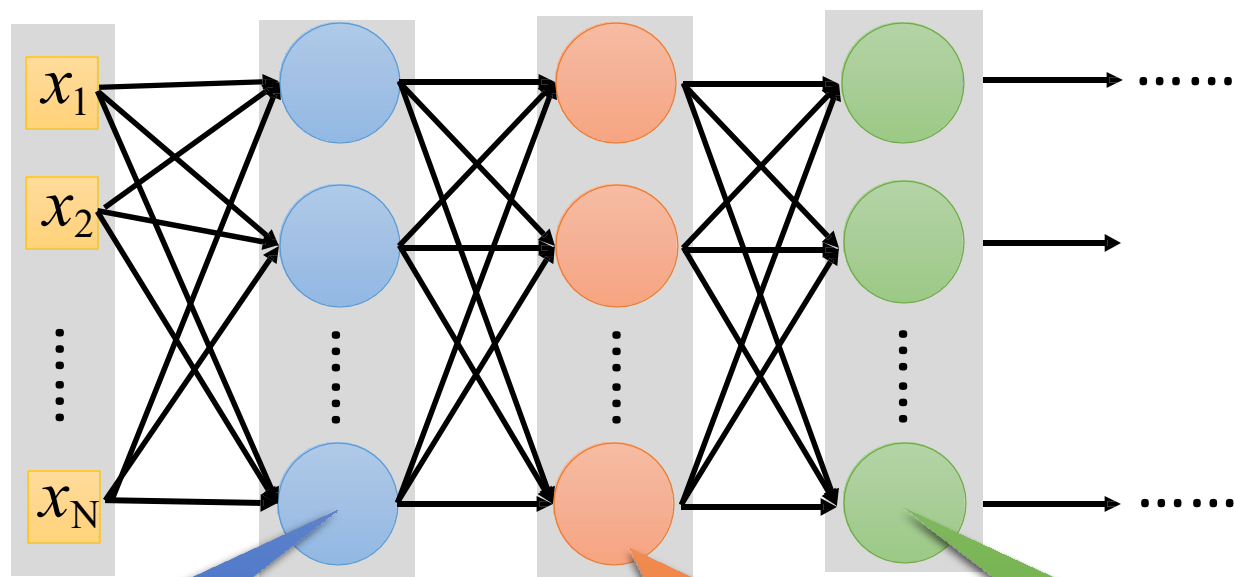
Modularization

- Deep → Modularization



Modularization

- Deep → Modularization → Less training data?



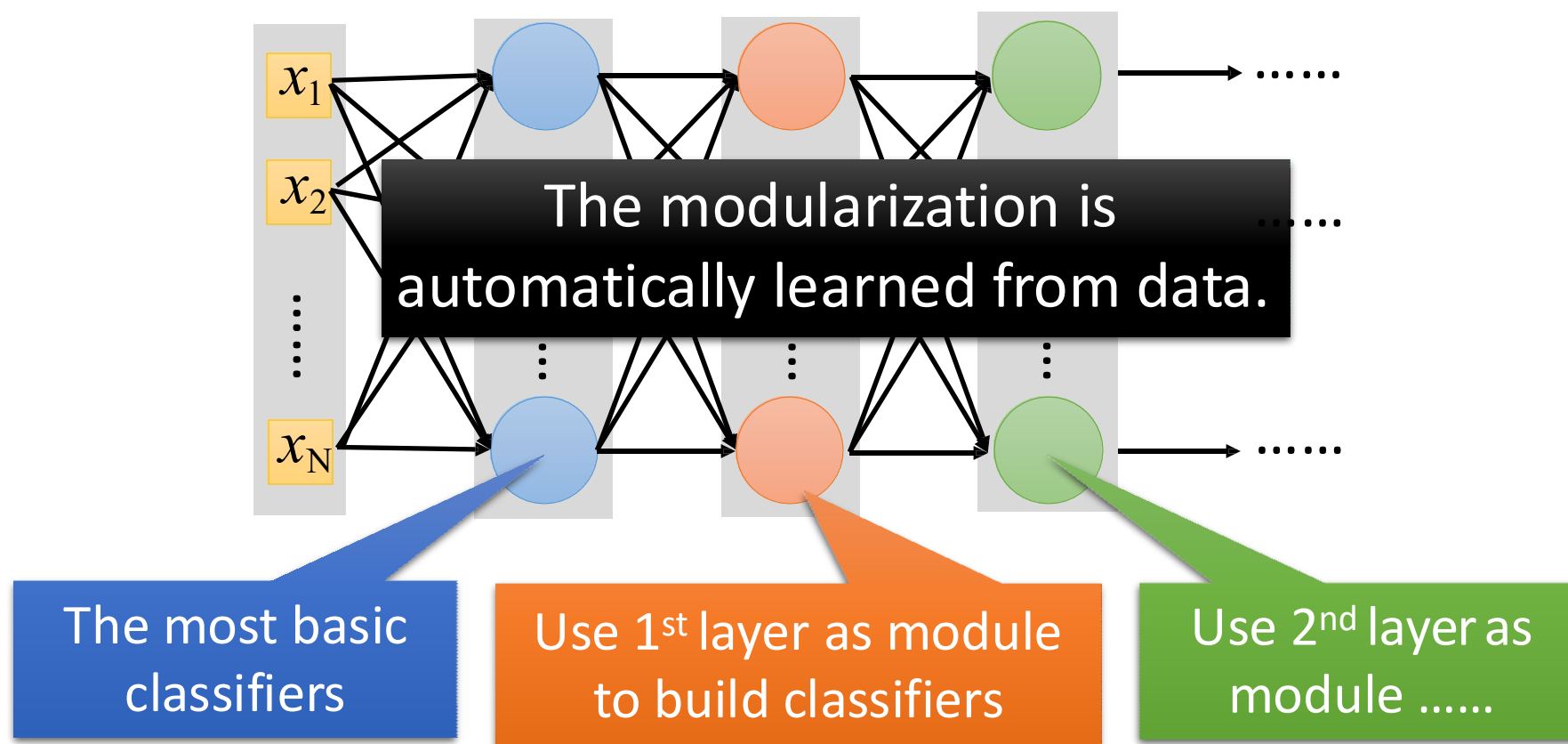
The most basic
classifiers

Use 1st layer as module
to build classifiers

Use 2nd layer as
module

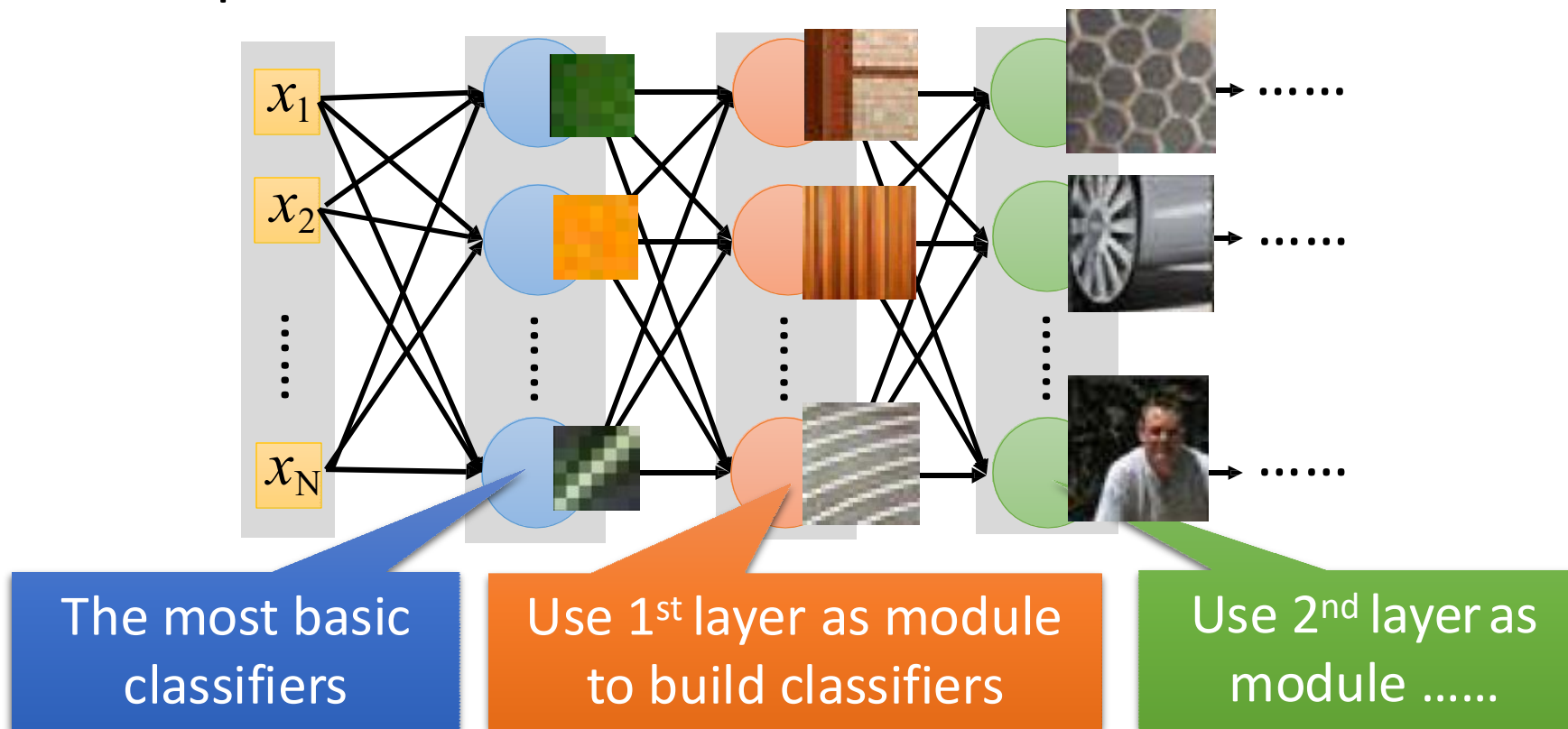
Modularization

- Deep → Modularization → Less training data?



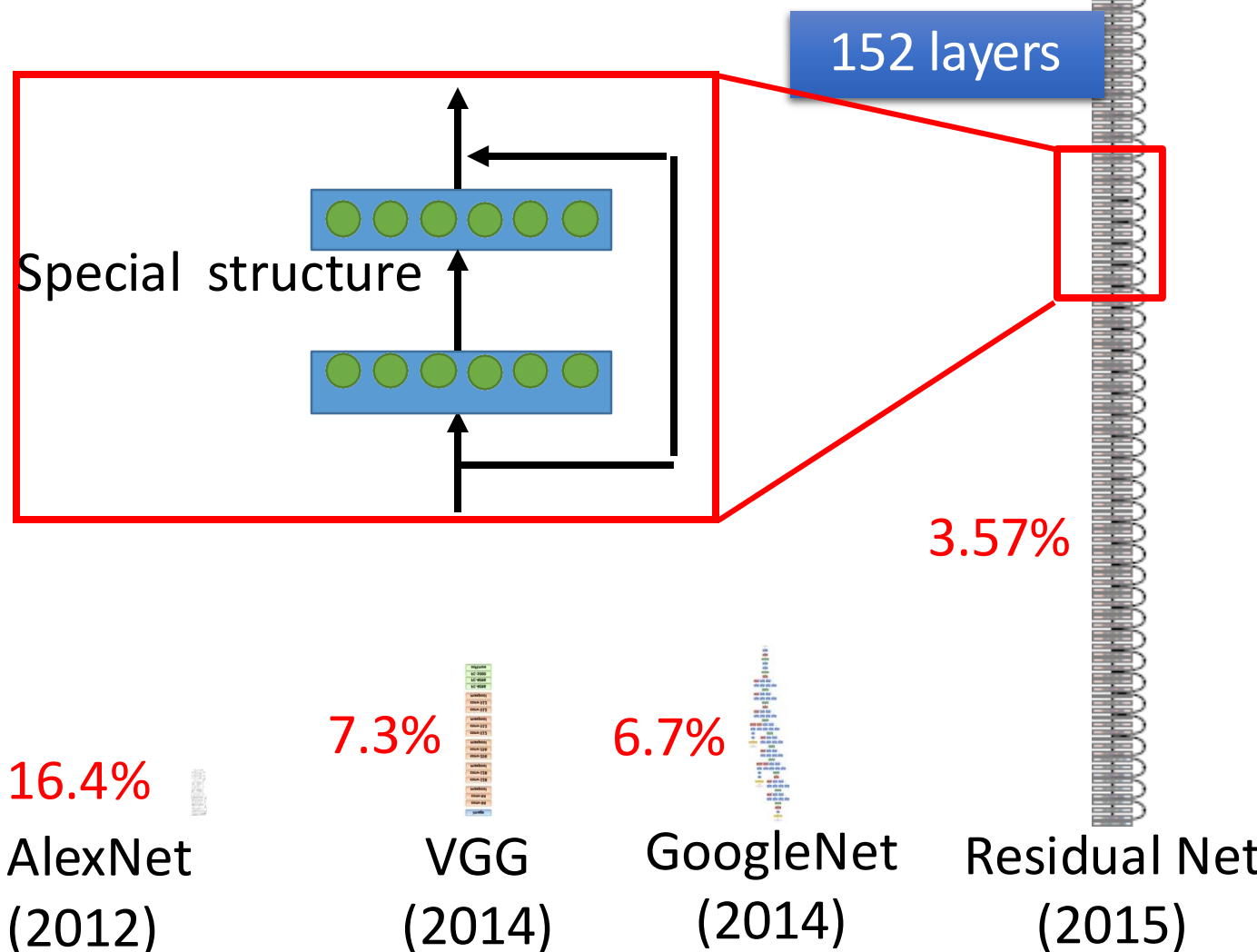
Modularization

- Deep $\rightarrow$ Modularization



Zeiler, Matthew D., and Rob Fergus. "Visualizing and Understanding Convolutional Networks." In Proceedings of the European Conference on Computer Vision, pp. 818-833, 2014

Deep = Many hidden layers



典型深度学习模型

Models

Convolutional Neural Networks(CNN)

Lenet, AlexNet, VGG, Inception, ResNet, SENet

Recurrent Neural Networks (RNN)

RNN, GRU, LSTM, seq2seq

Variational autoencoder(VAE)

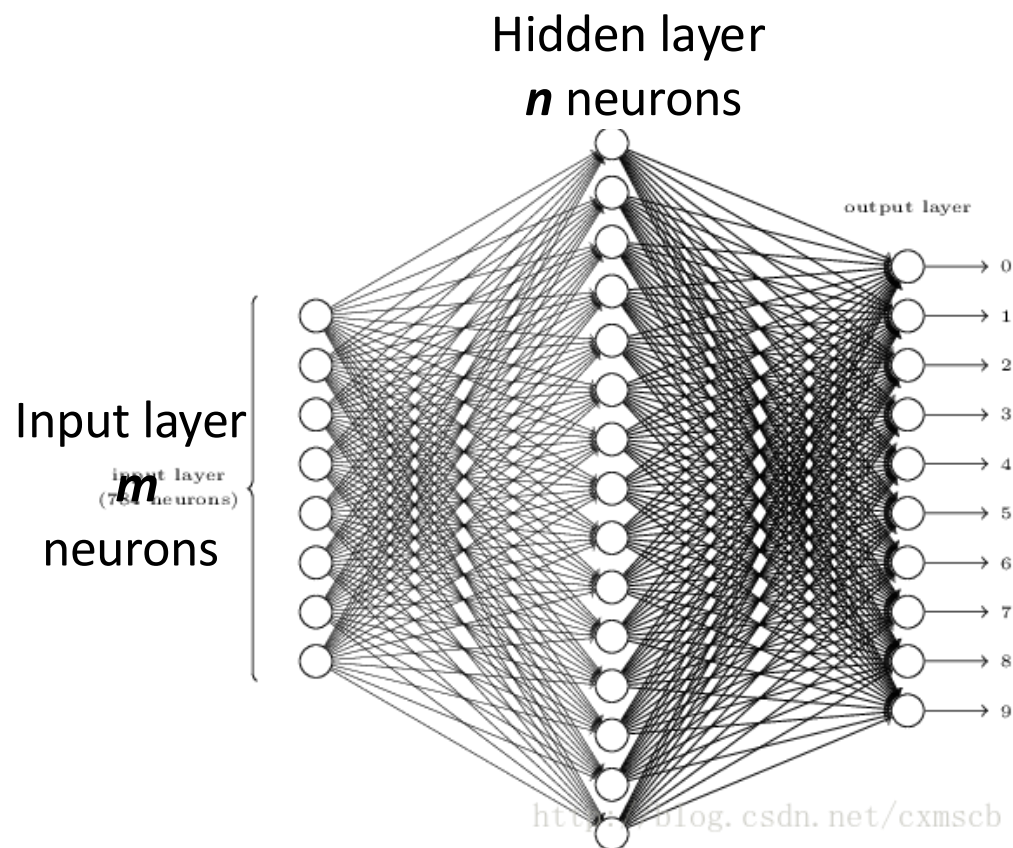
Generative Adversarial Networks (GAN)

Transformer

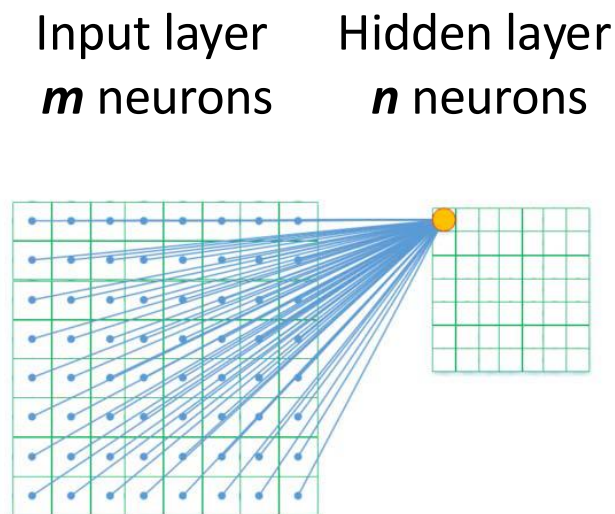
BERT ,GPT

....

Fully Connected Net VS CNN

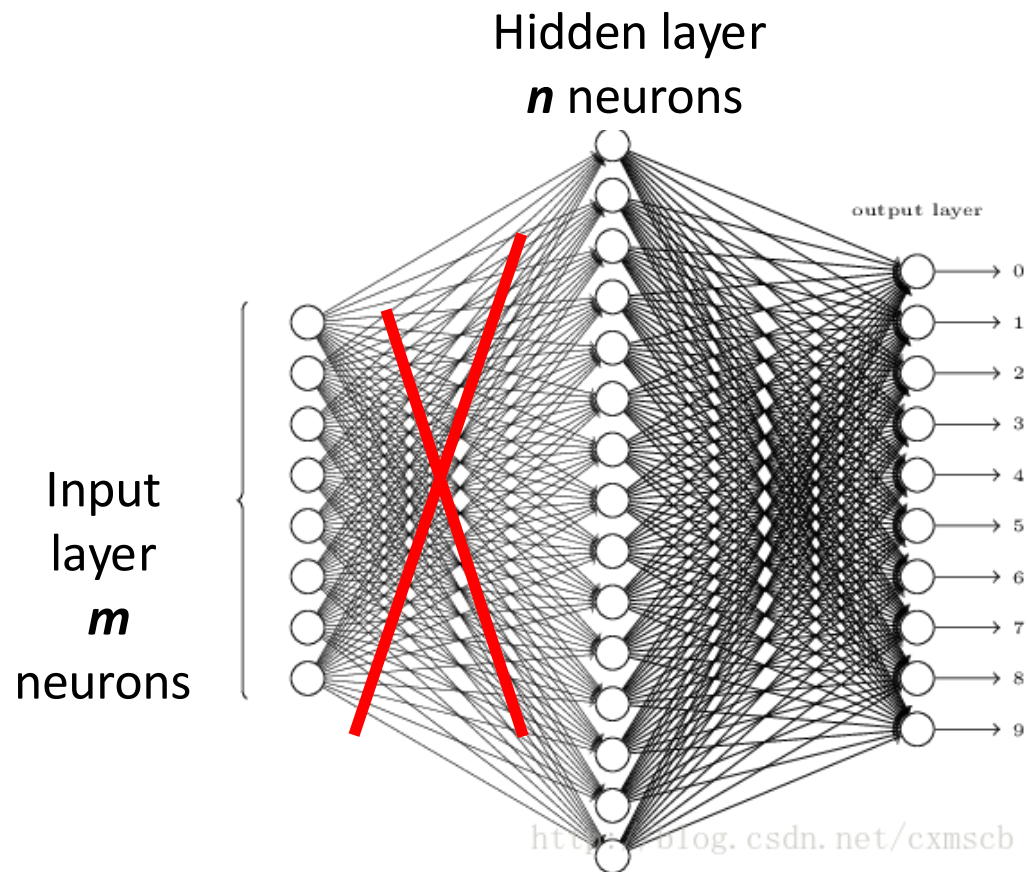


$$m * n \longrightarrow 1000 \times 10000$$



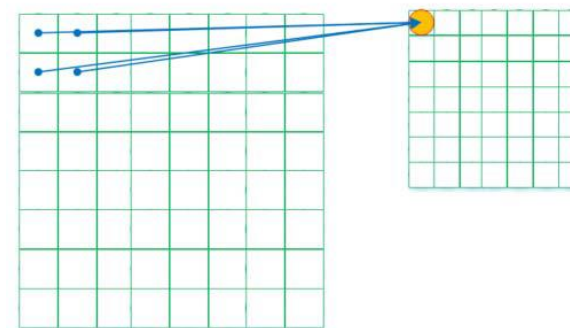
- ▶ 全连接网络
 - ▶ 权重矩阵的参数非常多
- ▶ 卷积神经网络
 - ▶ 生物学上局部感受野

Fully Connected Net VS CNN



Receptive fields
 $k*k$

Hidden layer
 n neurons



► 卷积神经网络有两个结构上的特性：

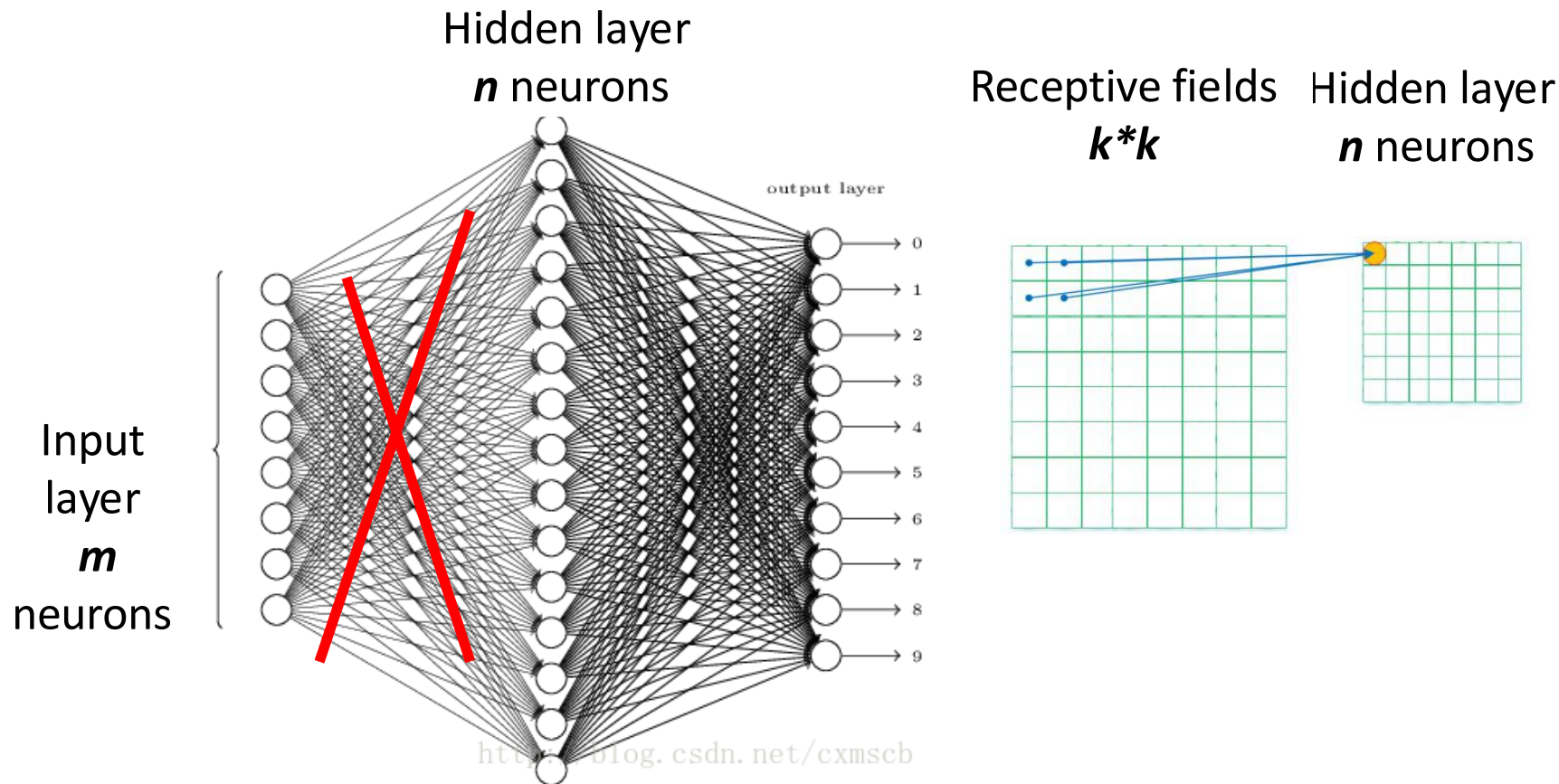
- 局部连接
- 权重共享

Sparse Connectivity

$$m * n \longrightarrow k * k * n$$

1000x10000 10x10x10000

Fully Connected Net VS CNN



$$\begin{array}{ccccc}
 & \text{Sparse Connectivity} & & \text{Shared Weights} & \\
 m*n & \longrightarrow & k*k*n & \longrightarrow & k*k \\
 1000 \times 10000 & & 10 \times 10 \times 10000 & & 10 \times 10
 \end{array}$$

CNN: Sparse Connectivity & Shared Weights

- **Sparse connectivity** by local correlation
 - Filter: the input of a hidden unit in layer m are from a subset of units in layer $m-1$ that have spatially connected **receptive fields**
- **Shared weights**
 - each filter is replicated across the entire visual field. These replicated units share the same weights and form a feature map.

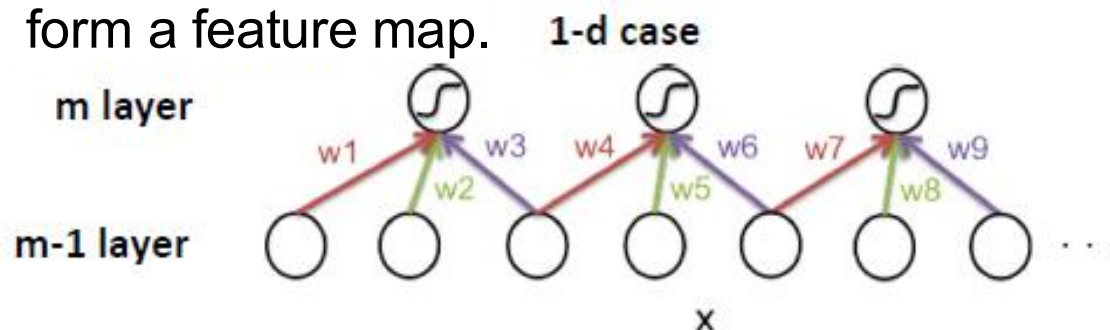
2-d case (subscripts are weights)

1 _{x1}	1 _{x0}	1 _{x1}	0	0
0 _{x0}	1 _{x1}	1 _{x0}	1	0
0 _{x1}	0 _{x0}	1 _{x1}	1	1
0	0	1	1	0
0	1	1	0	0

Image

4		

Convolved Feature



edges that have the same color have the same weight <http://deeplearning.net/tutorial/lenet.html>

CNN: Propose & Improve

- LeNet-5

- LeCun, Bottou, Bengio and Haffner in 1998

sparse connectivity and parameter sharing and equivariant representation

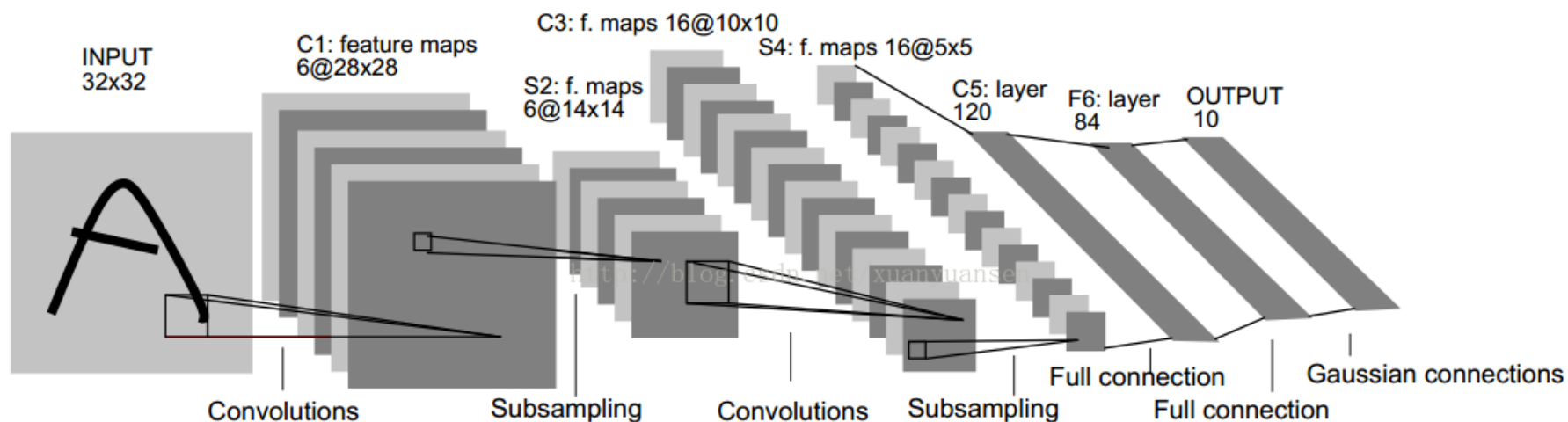
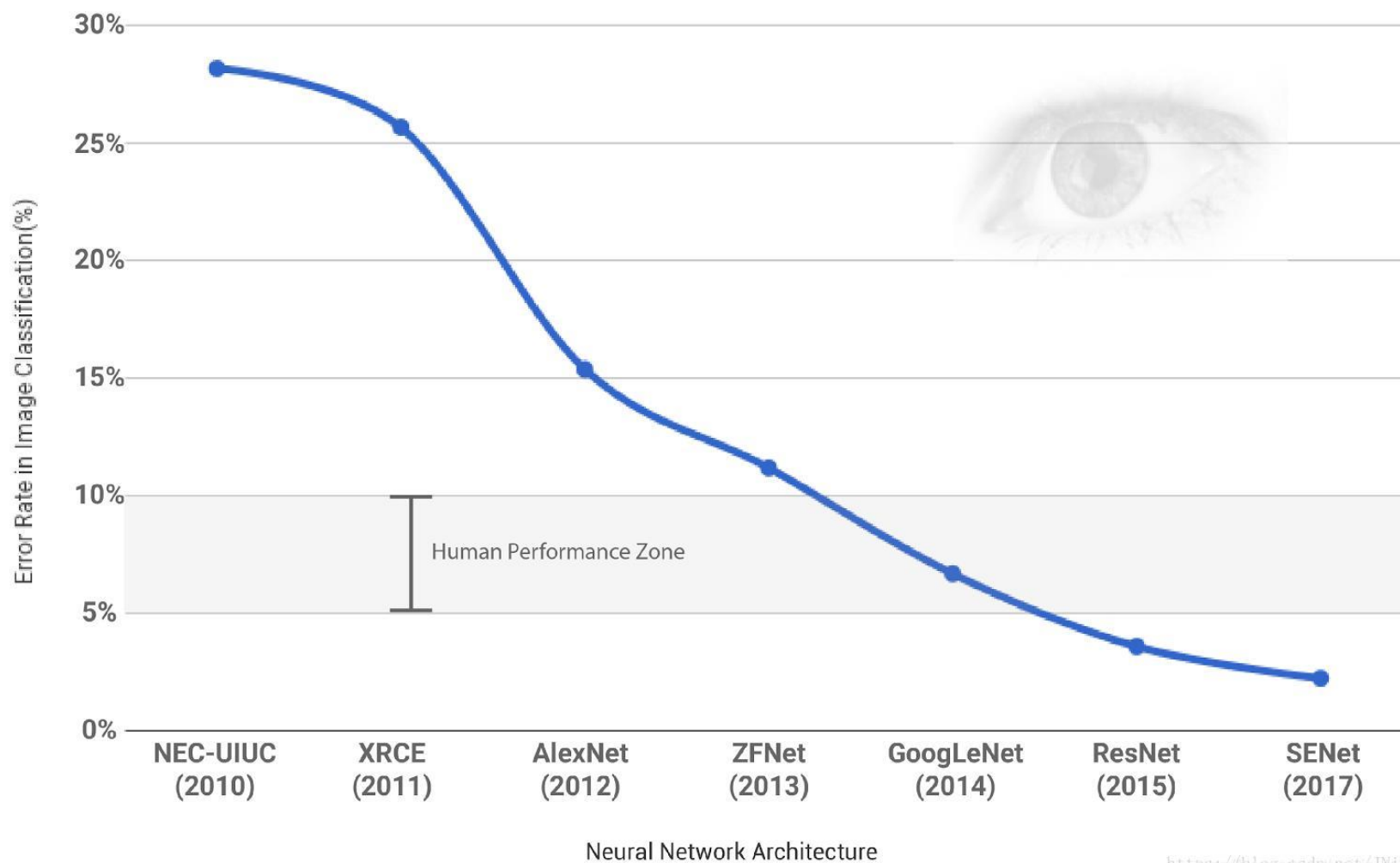


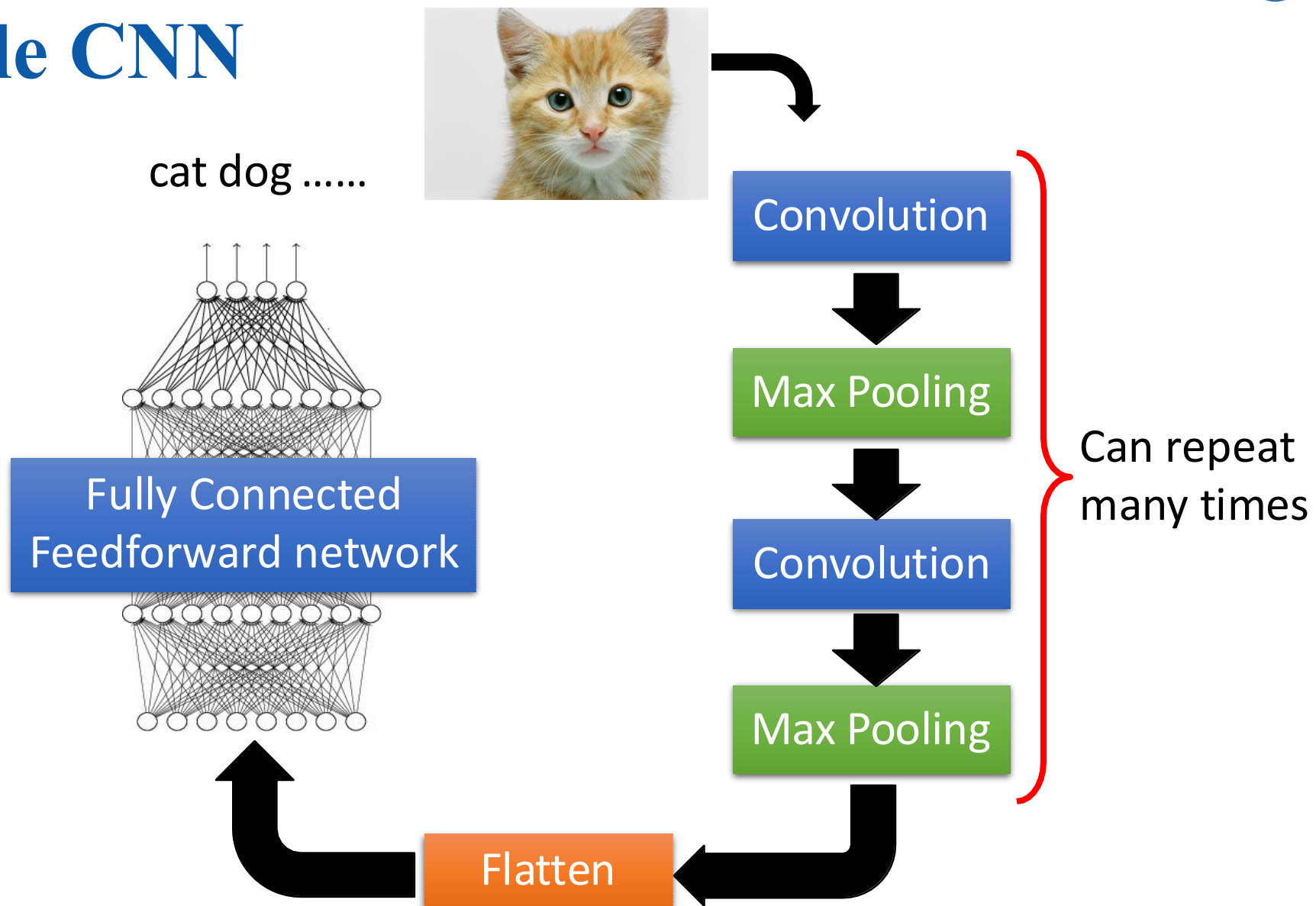
Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

[Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. "Gradient-based learning applied to document recognition." Proceedings of the IEEE, 86(11) 1998]

Leadertable (ImageNet image classification)



The whole CNN



Why CNN for Image?

- Some patterns are much smaller than the whole image

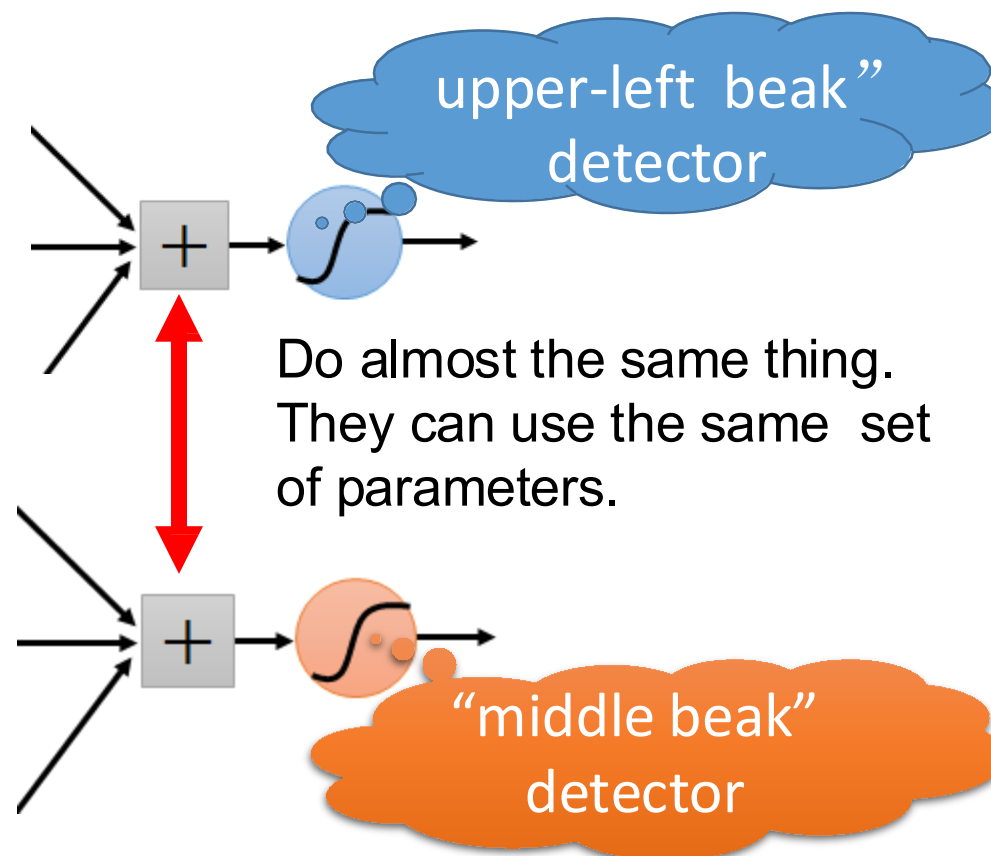
A neuron does not have to see the whole image to discover the pattern.

Connecting to small region with less parameters



Why CNN for Image?

- The same patterns appear in different regions.



Why CNN for Image?

- Subsampling the pixels will not change the object

bird



subsampling

bird

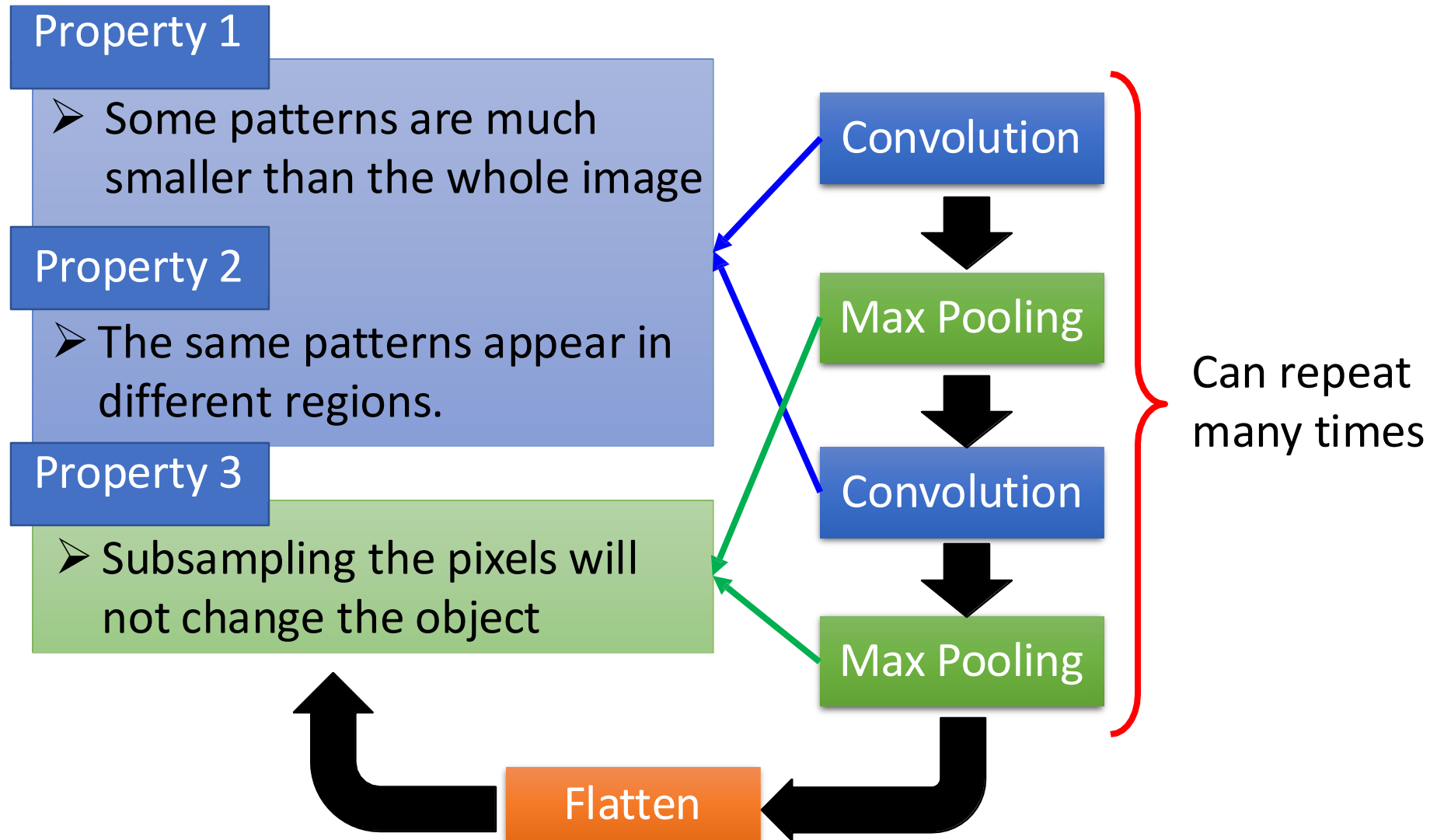


We can subsample the pixels to make image smaller



Less parameters for the network to process the image

The whole CNN



CNN – Convolution

Those are the network parameters to be learned.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1
Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2
Matrix

⋮

Property 1

Each filter detects a small pattern (3 x 3).

CNN – Convolution

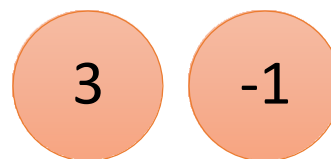
stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

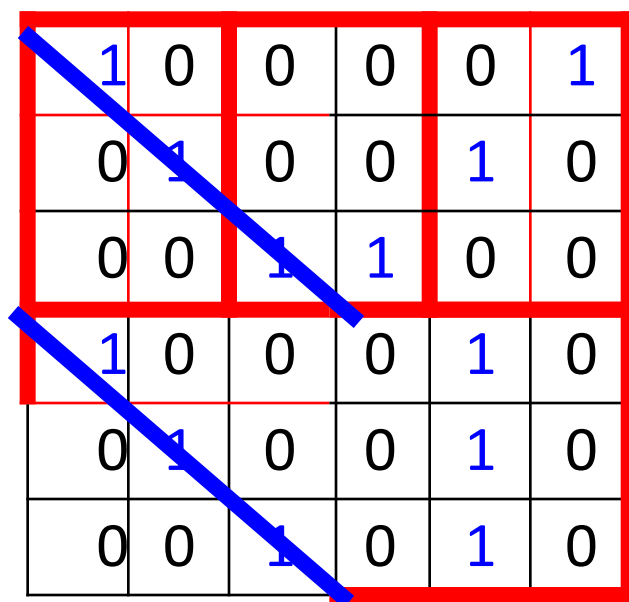
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

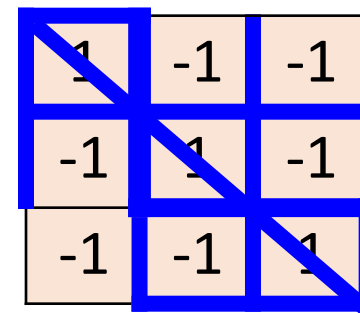


CNN – Convolution

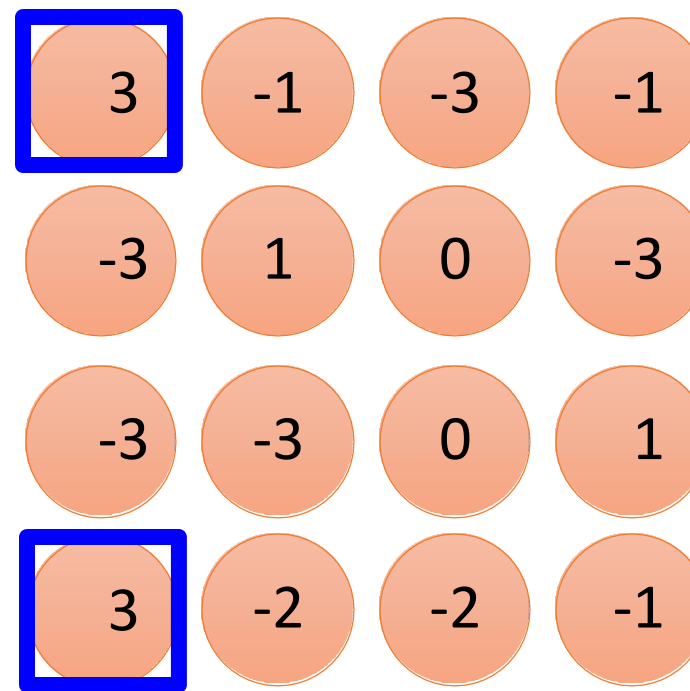
stride=1



6 x 6 image



Filter 1



Property 2

CNN – Convolution

stride=1

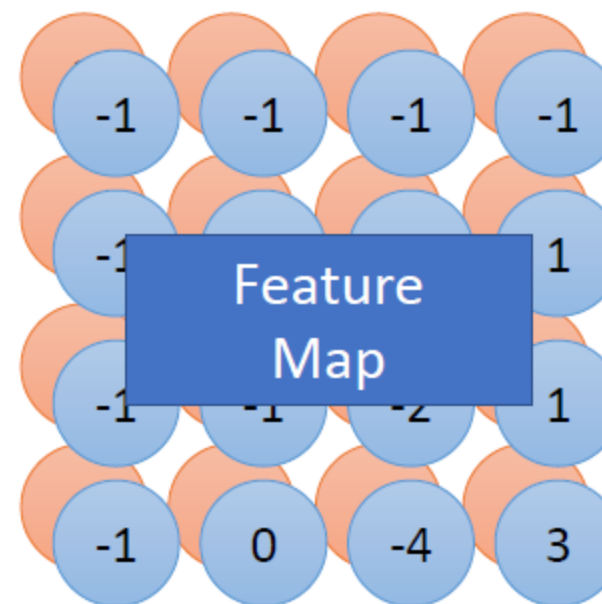
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Do the same process for
every filter

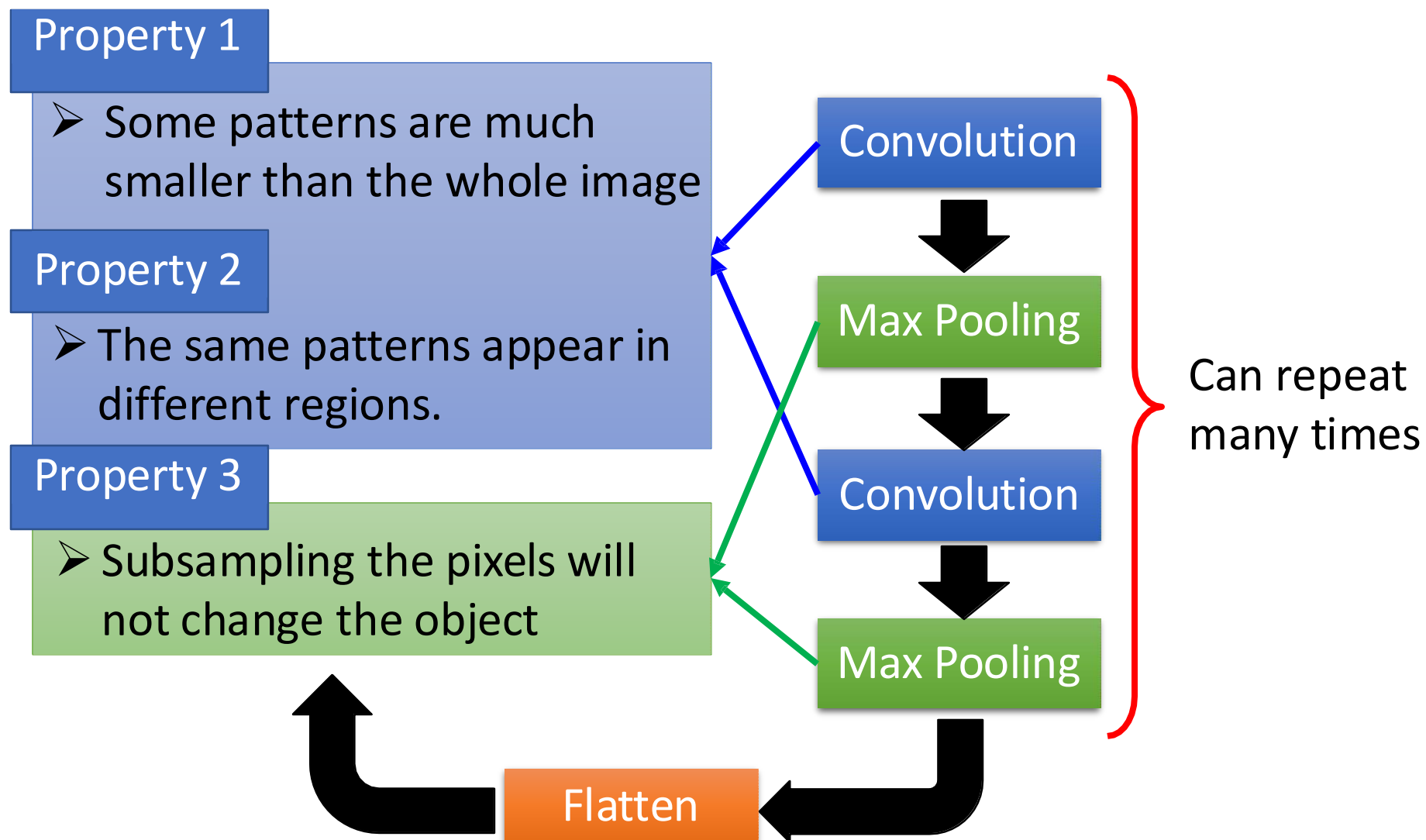
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

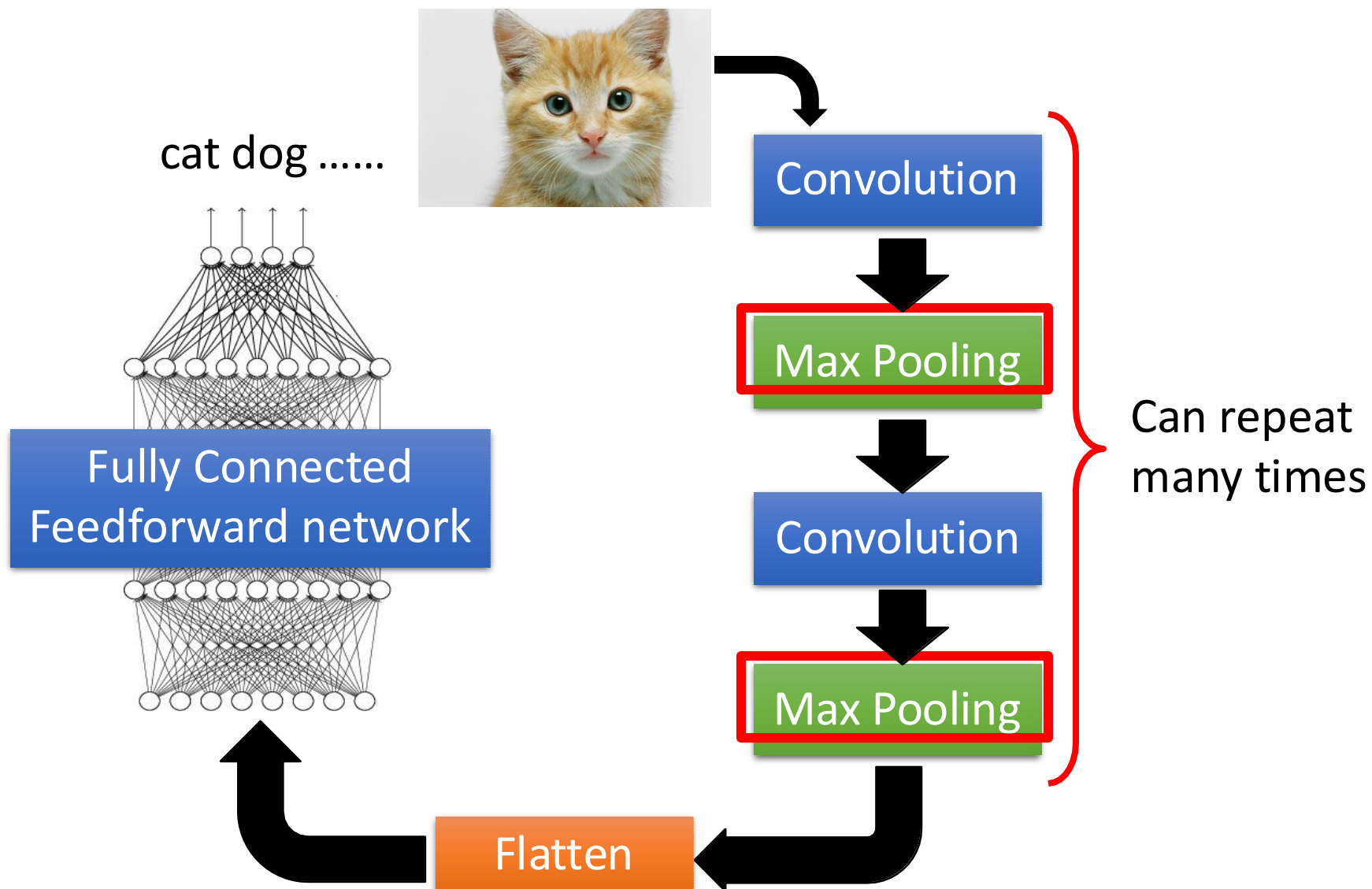


4 x 4 image

The whole CNN



The whole CNN



CNN – Max Pooling

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

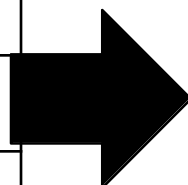
3	-1	-3	-1
-3	1	0	-3
-3	-3	0	1
3	-2	-2	-1

-1	-1	-1	-1
-1	-1	-2	1
-1	-1	-2	1
-1	0	-4	3

CNN – Max Pooling

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

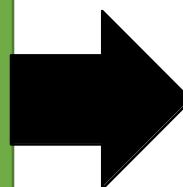
6 x 6 image



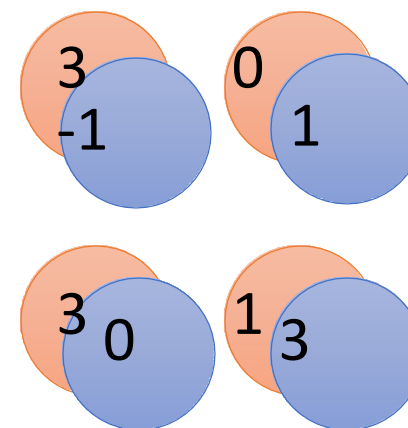
Conv



Max
Pooling



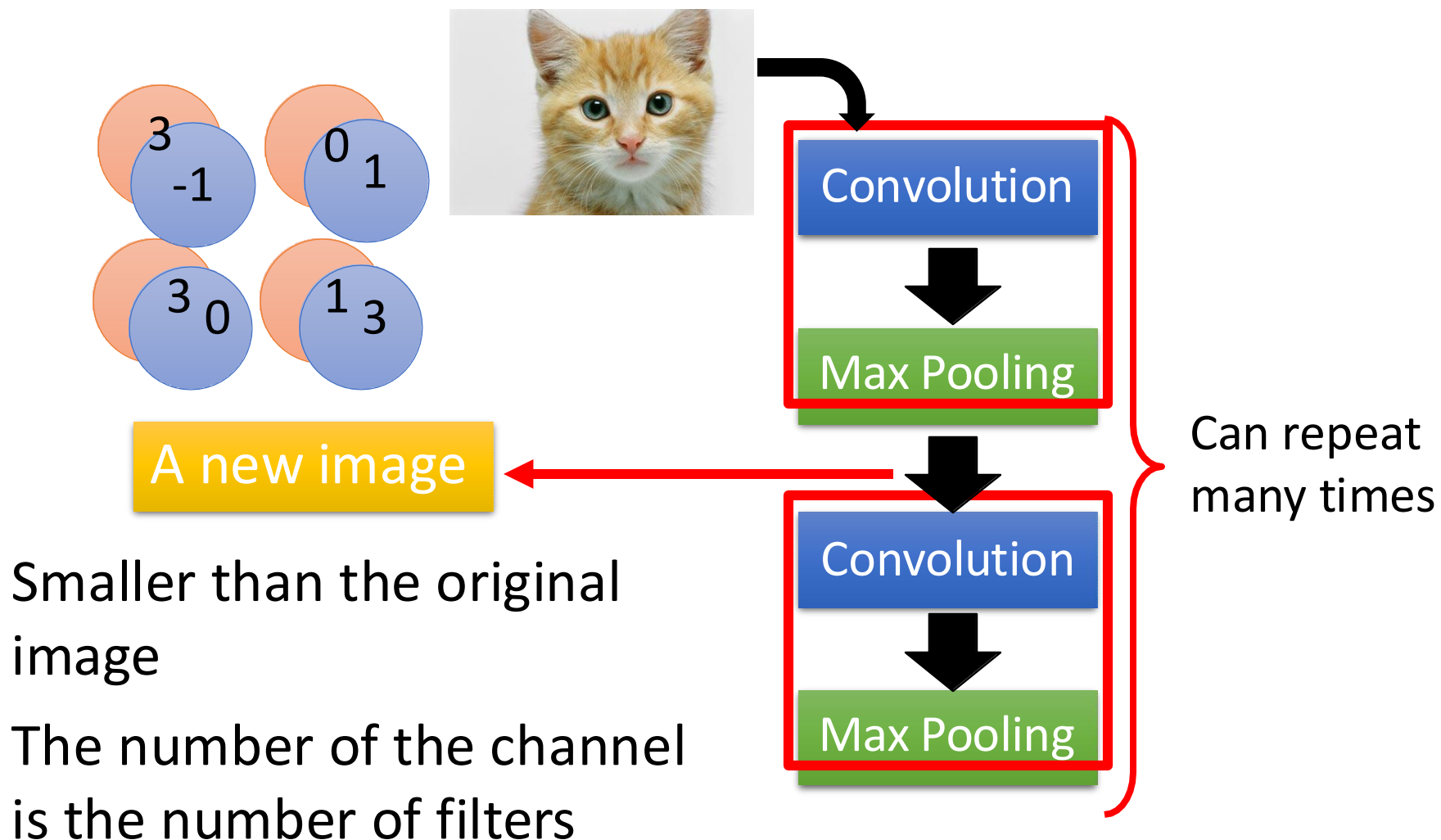
New image
but smaller



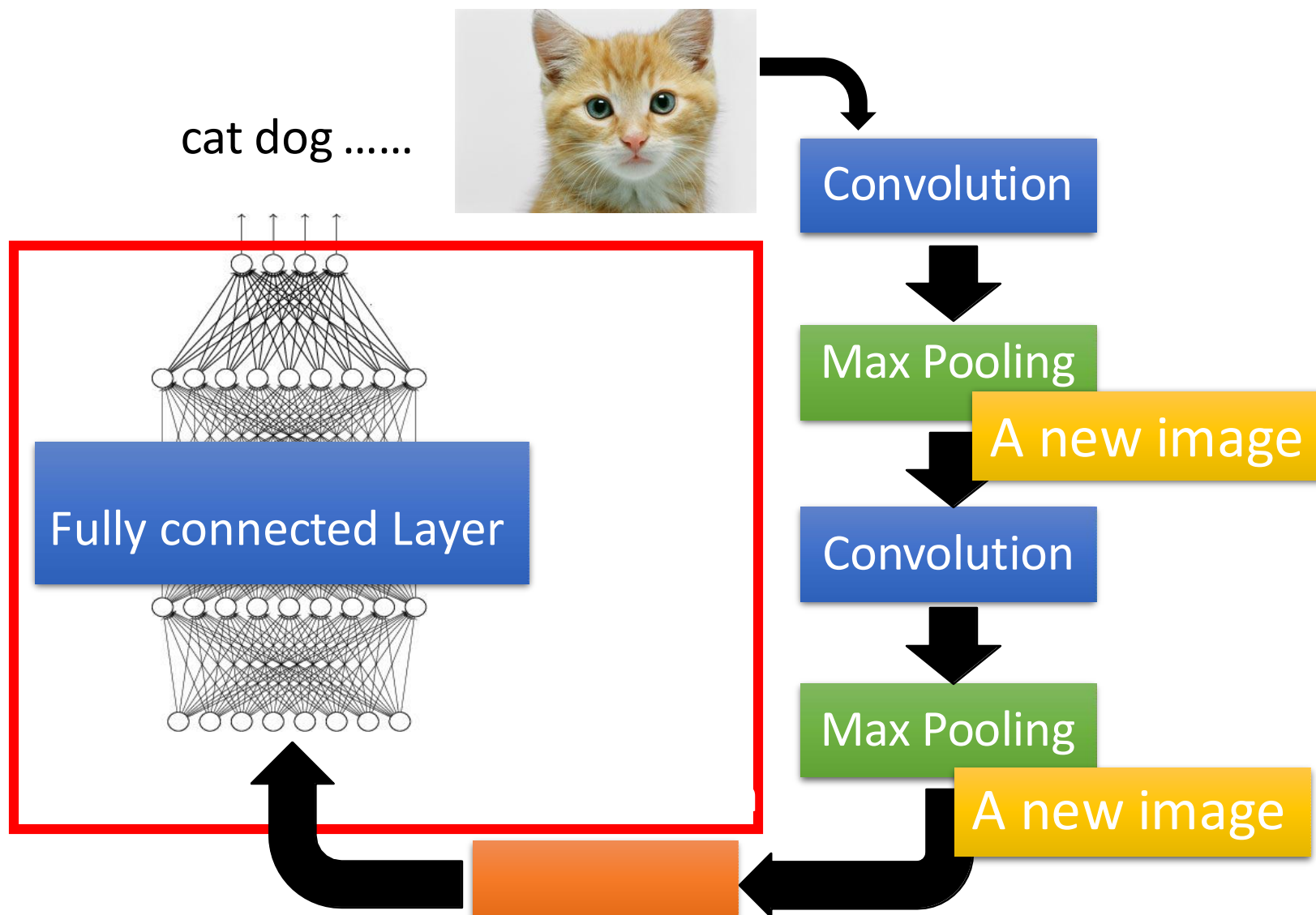
2 x 2 image

Each filter
is a channel

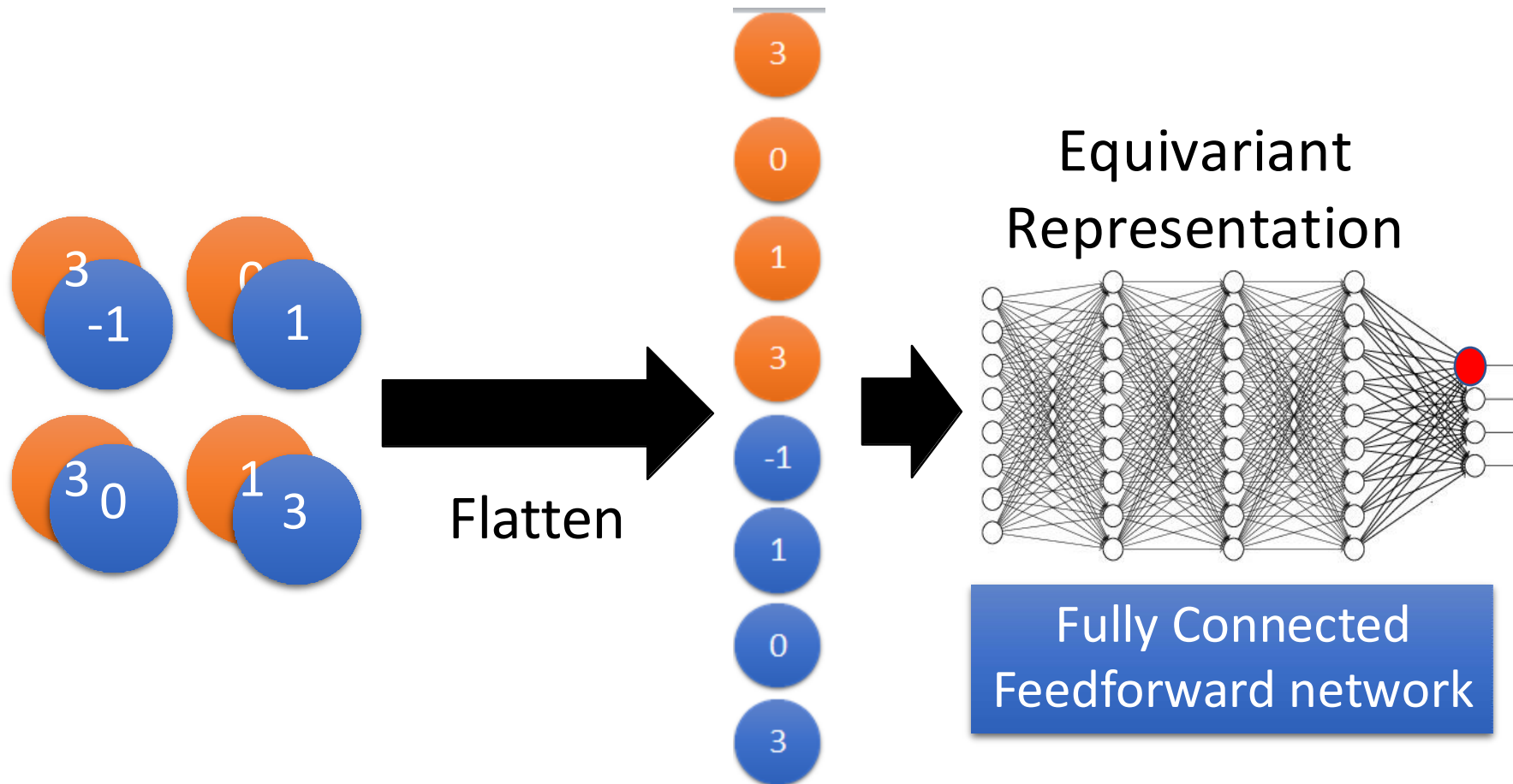
The whole CNN



The whole CNN



CNN – Fully Connected Layer



怎么去做“深度学习”？

监督学习 Supervised Learning

- Given the training dataset of (data,label) pairs,

$$D = \{(x^{(i)}, y^{(i)})\}_{i=1,2,\dots,N}$$

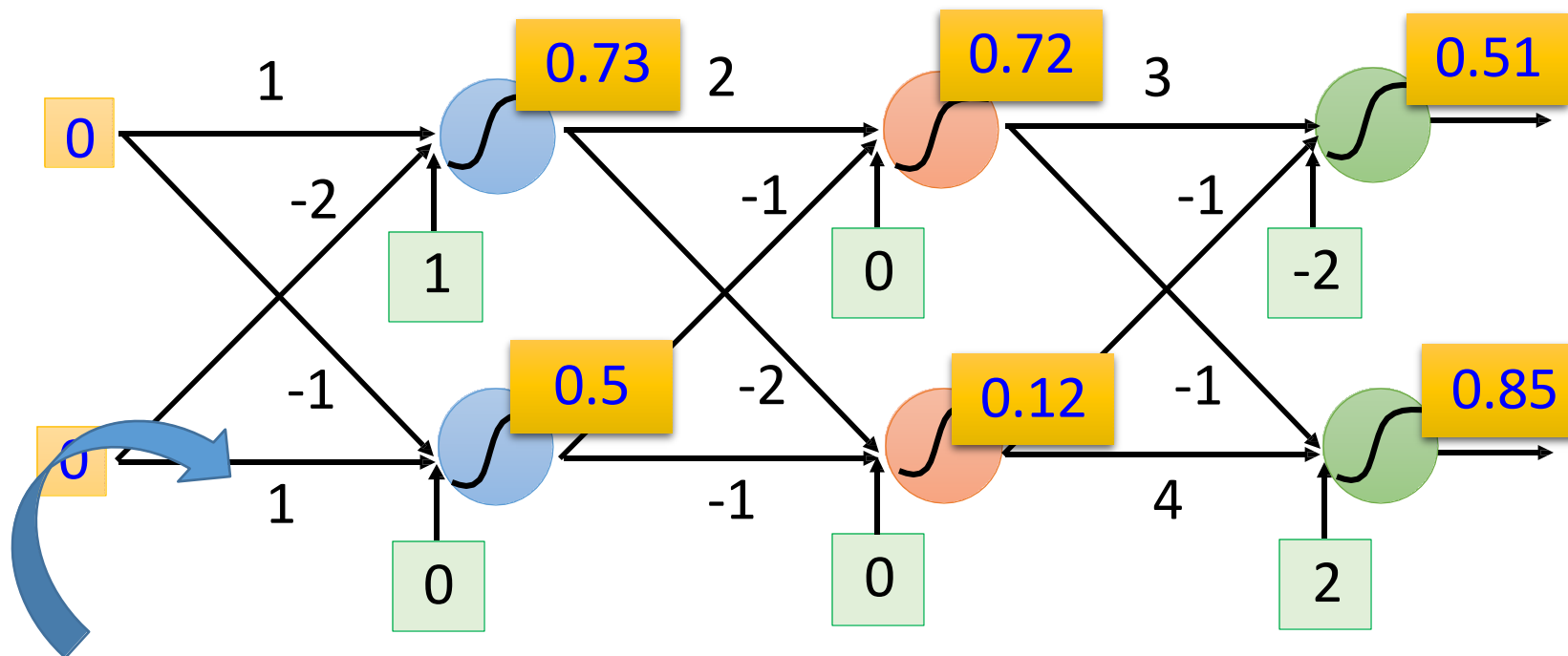
$x^{(i)}$ = input data(features) of i^{th} training example
 $y^{(i)}$ = output data(label) of i^{th} training example

let the machine learn a function from data to label

$$y^{(i)} \approx f_{\theta}(x^{(i)})$$

- Function set $\{f_{\theta}(x^{(i)})\}$ is called hypothesis space
- Learning is referred to as updating the parameter θ to make the prediction closed to the corresponding label

Fully Connect Feedforward Network



This is a function.

Input vector, output vector

$$f\left(\begin{bmatrix} 1 \\ -1 \end{bmatrix}\right) = \begin{bmatrix} 0.62 \\ 0.83 \end{bmatrix} \quad f\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.51 \\ 0.85 \end{bmatrix}$$

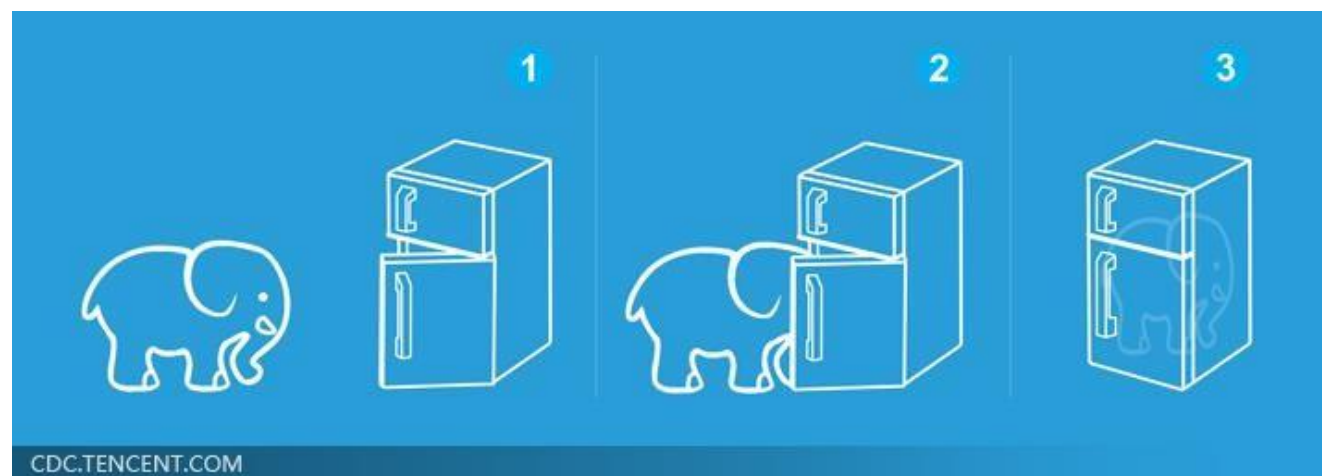
Given network structure, define a function set

Different w and bias

Three Steps for Deep Learning



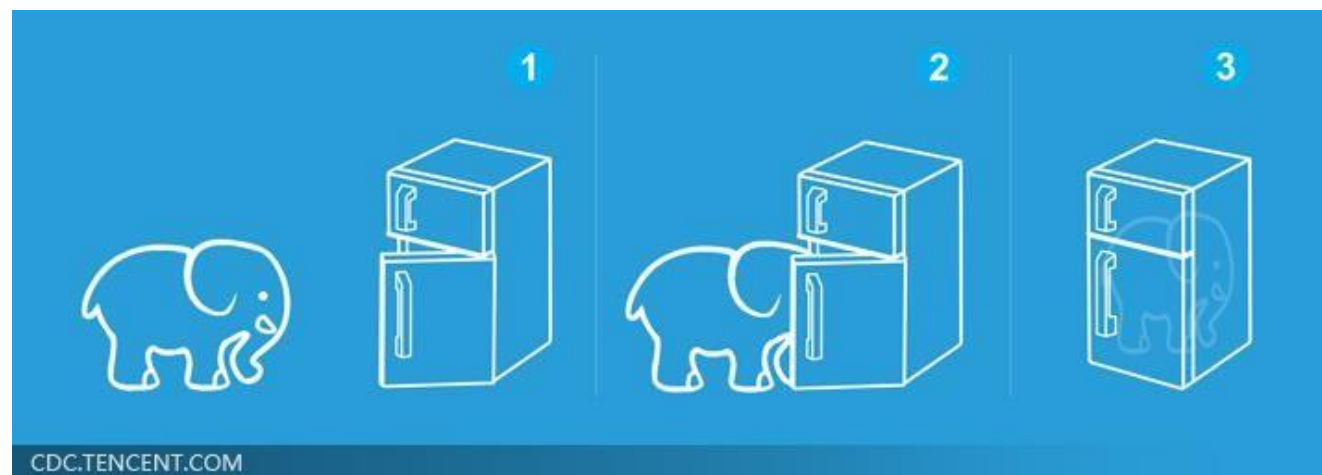
Deep Learning is so simple



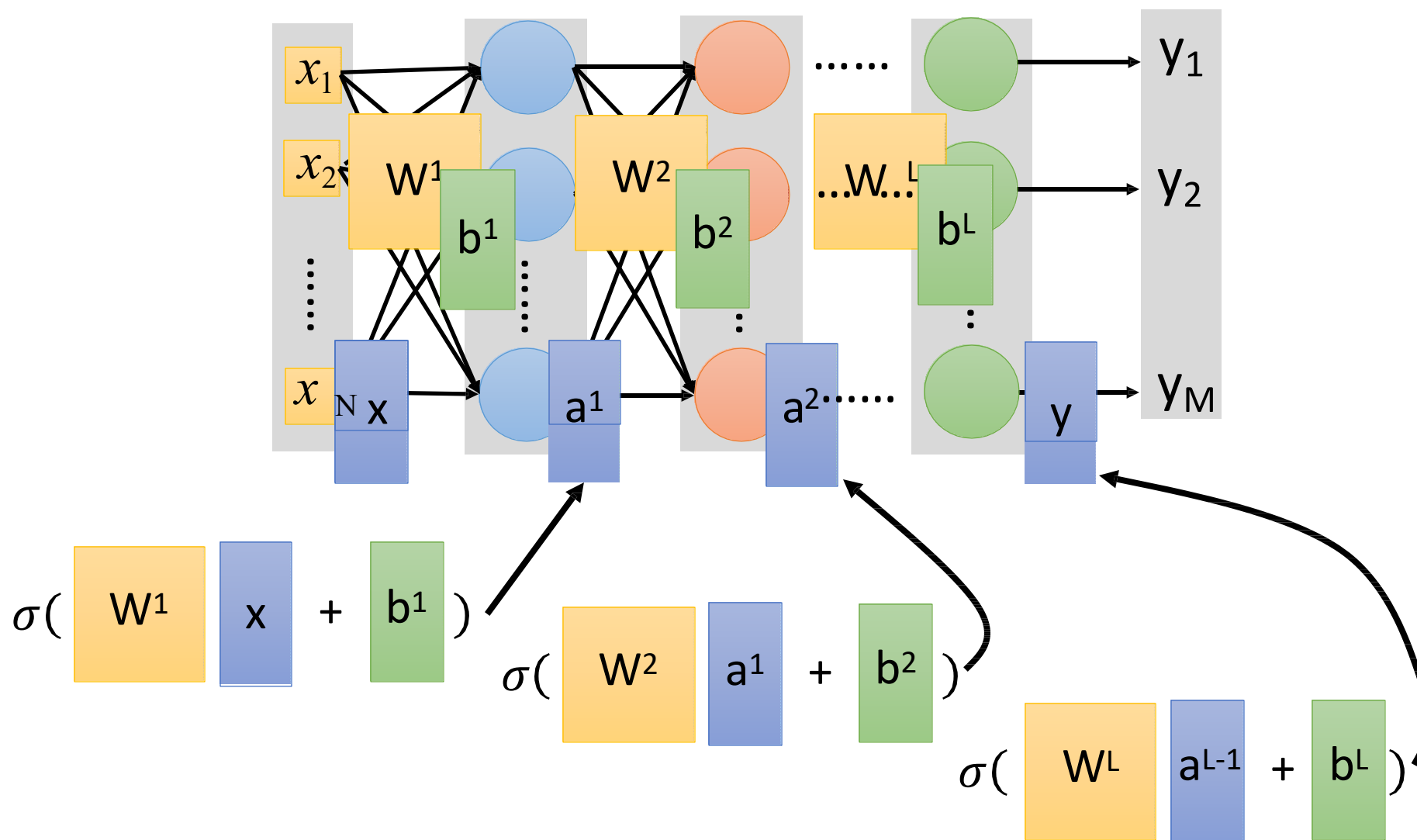
Three Steps for Deep Learning



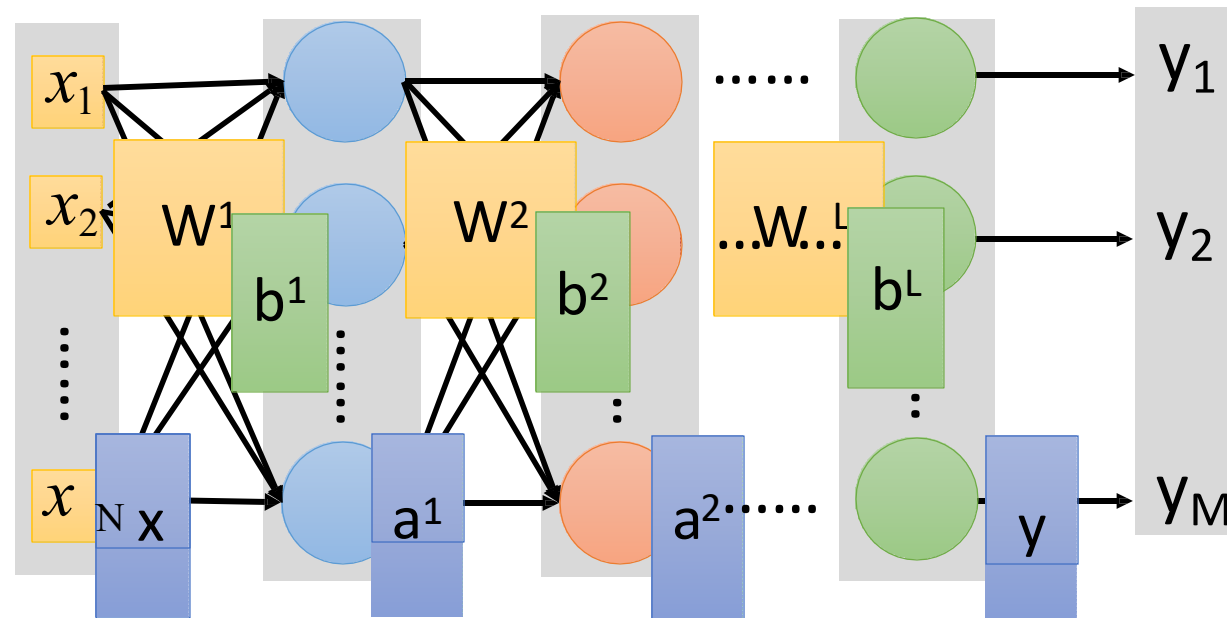
Deep Learning is so simple



Neural Network



Neural Network

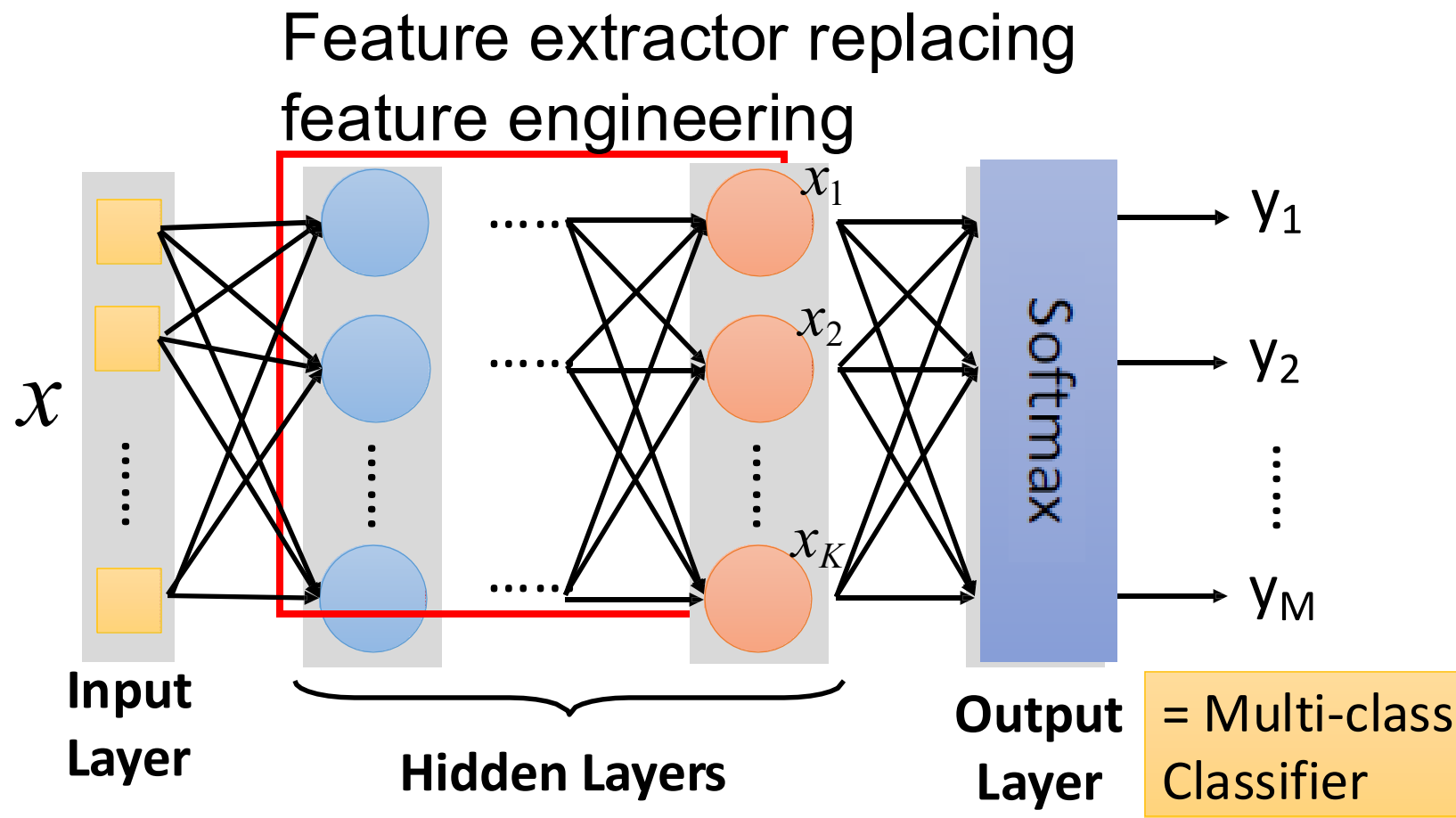


$$y = f(x)$$

Using parallel computing techniques to speed up matrix operation

$$= \sigma(W^L \dots \sigma(W^2 \sigma(W^1 x + b^1) + b^2) \dots + b^L)$$

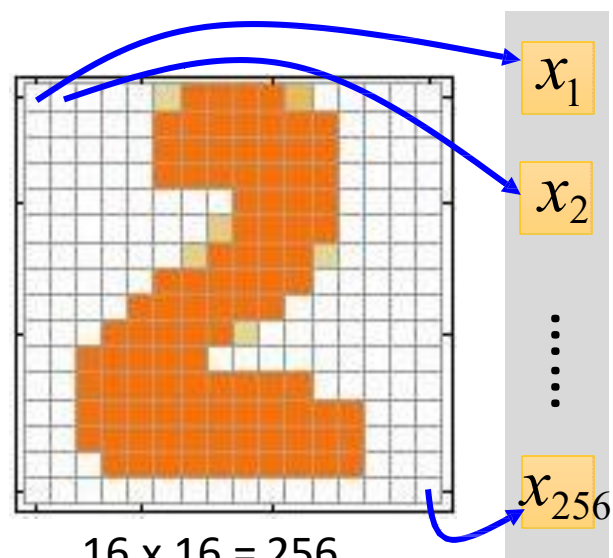
Output Layer



Example Application



Input

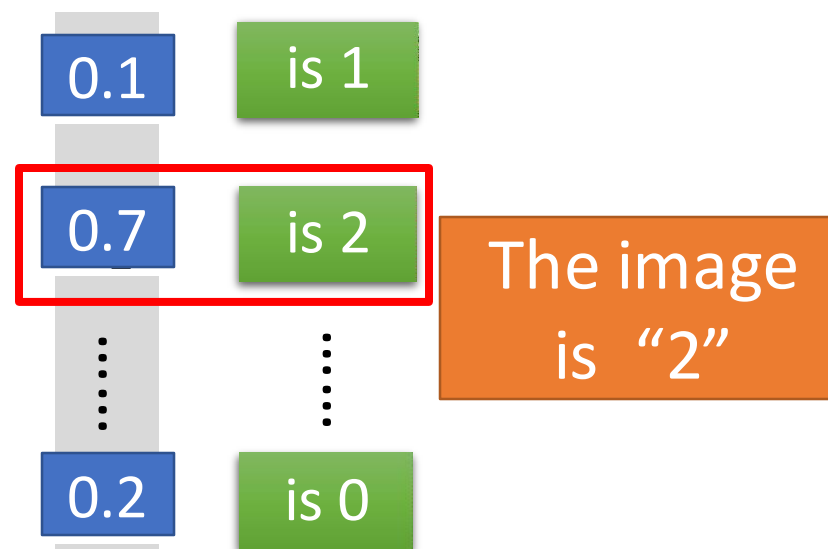


$16 \times 16 = 256$

Ink $\rightarrow 1$

No ink $\rightarrow 0$

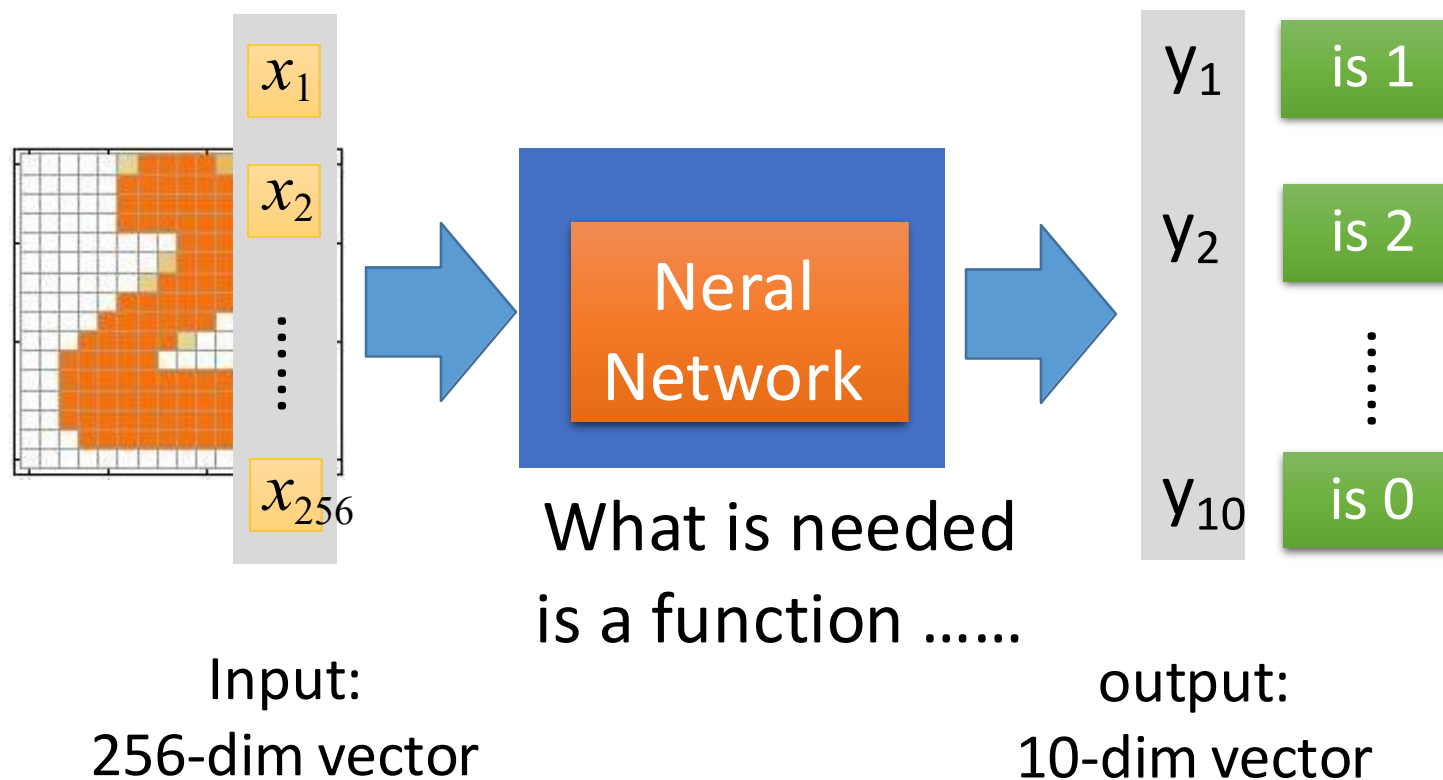
Output



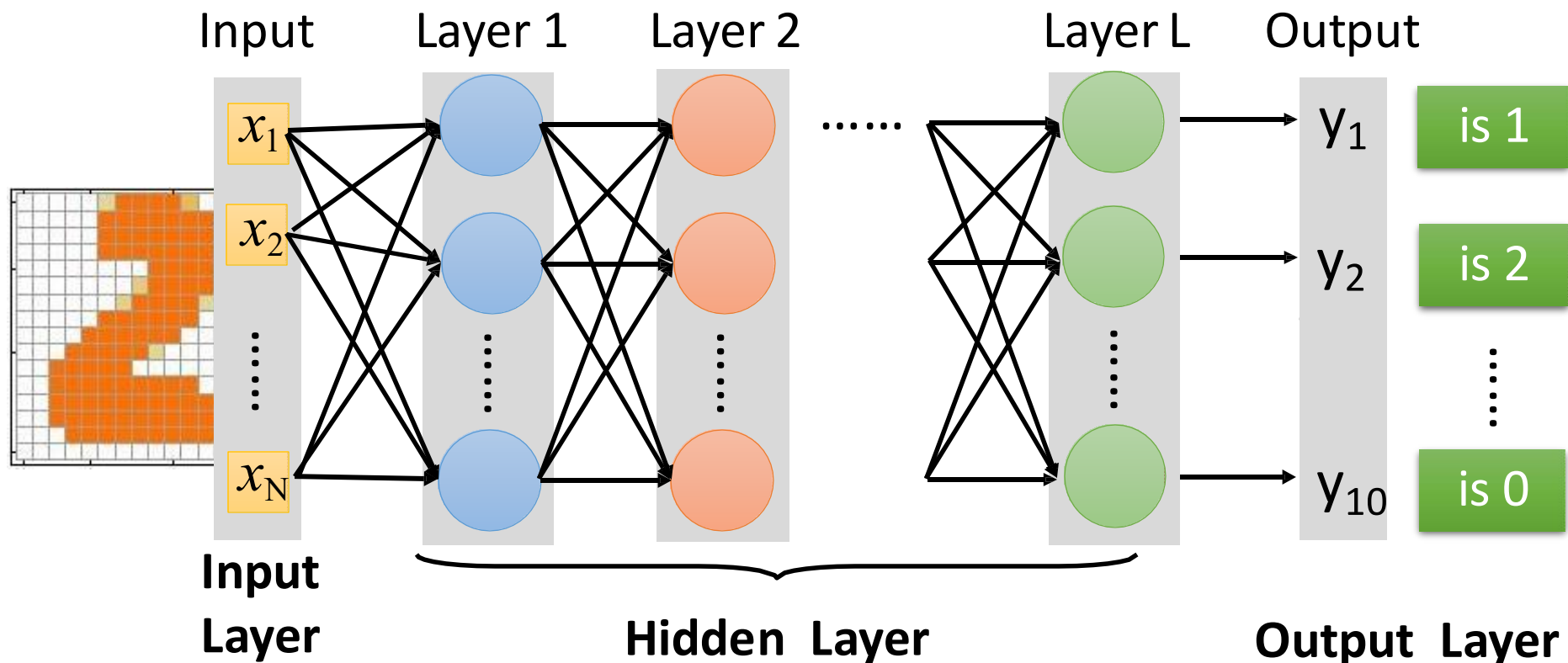
Each dimension represents the confidence of a digit.

Example Application

- Handwriting Digit Recognition

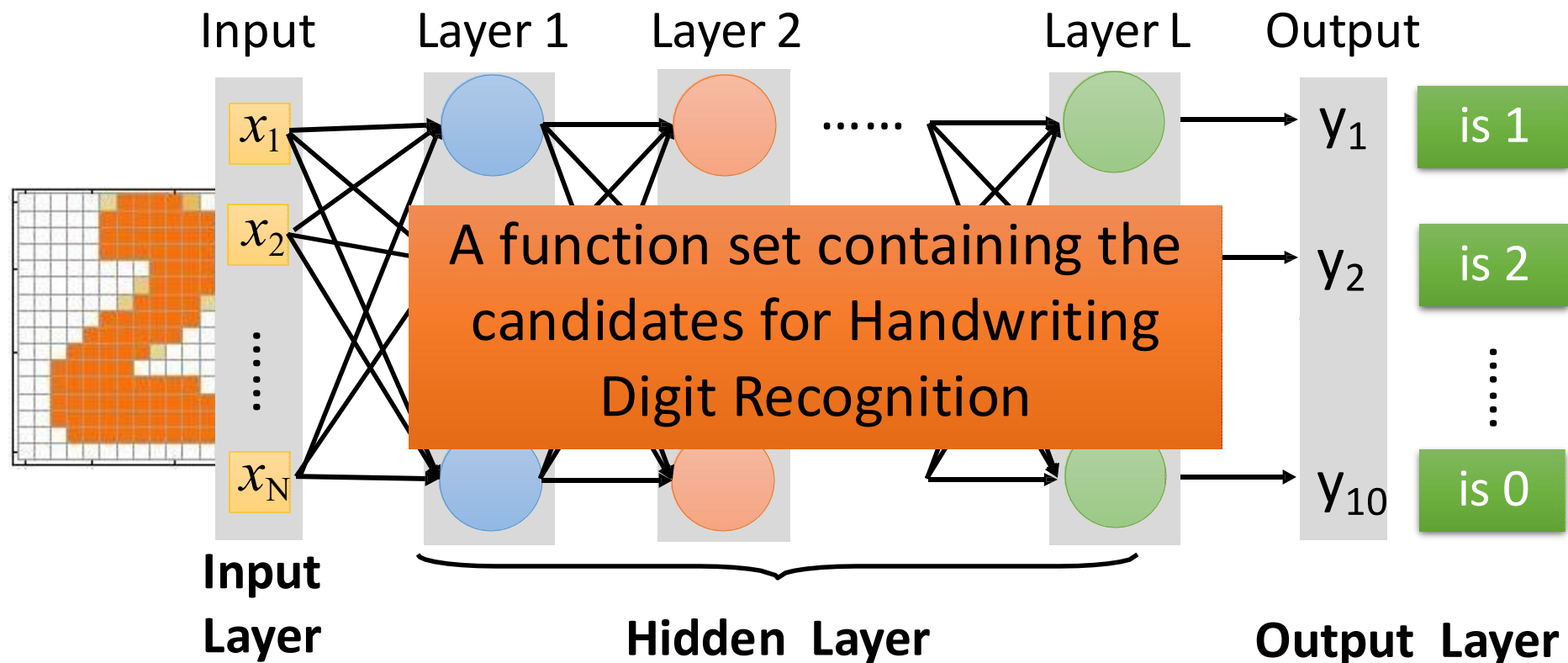


Example Application

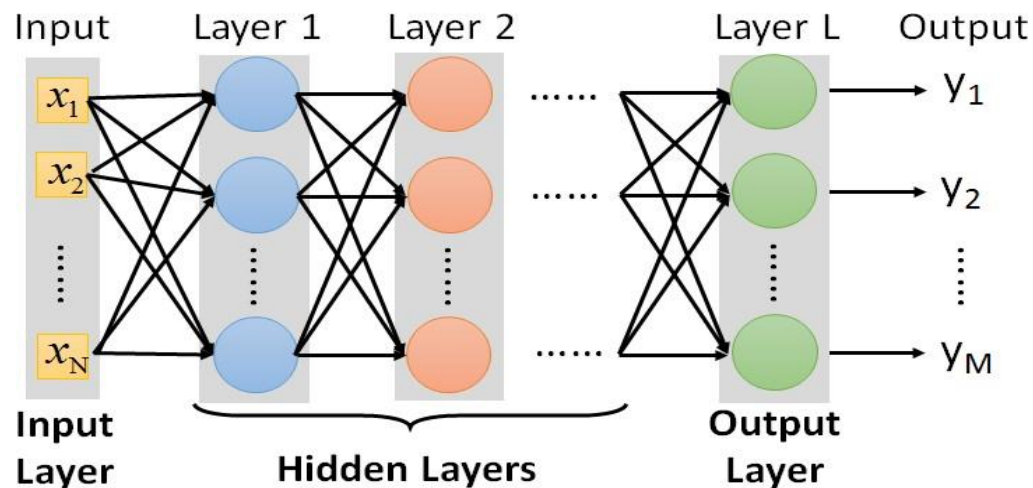


You need to decide the network structure to let a good function in your function set.

Example Application



You need to decide the network structure to let a good function in your function set.



- Q: How many layers? How many neurons for each layer?

Trial and Error

+

Intuition

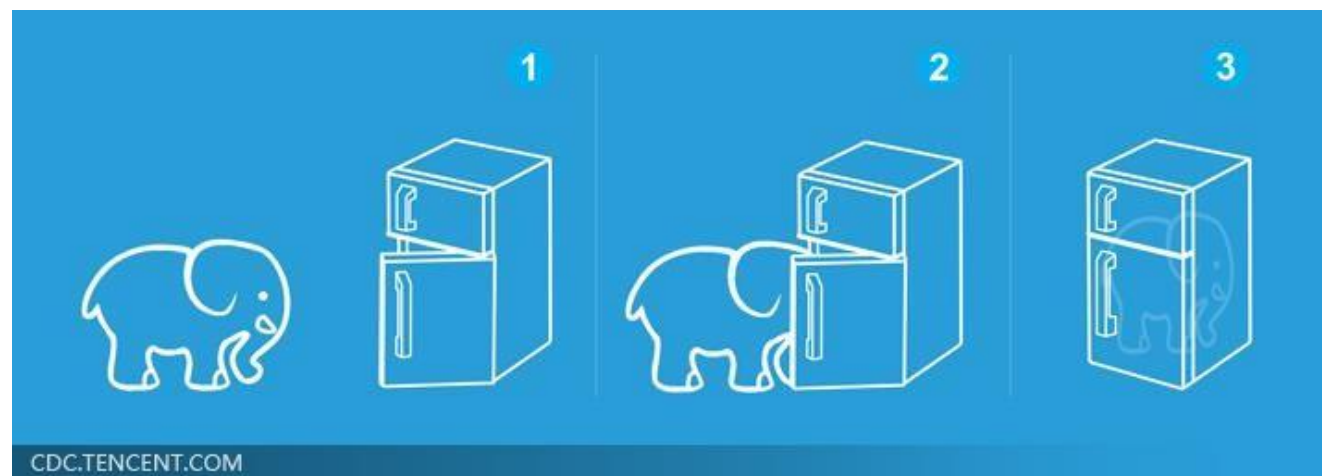
- Q: Can the structure be automatically determined?
 - E.g. Evolutionary Artificial Neural Networks
- Q: Can we design the network structure?

Convolutional Neural Network (CNN)

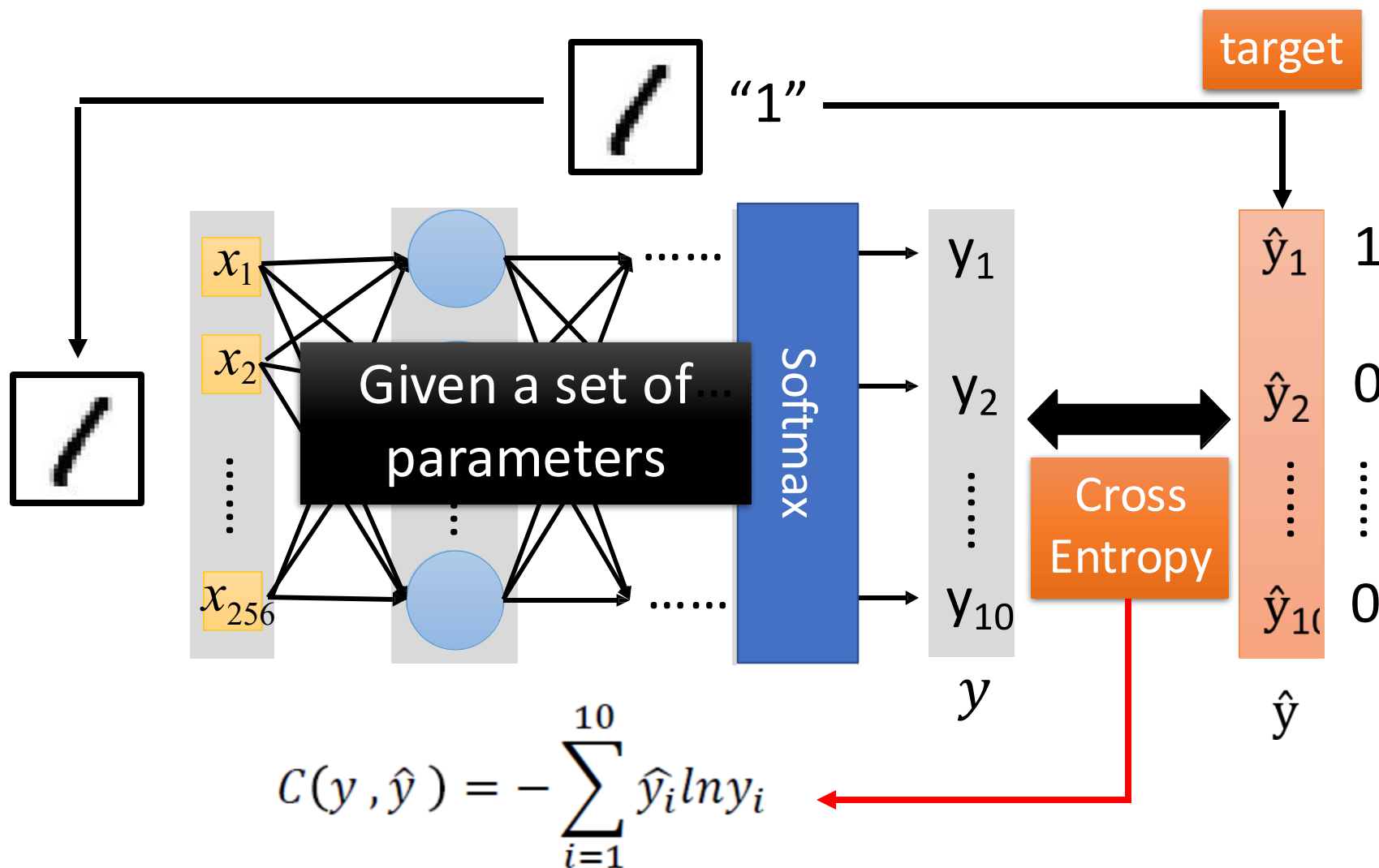
Three Steps for Deep Learning



Deep Learning is so simple

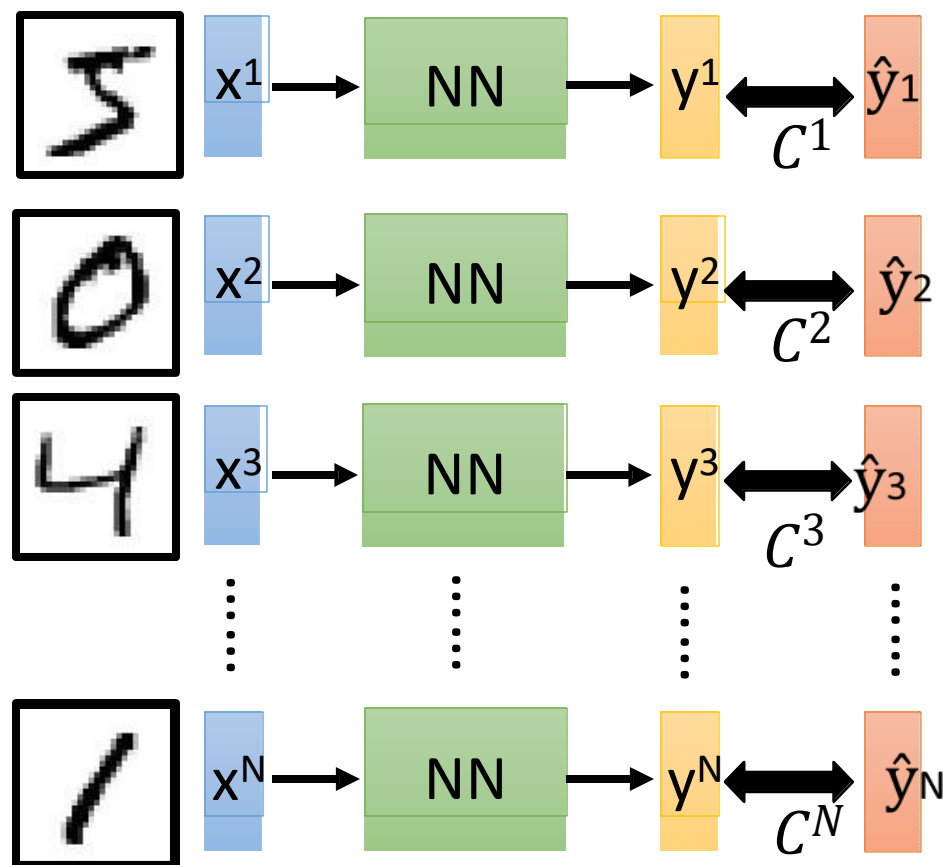


Loss for an Example



Total Loss

For all training data ...



Total Loss:

$$L = \sum_{n=1}^N C^n$$

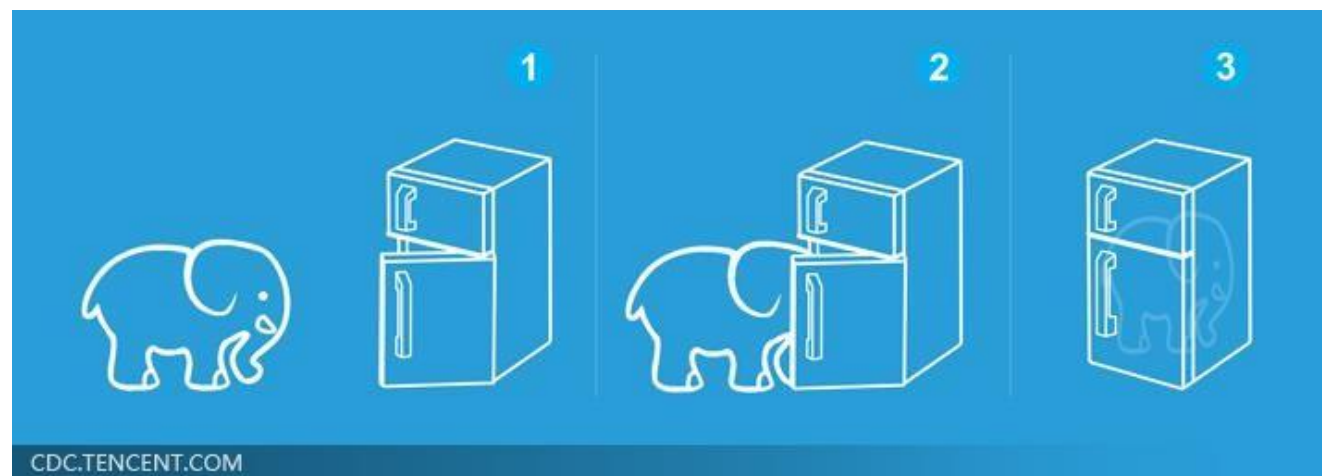
Find a function in function set that minimizes total loss L

Find the network parameters θ^* that minimize total loss L

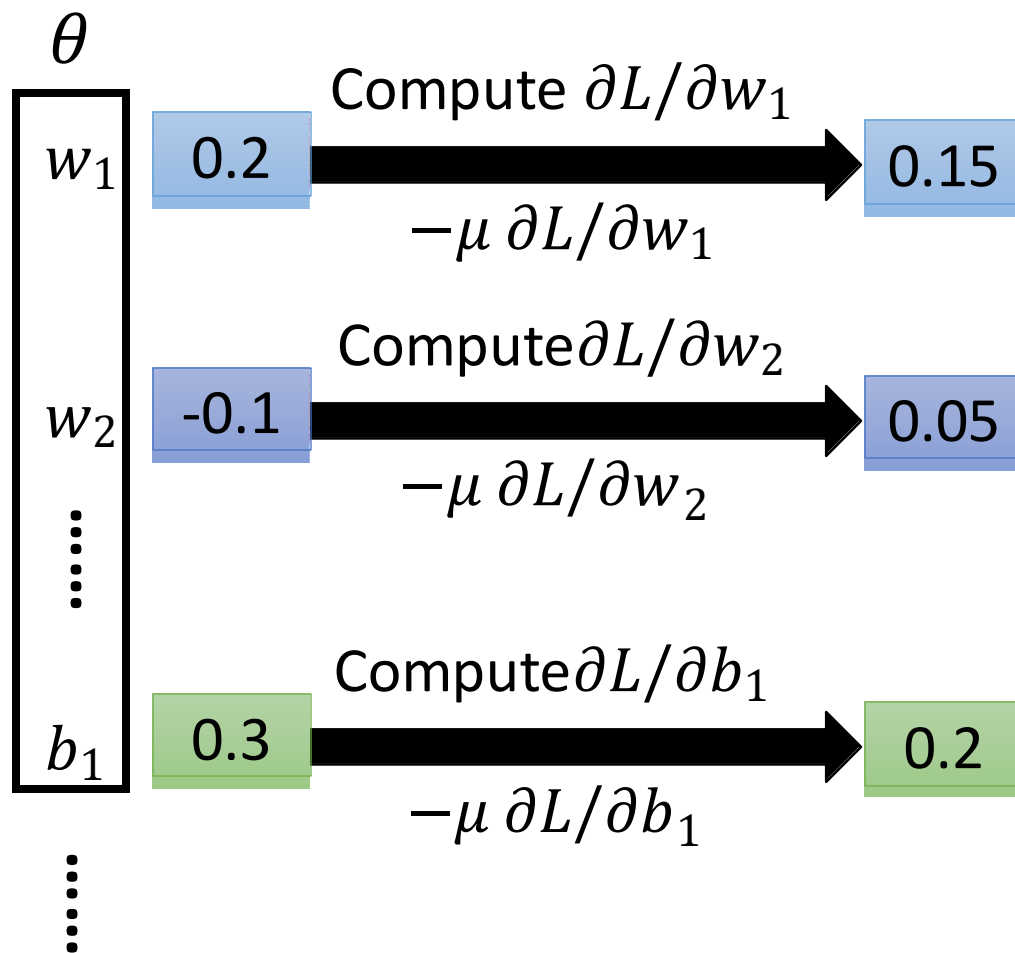
Three Steps for Deep Learning



Deep Learning is so simple



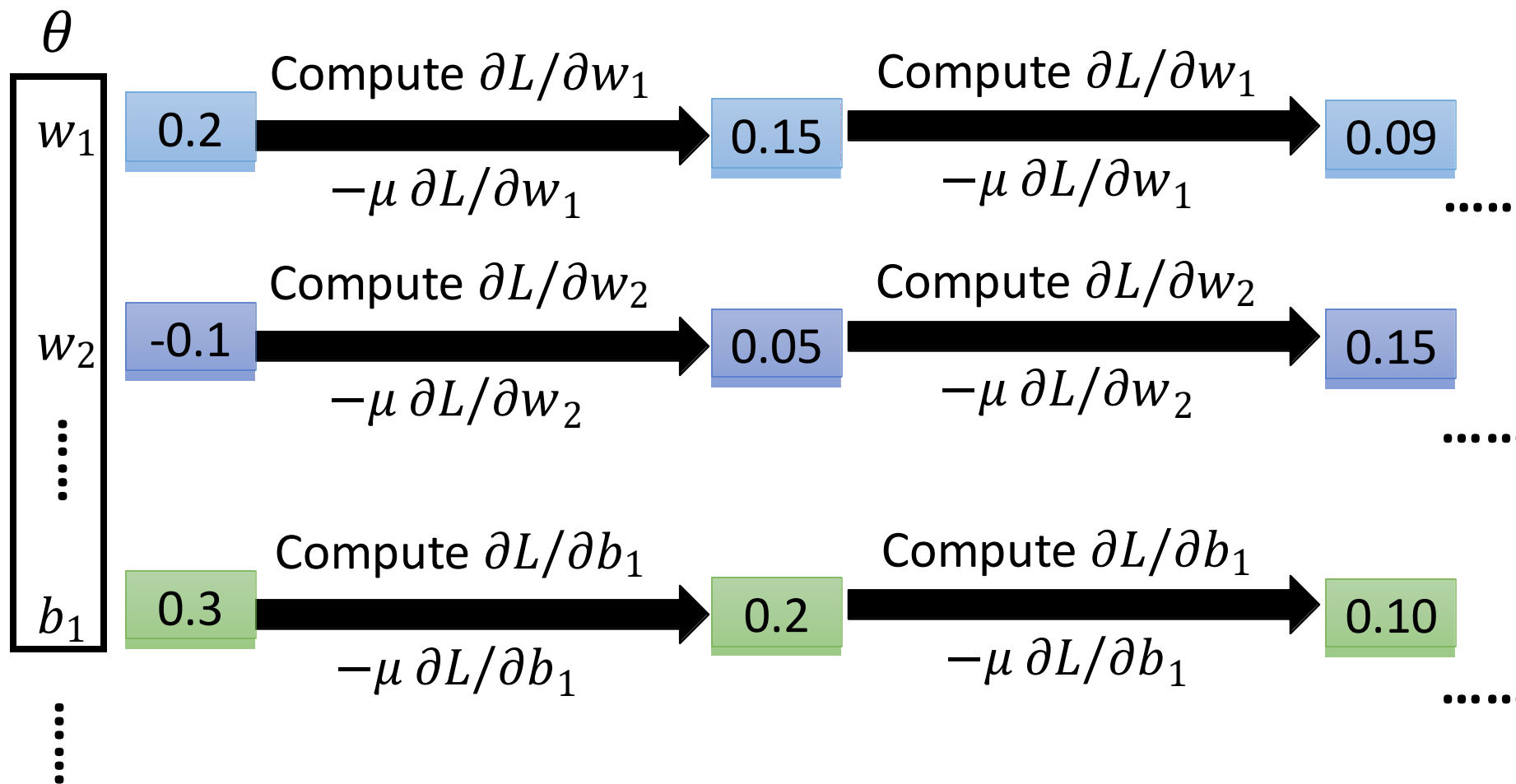
Gradient Descent



$$\nabla L = \begin{bmatrix} \frac{\partial L}{\partial w_1} \\ \frac{\partial L}{\partial w_2} \\ \vdots \\ \frac{\partial L}{\partial b_1} \\ \vdots \end{bmatrix}$$

gradient

Gradient Descent

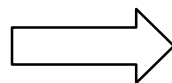
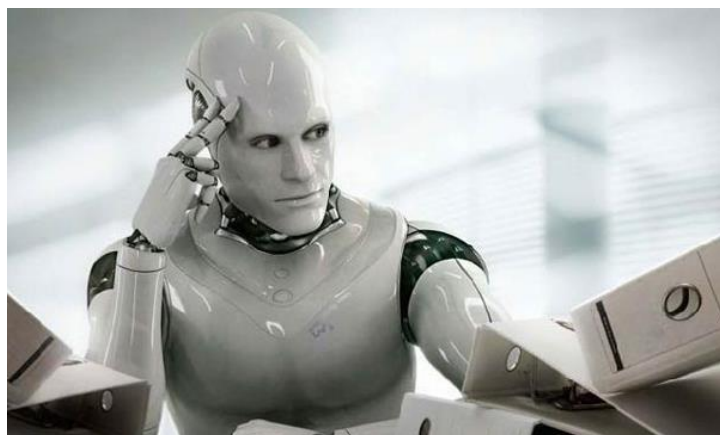


Gradient Descent

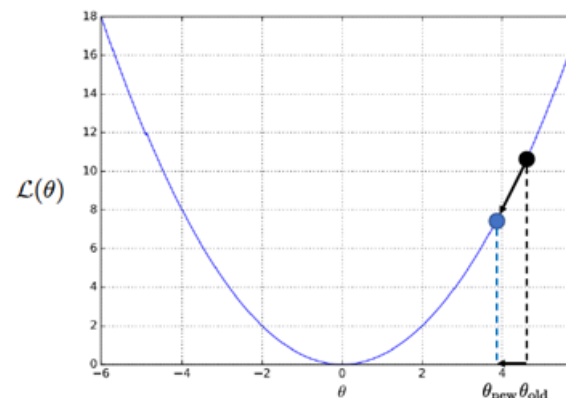
This is the “learning” of machines in deep learning

Even alpha go using this approach.

People image



Actually

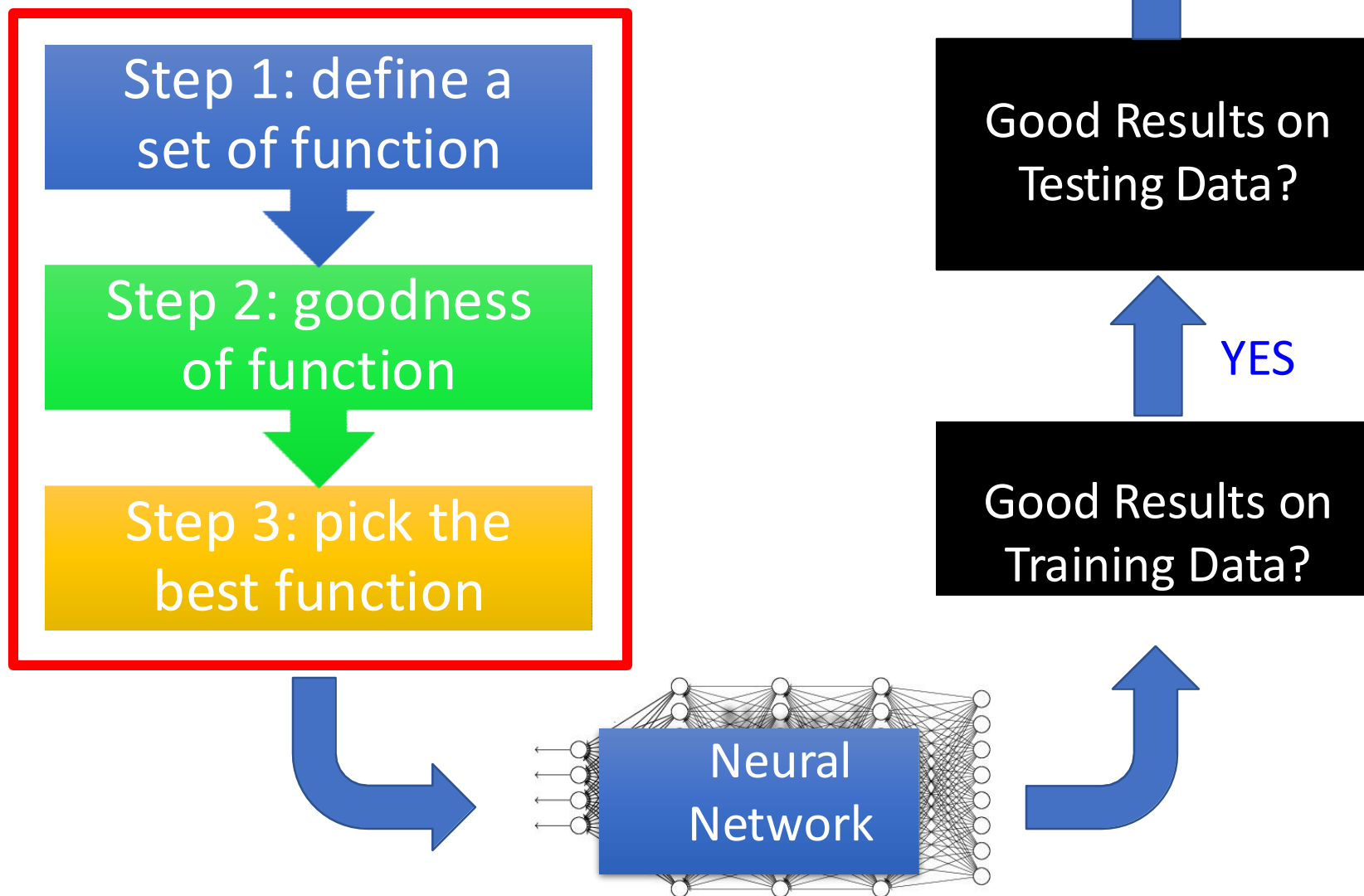


$$\theta_{\text{new}} \leftarrow \theta_{\text{old}} - \eta \frac{\partial \mathcal{L}(\theta)}{\partial \theta}$$

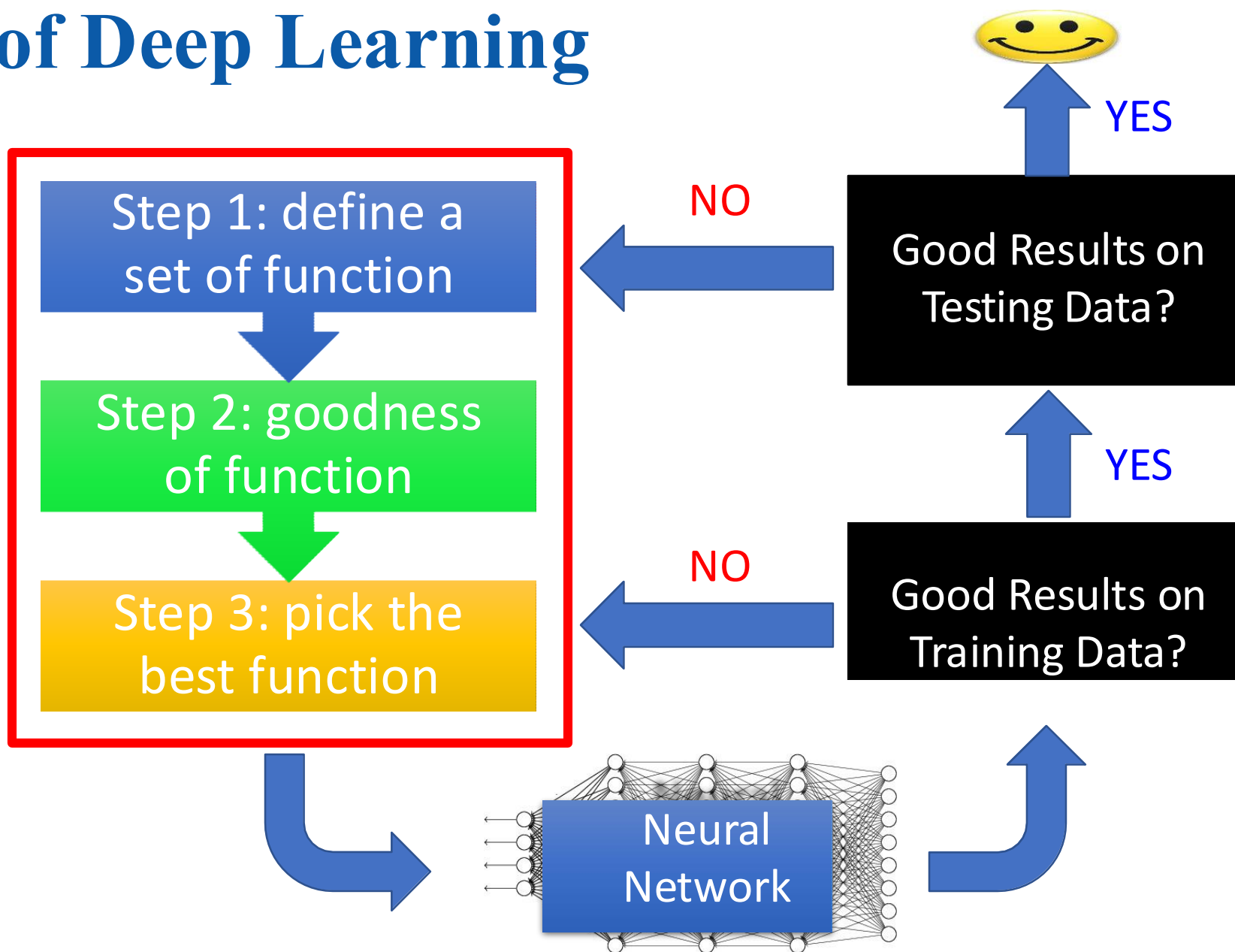
I hope you are not too disappointed :p

“深度学习”中的困难是什么？

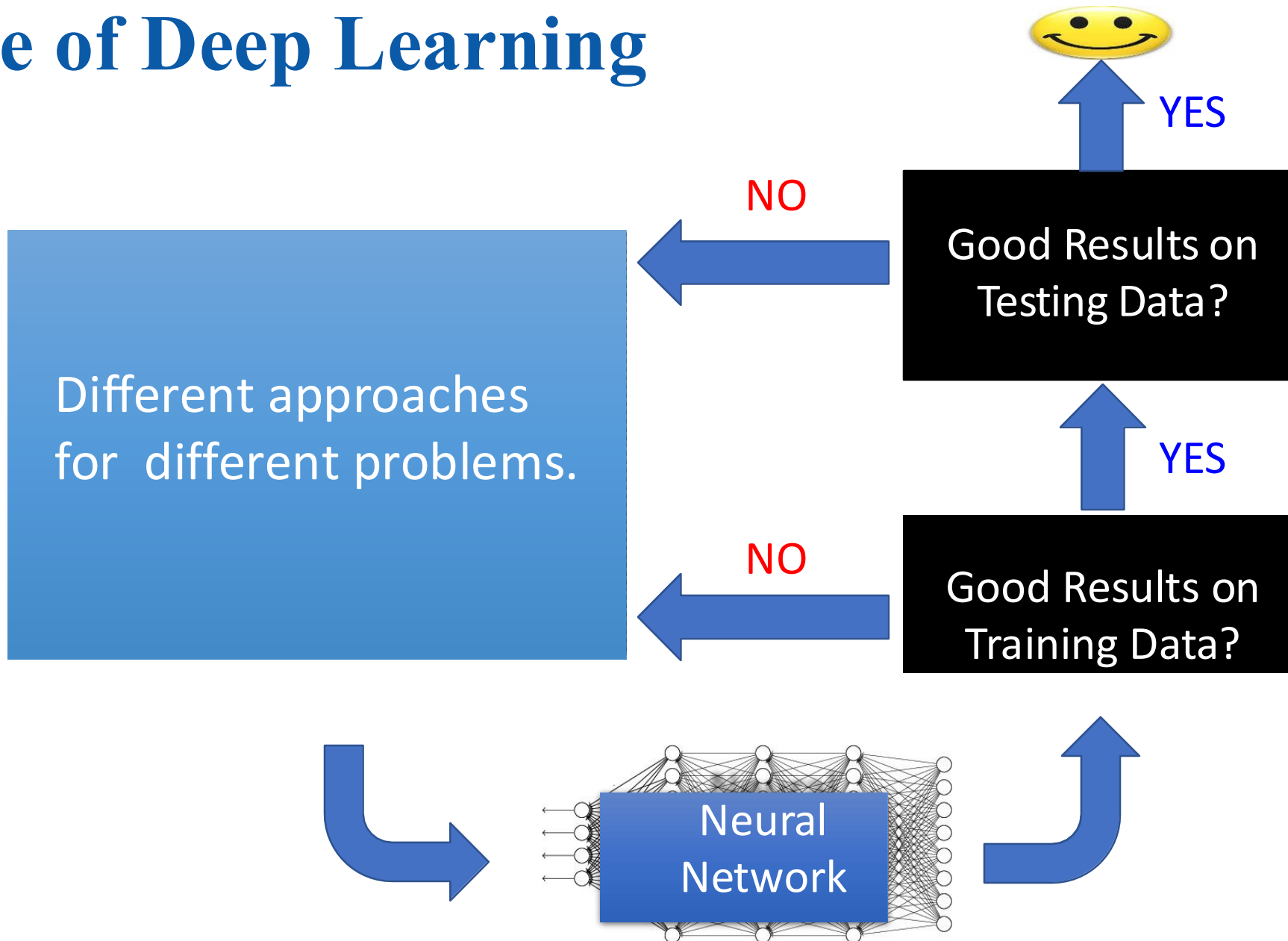
Recipe of Deep Learning



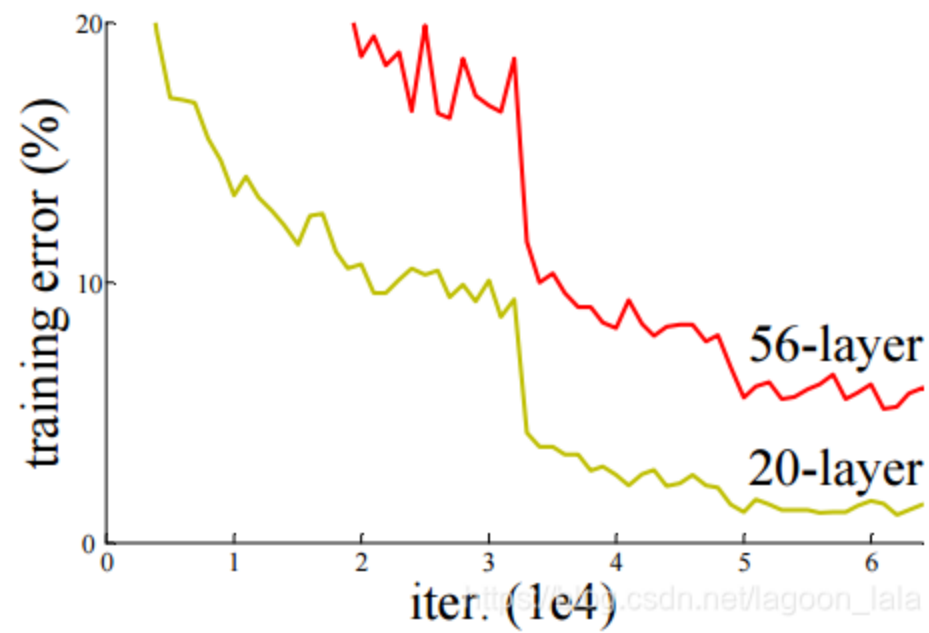
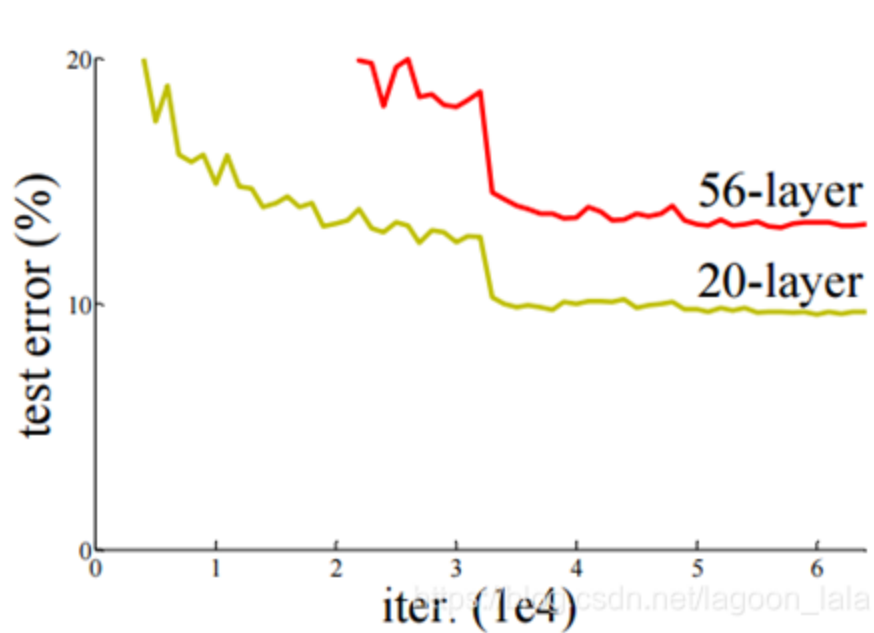
Recipe of Deep Learning



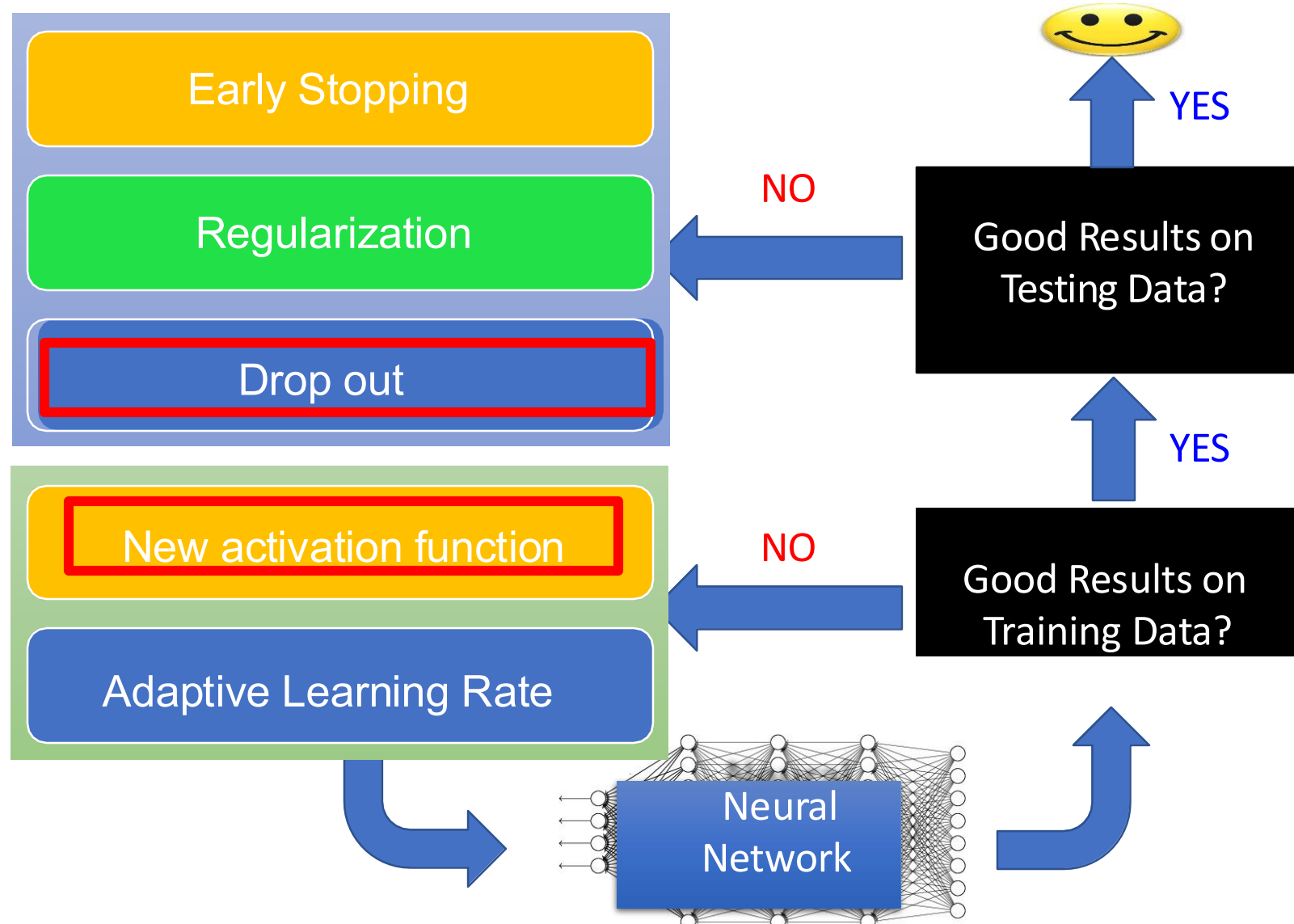
Recipe of Deep Learning



Overfitting? Underfitting?



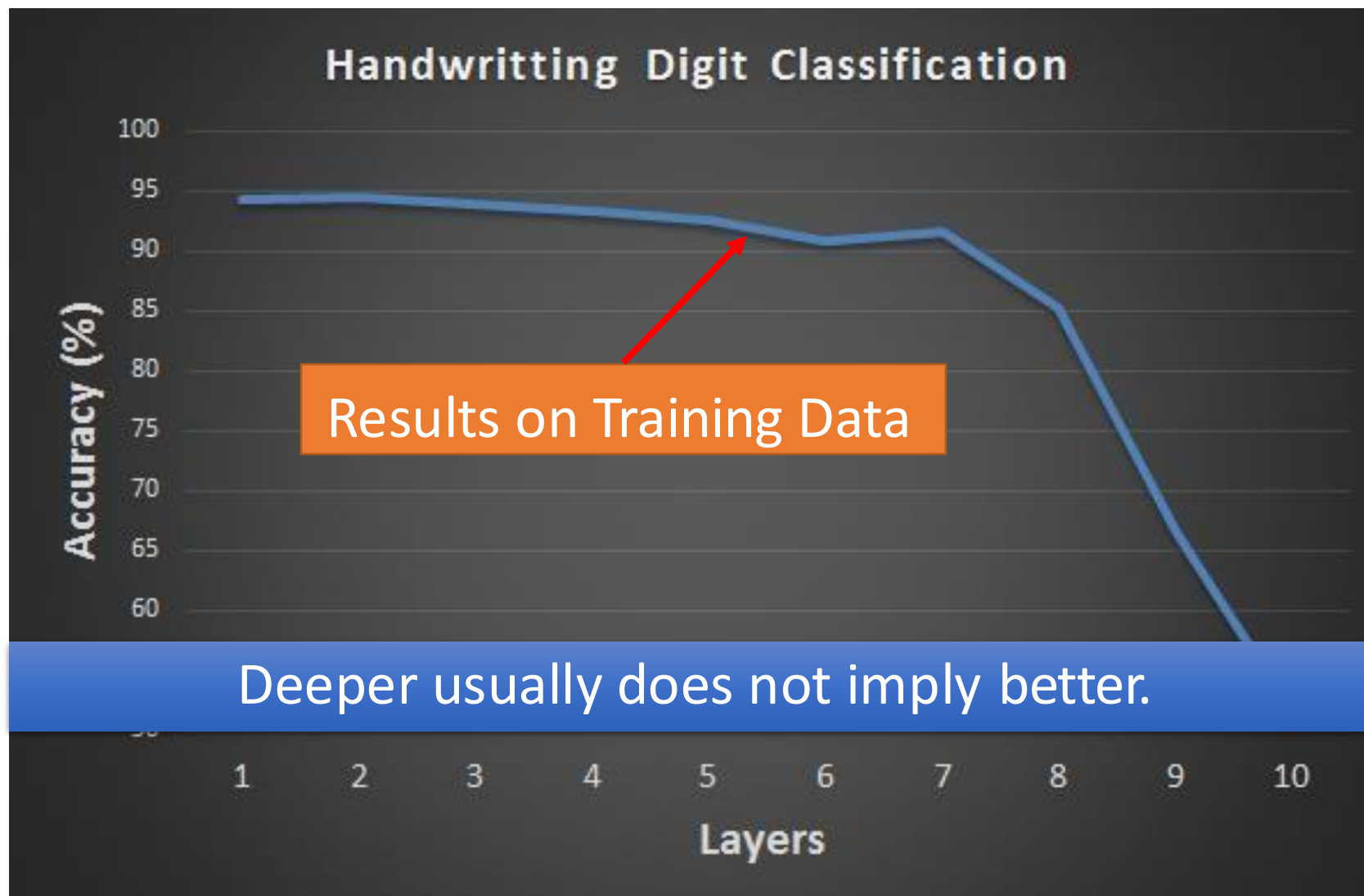
Recipe of Deep Learning



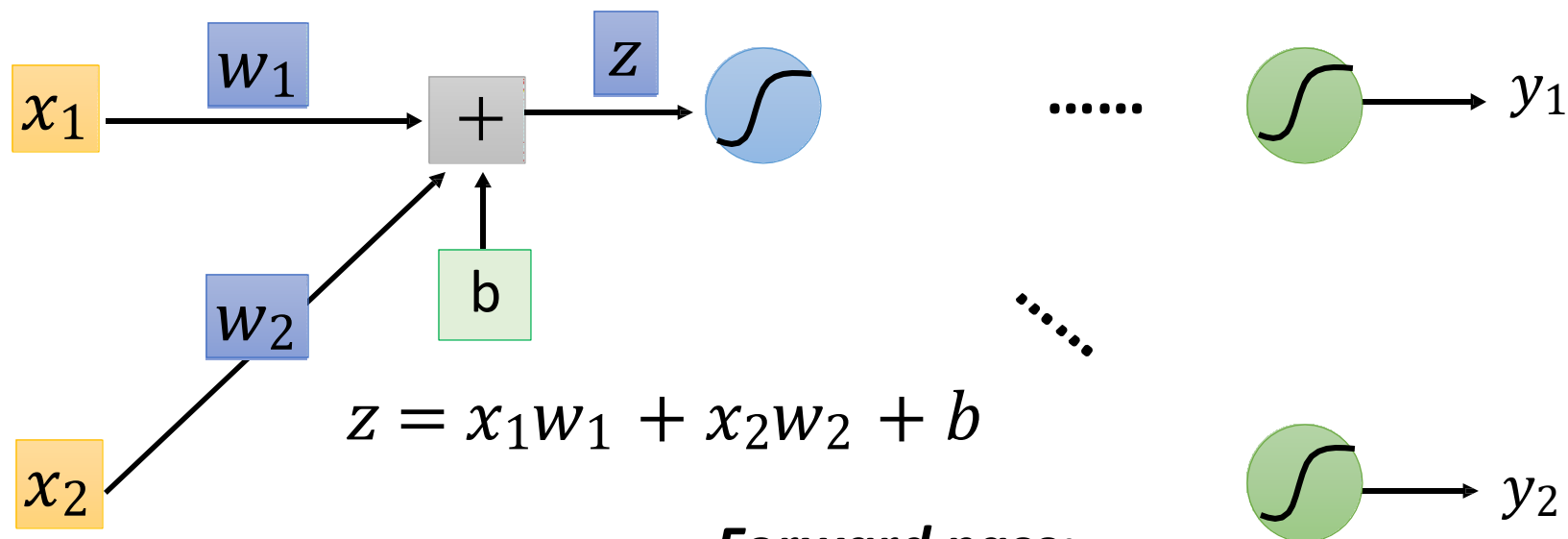
Adaptive Learning Rate

- **Momentum**
helps accelerate SGD in the relevant direction and dampens oscillations
- **Adagrad**
(Adaptive Gradient)
adapts the learning rate to the parameters, performing larger updates for infrequent and smaller updates for frequent parameters
- **RMSProp**
(Root Mean Square propagation)
divides the learning rate by an exponentially decaying average of squared gradients
- **Adam**
(Adaptive Moment Estimation)
stores an exponentially decaying average of past squared gradients like RMSprop, also keeps an exponentially decaying average of past gradients, similar to momentum

Hard to get the power of Deep ...



Backpropagation



Forward pass:

Compute $\partial z / \partial w$ for all parameters

Backward pass:

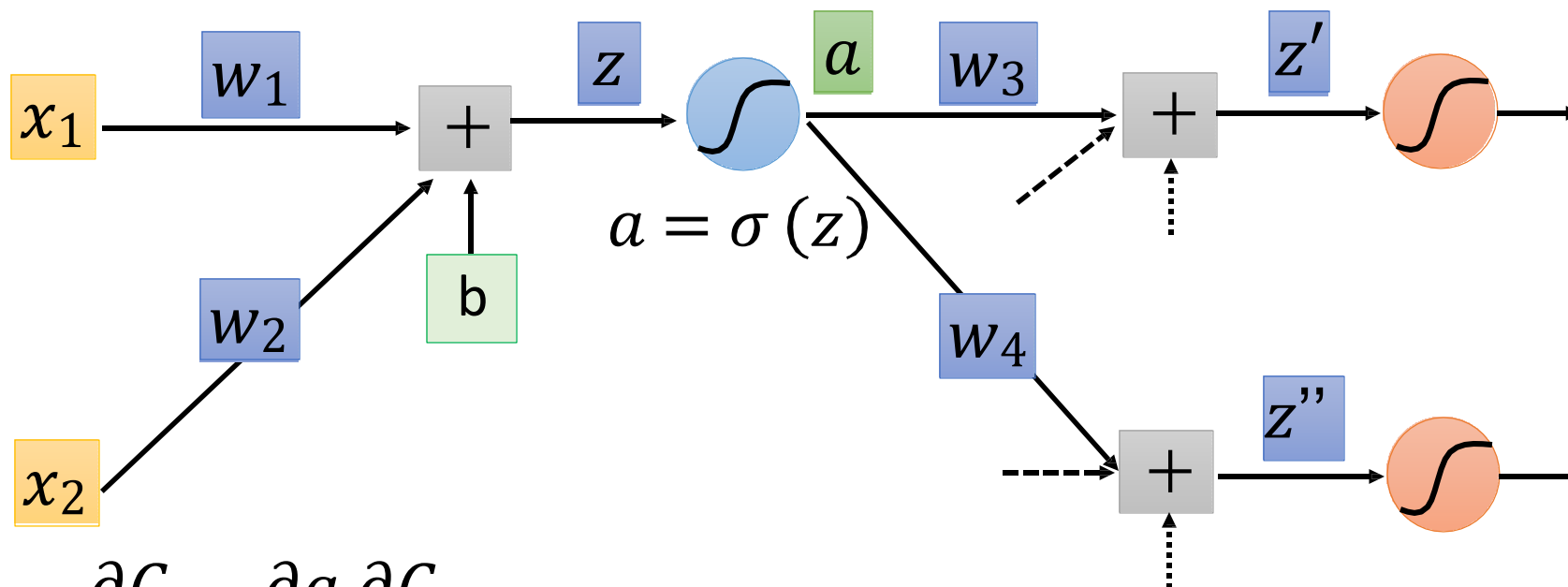
Compute $\partial C / \partial z$ for all activation function inputs z

$$\frac{\partial C}{\partial w} = \frac{\partial z}{\partial w} \frac{\partial C}{\partial z}$$

(Chain rule)

Backpropagation – Forward pass

Compute $\partial C / \partial z$ for all activation function inputs z

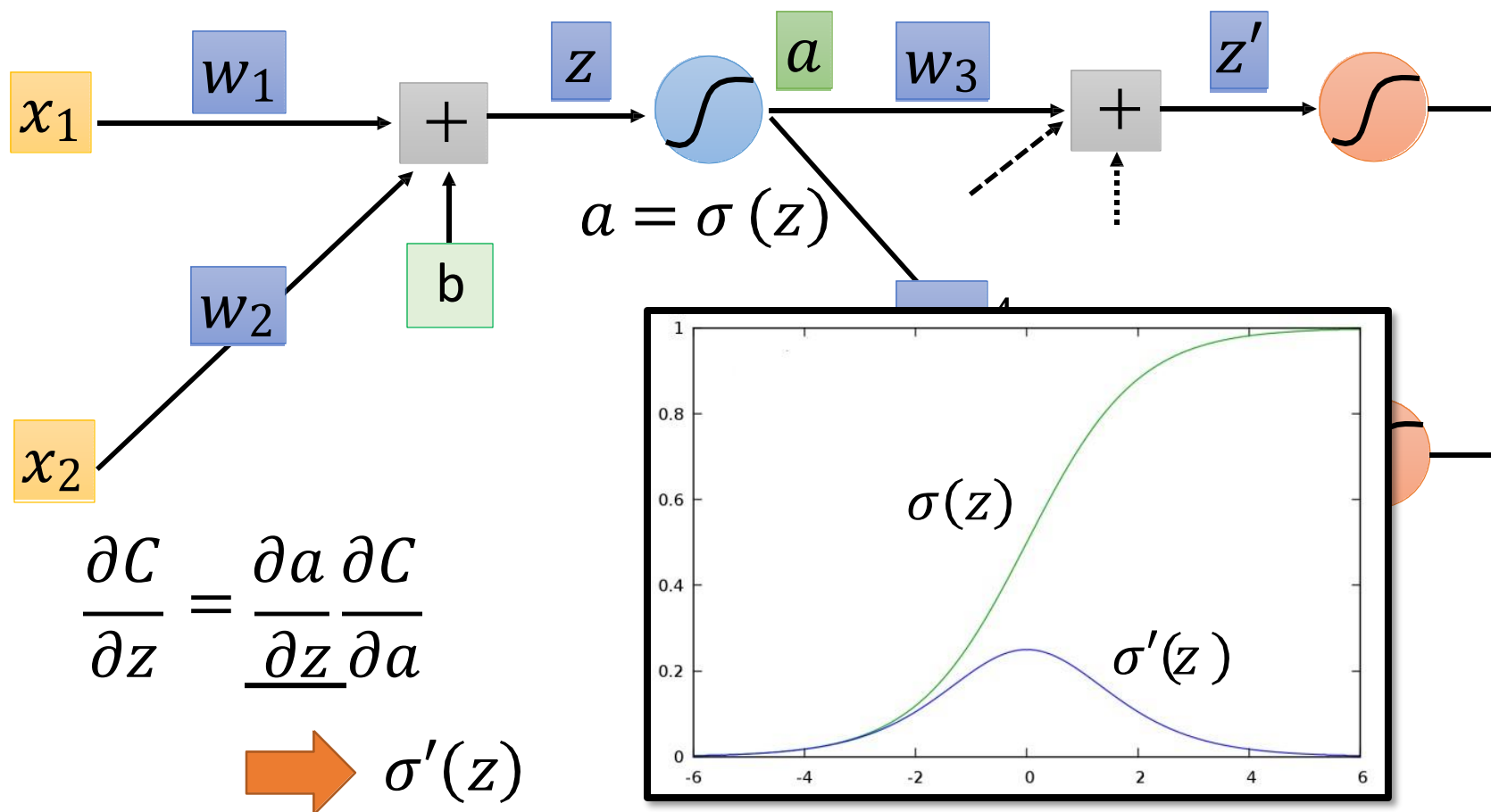


$$\frac{\partial C}{\partial z} = \frac{\partial a}{\partial z} \frac{\partial C}{\partial a}$$

➡ $\sigma'(z)$

Backpropagation – Forward pass

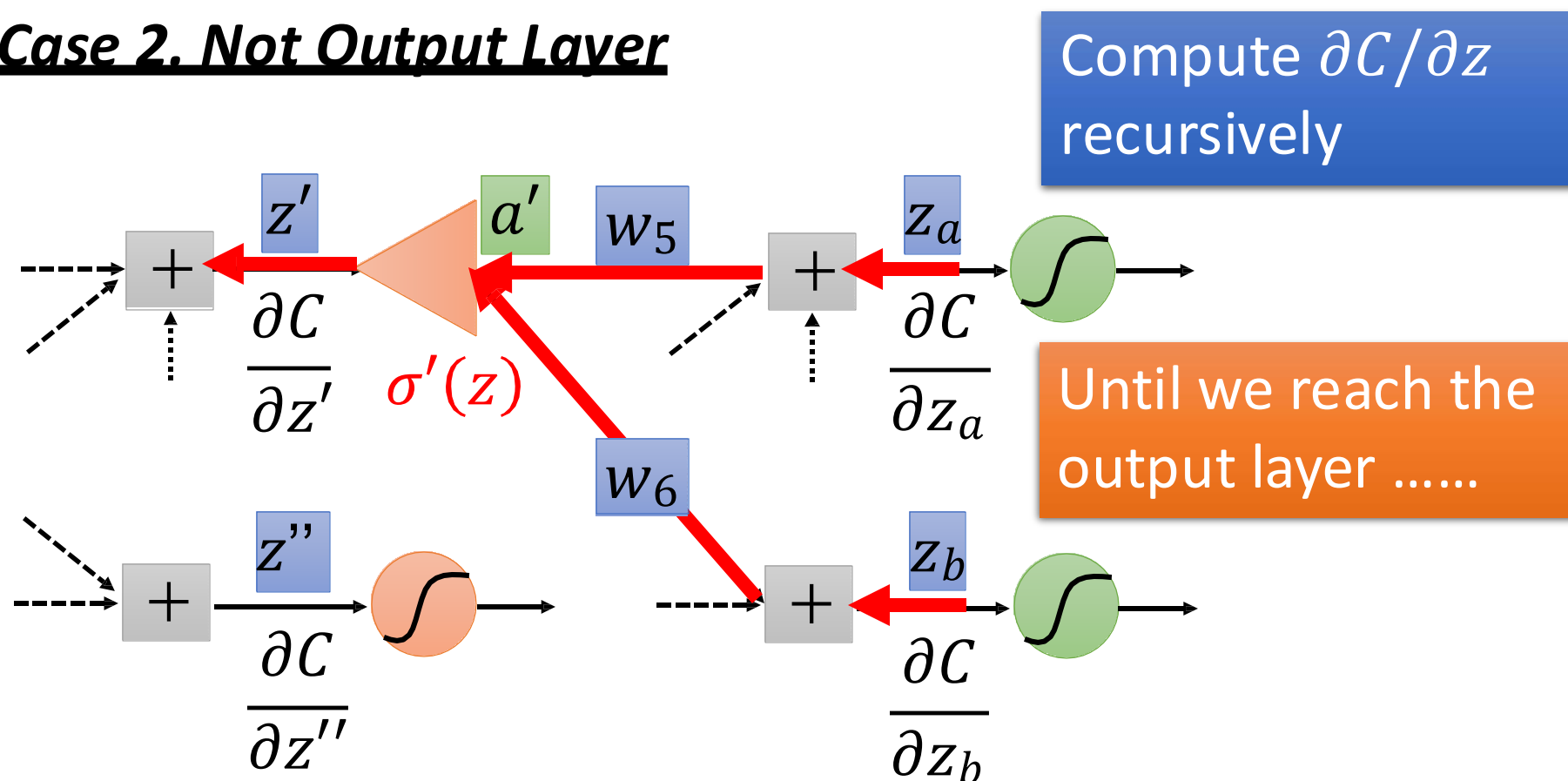
Compute $\partial C / \partial z$ for all activation function inputs z



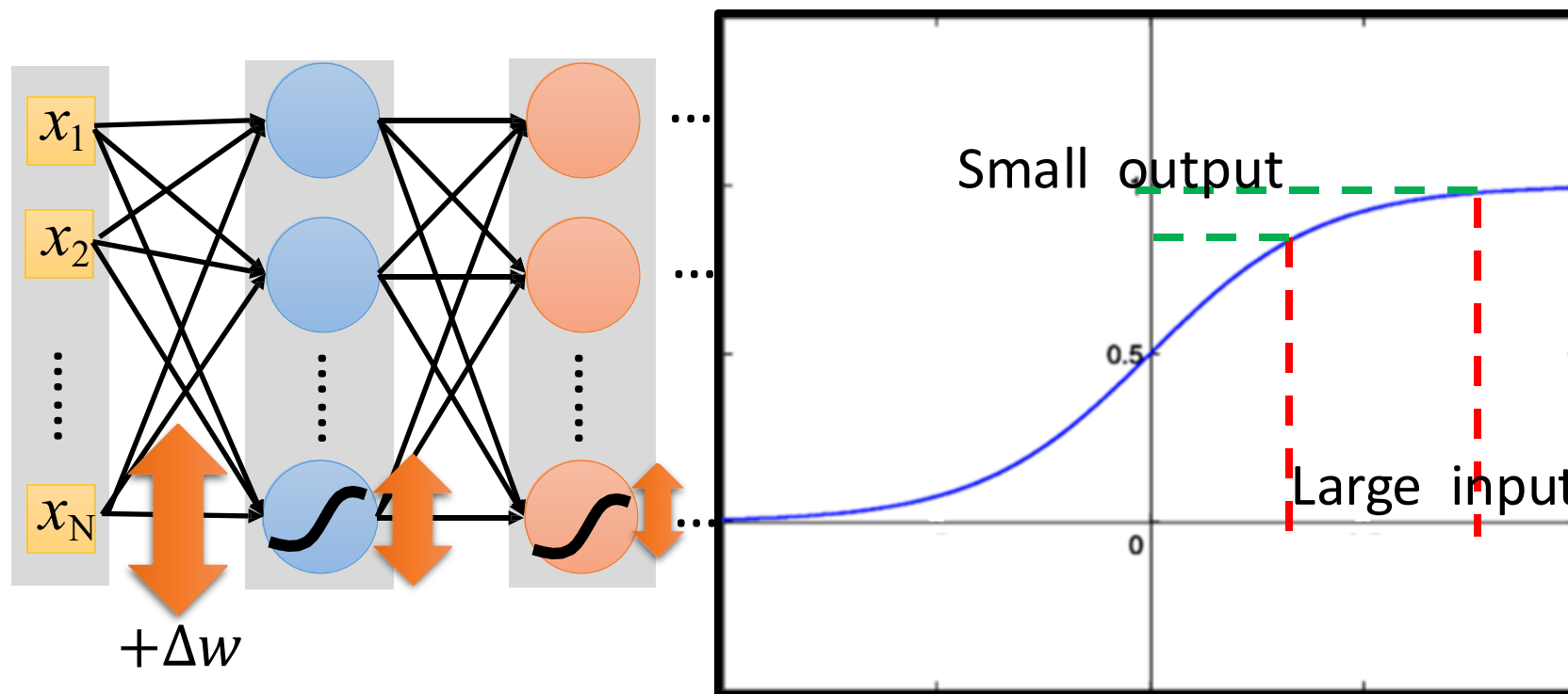
Backpropagation – Backward pass

Compute $\partial C / \partial z$ for all activation function inputs z

Case 2. Not Output Layer



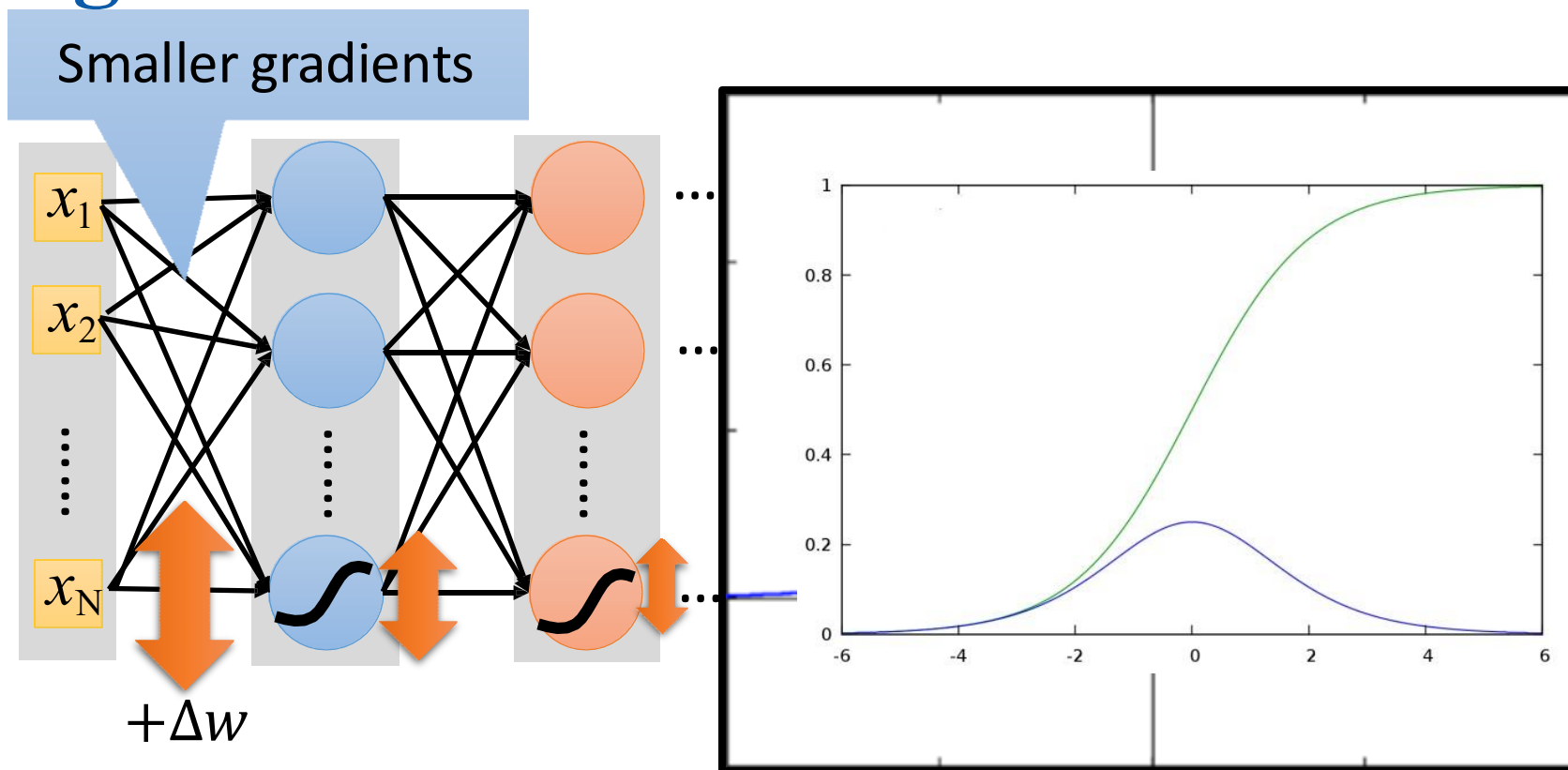
Vanishing Gradient Problem



$$\Delta w_1 = \frac{\partial \text{Loss}}{\partial w_2} = \frac{\partial \text{Loss}}{\partial f_4} \frac{\partial f_4}{\partial f_3} \frac{\partial f_3}{\partial f_2} \frac{\partial f_2}{\partial w_2}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial y_4} \frac{\partial y_4}{\partial z_4} \frac{\partial z_4}{\partial y_3} \frac{\partial y_3}{\partial z_3} \frac{\partial z_3}{\partial y_2} \frac{\partial y_2}{\partial z_2} \frac{\partial z_2}{\partial y_1} \frac{\partial y_1}{\partial z_1} \frac{\partial z_1}{\partial b_1}$$

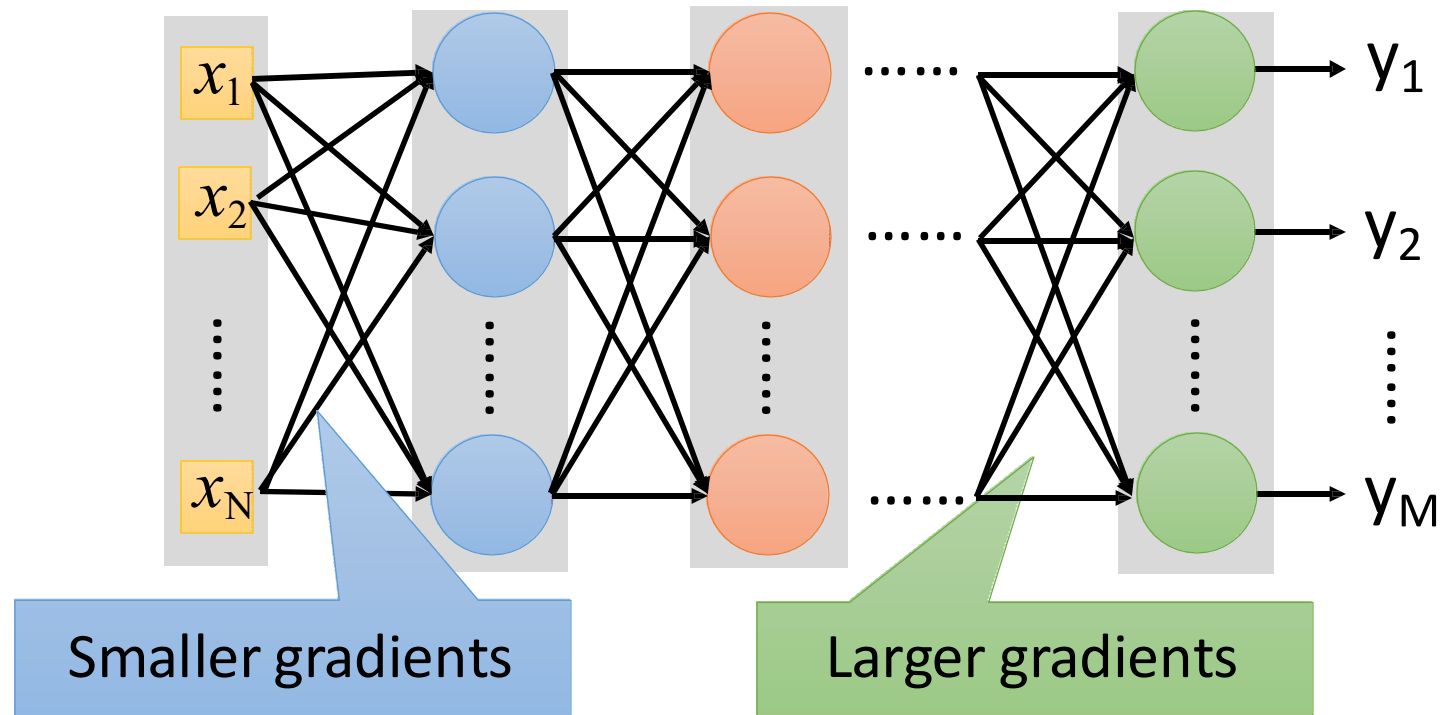
Vanishing Gradient Problem



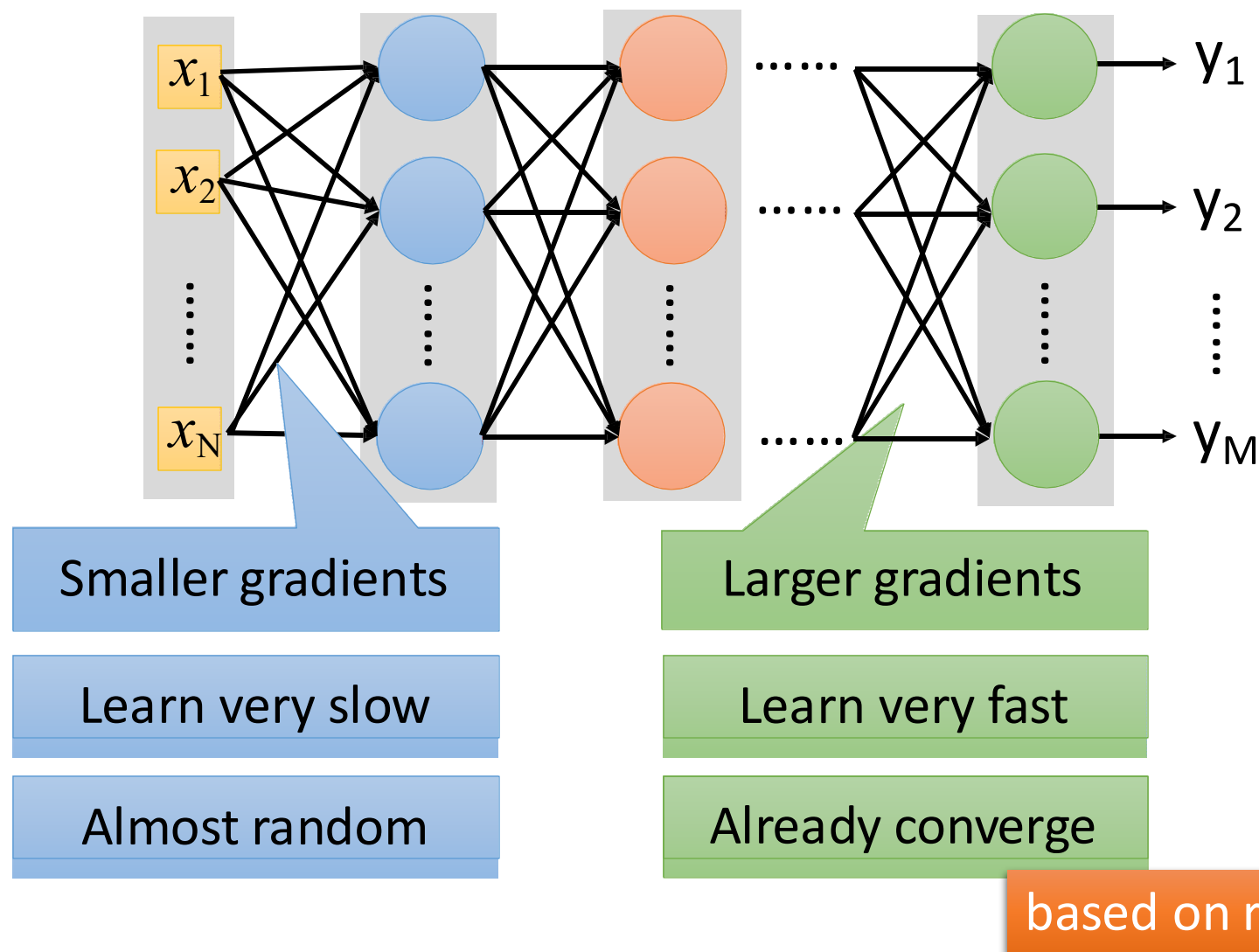
$$\Delta w_1 = \frac{\partial \text{Loss}}{\partial w_2} = \frac{\partial \text{Loss}}{\partial f_4} \frac{\partial f_4}{\partial f_3} \frac{\partial f_3}{\partial f_2} \frac{\partial f_2}{\partial w_2}$$

$$\frac{\partial C}{\partial b_1} = \frac{\partial C}{\partial y_4} \frac{\partial y_4}{\partial z_4} \frac{\partial z_4}{\partial y_3} \frac{\partial y_3}{\partial z_3} \frac{\partial z_3}{\partial y_2} \frac{\partial y_2}{\partial z_2} \frac{\partial z_2}{\partial y_1} \frac{\partial y_1}{\partial z_1} \frac{\partial z_1}{\partial b_1}$$

Vanishing Gradient Problem



Vanishing Gradient Problem

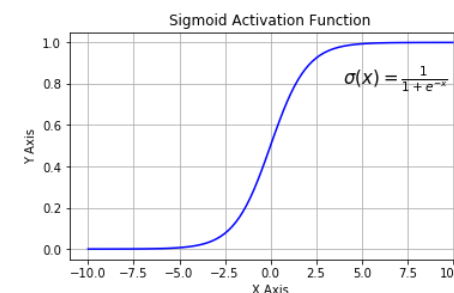


Activation Function

- Activation function must be **continuous, differentiable, non-linear, and easy to compute**

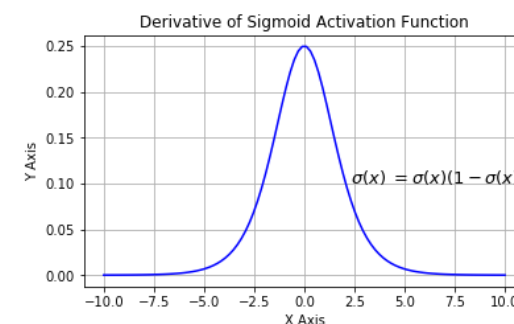
Logistic Sigmoid:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Its derivative:

$$\sigma(z)' = \sigma(z)(1 - \sigma(z))$$

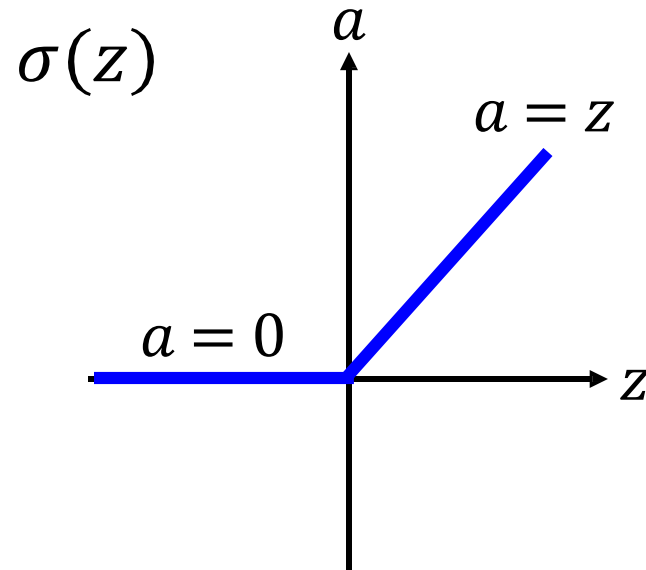


Output range [0,1];

Motivated by biological neurons and can be interpreted as the probability of an artificial neuron 'firing' given its inputs

New Activation Function ReLU

- Rectified Linear Unit (ReLU)

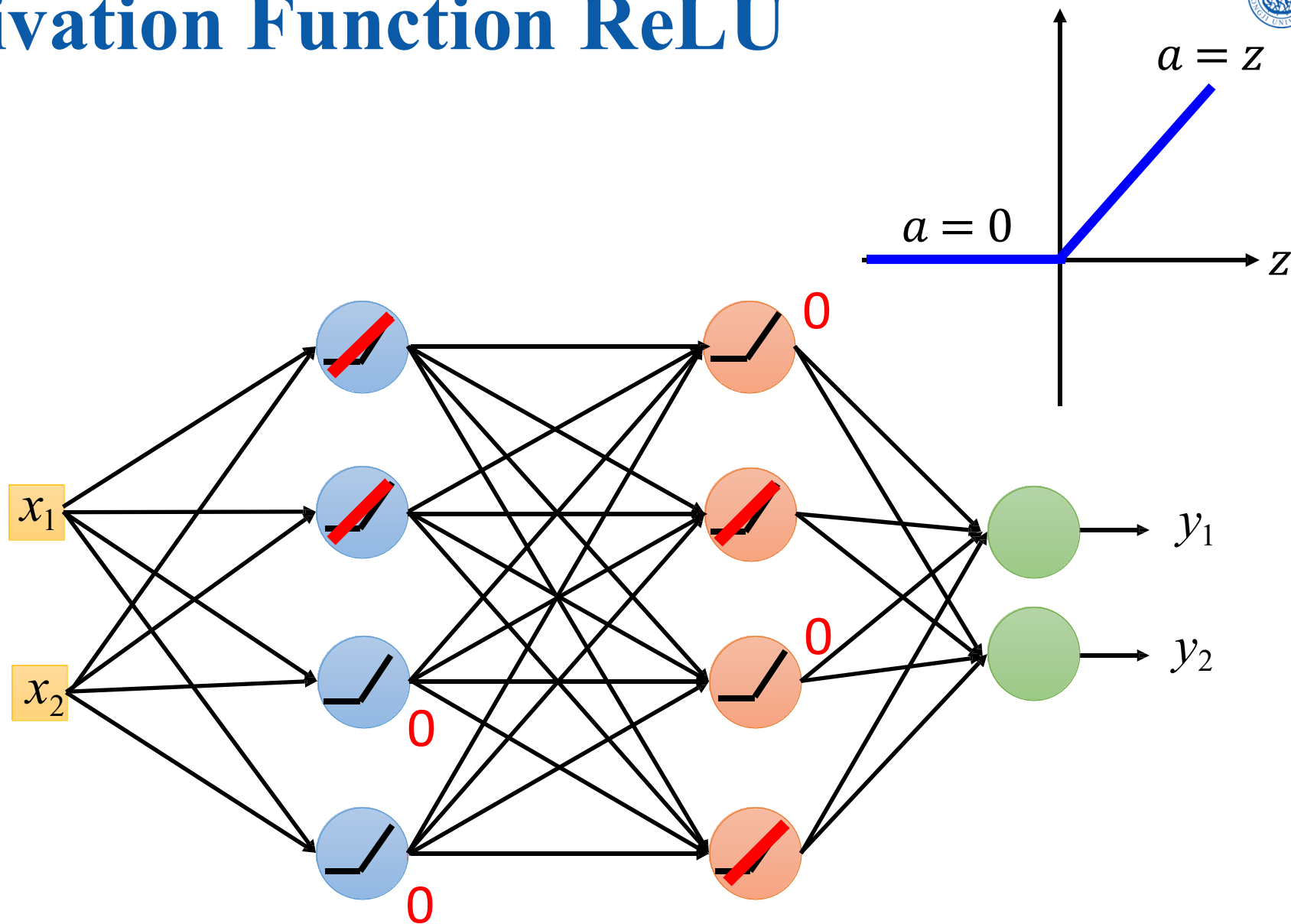


[Xavier Glorot, AISTATS'11]
[Andrew L. Maas, ICML'13]
[Kaiming He, arXiv'15]

Reason:

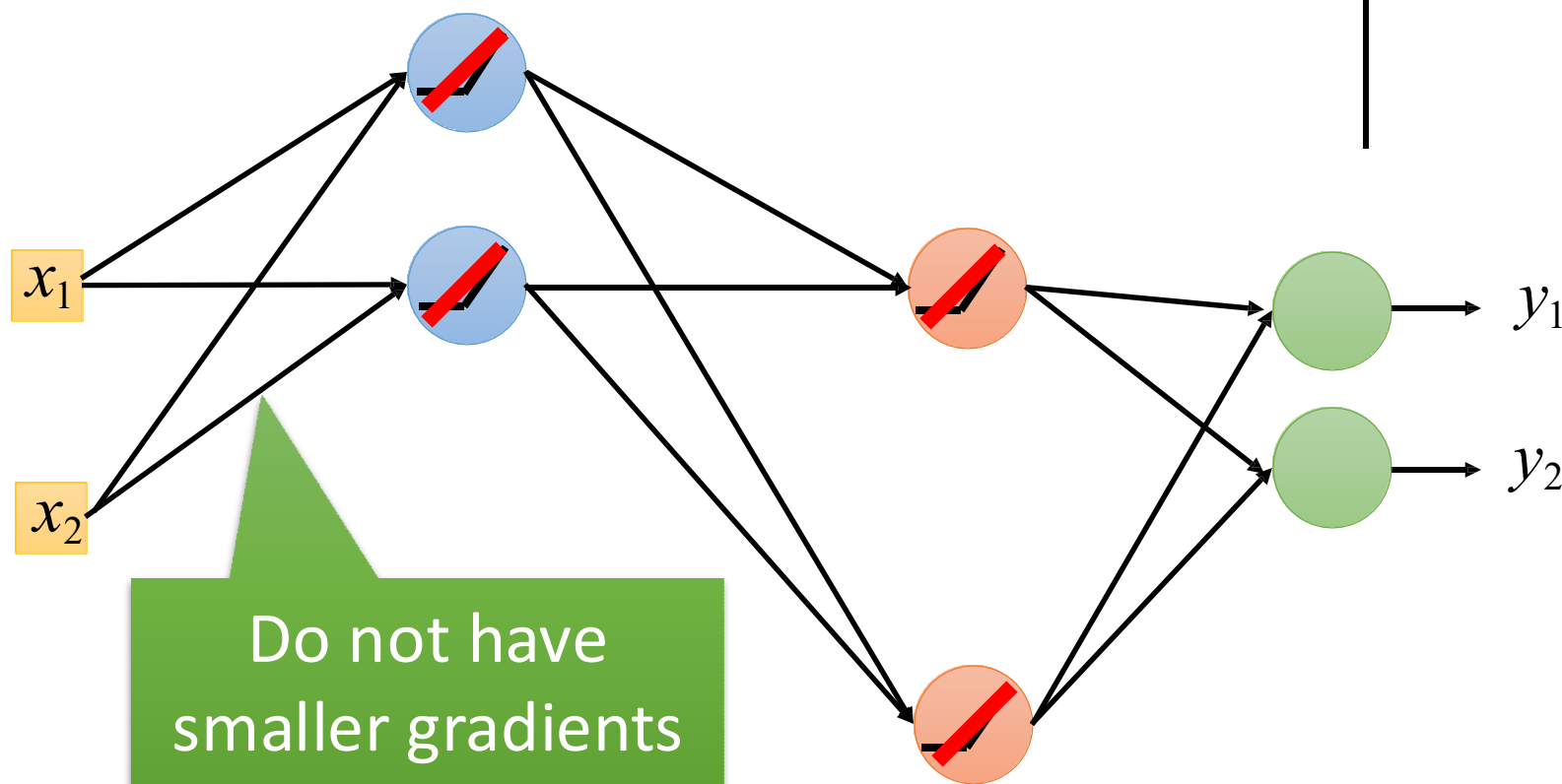
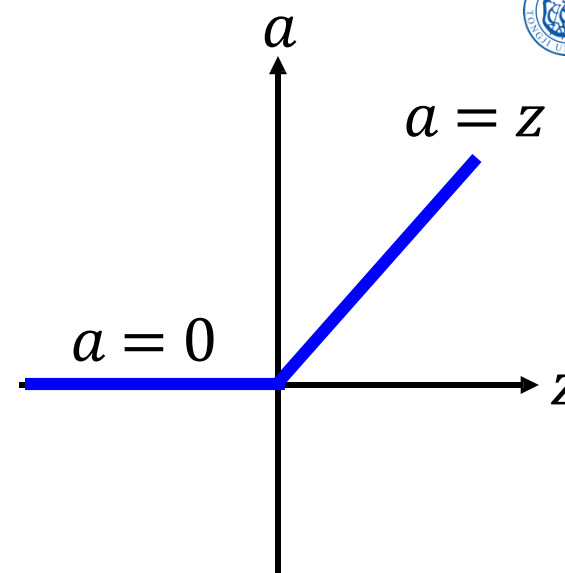
1. Fast to compute
2. Biological reason
3. Infinite sigmoid with different biases
4. Vanishing gradient problem

New Activation Function ReLU



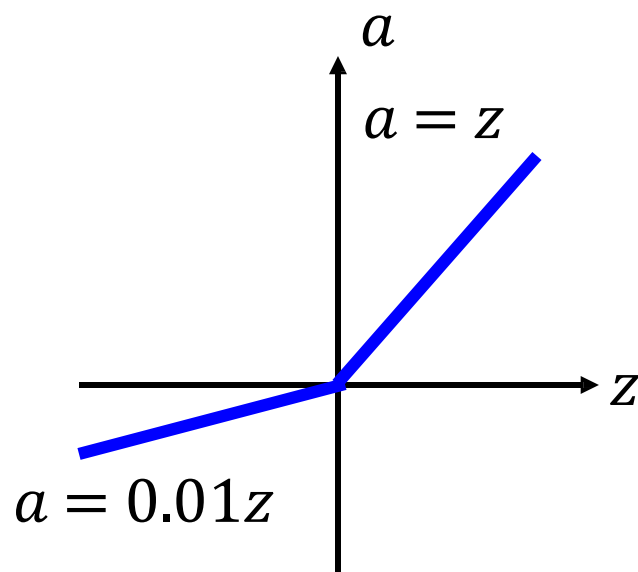
New Activation Function ReLU

A Thinner linear network

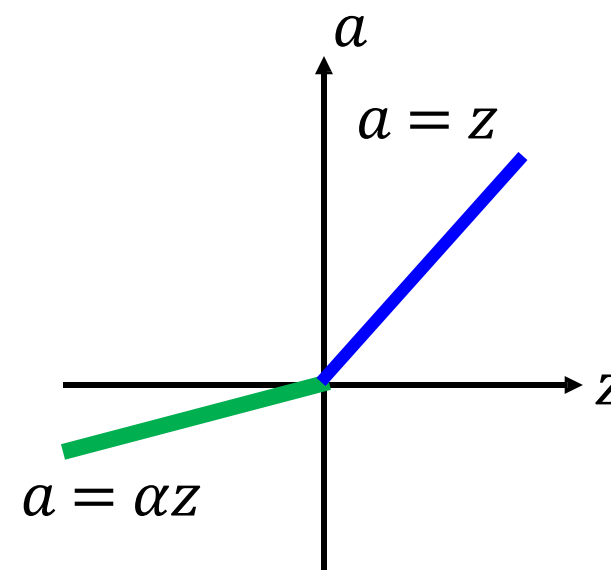


New Activation Function ReLU - variant

Leaky ReLU



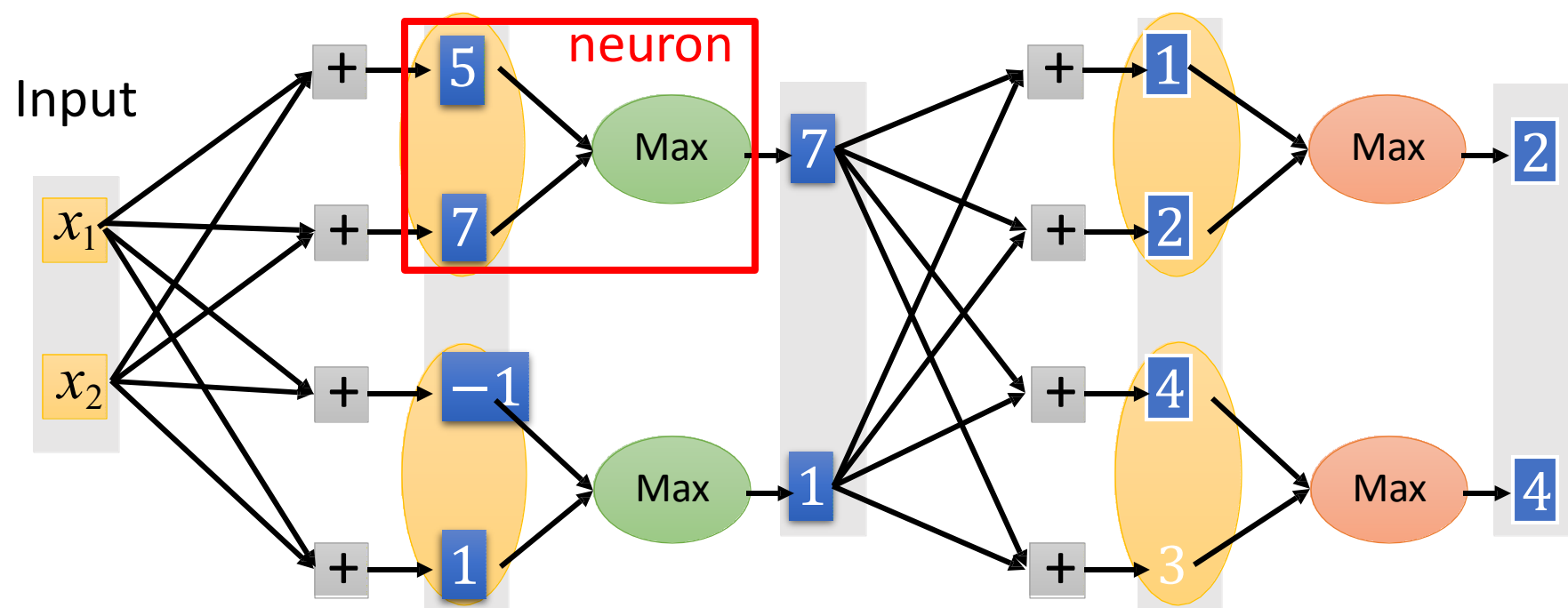
Parametric ReLU



α also learned by
gradient descent

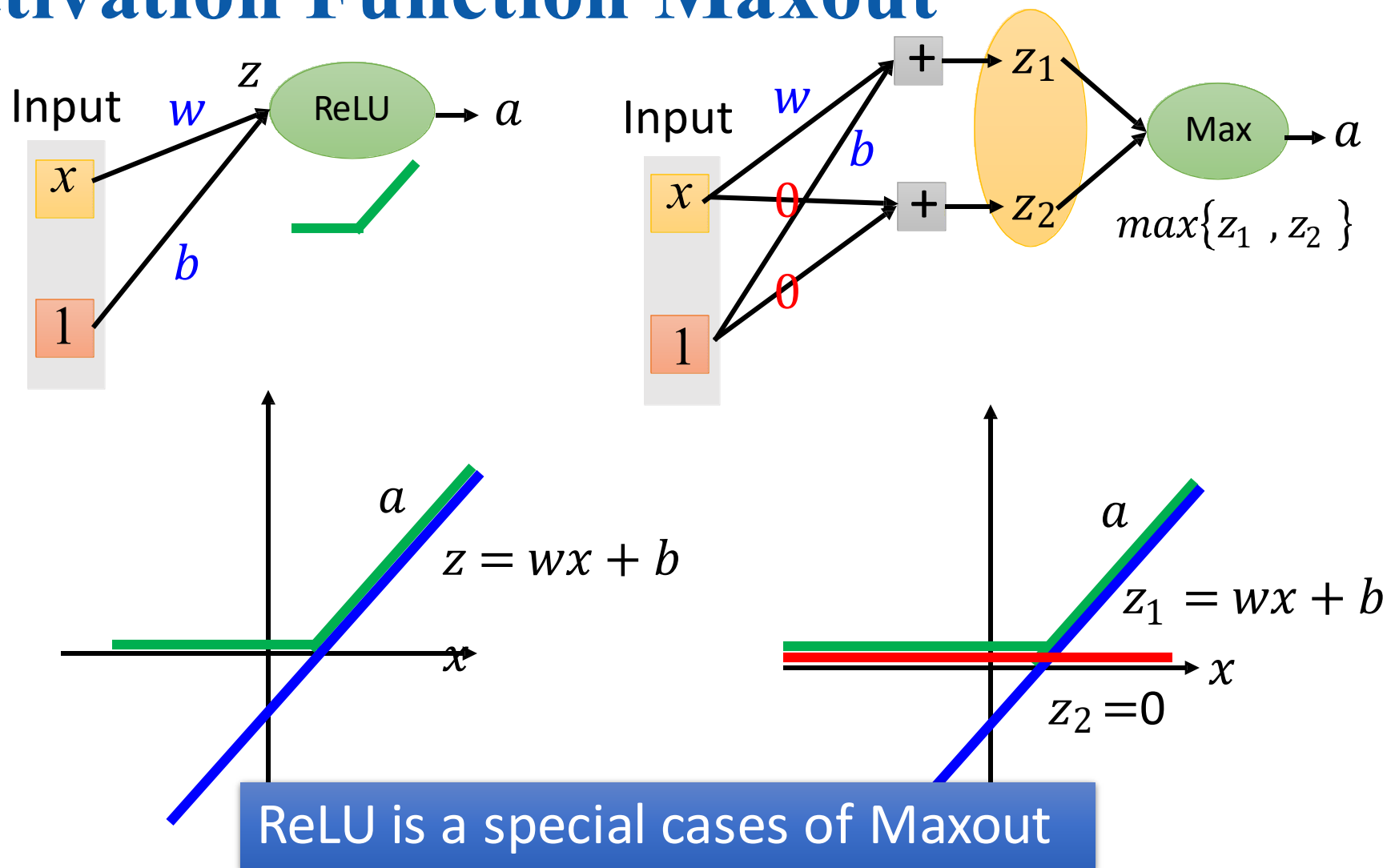
New Activation Function Maxout

- Learnable activation function [Ian J. Goodfellow, ICML'13]

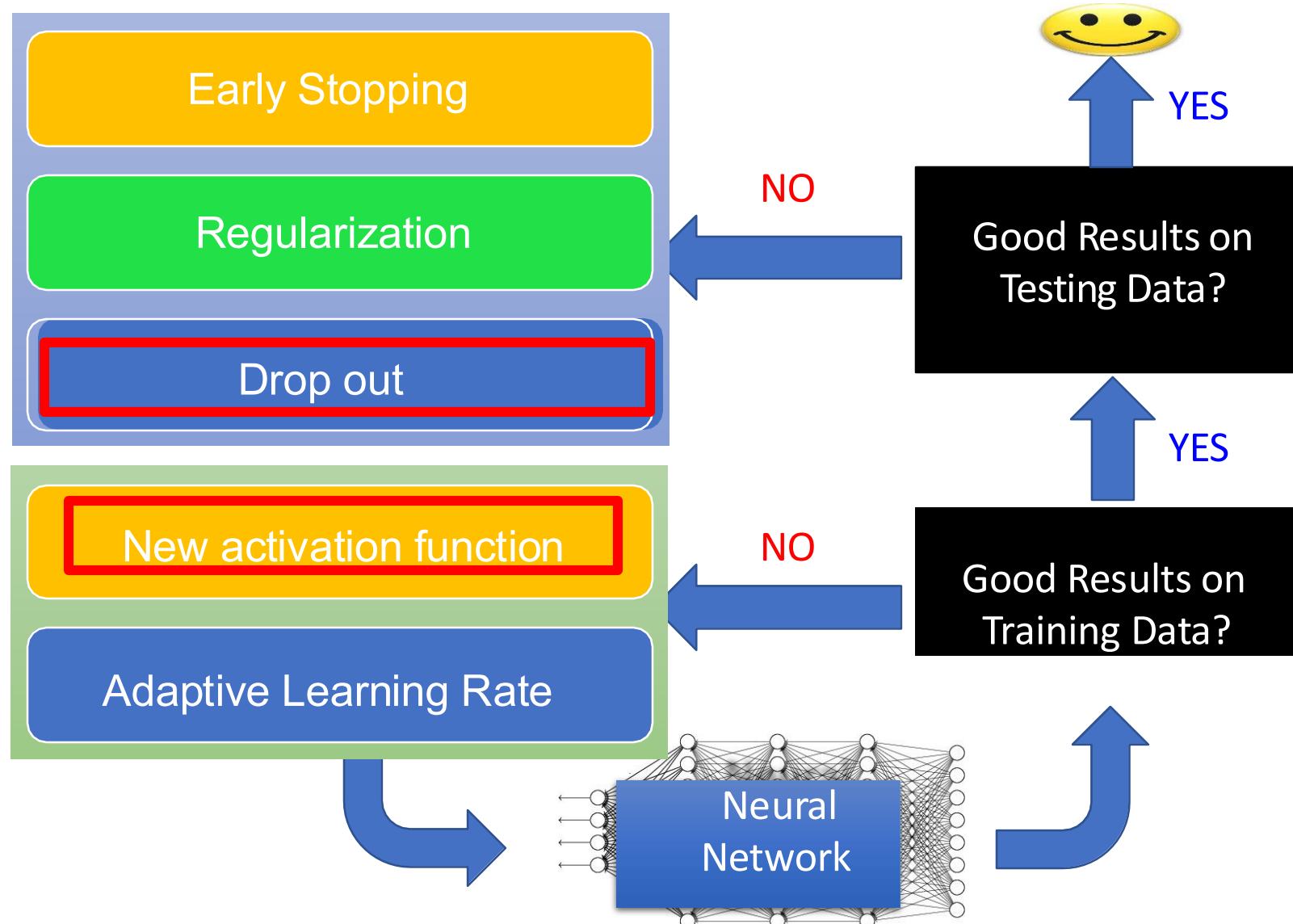


You can have more than 2 elements in a group.

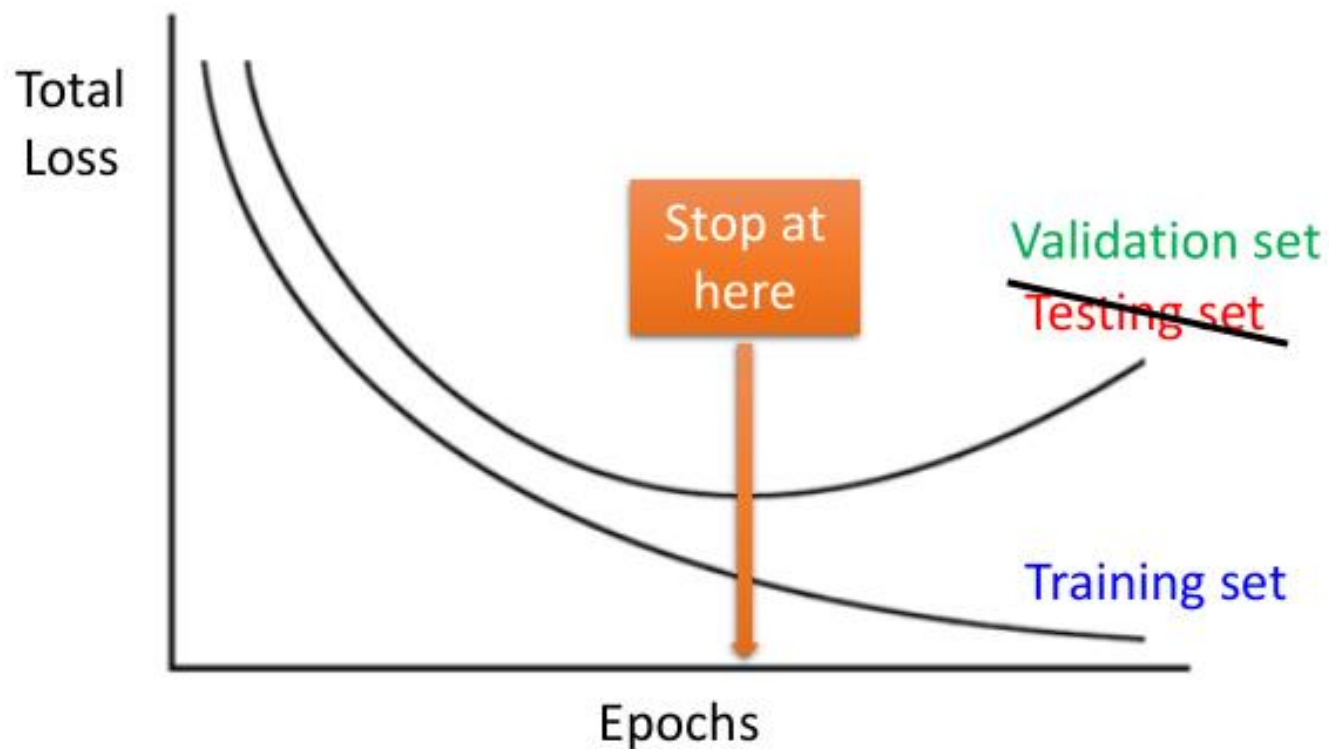
New Activation Function Maxout



Recipe of Deep Learning



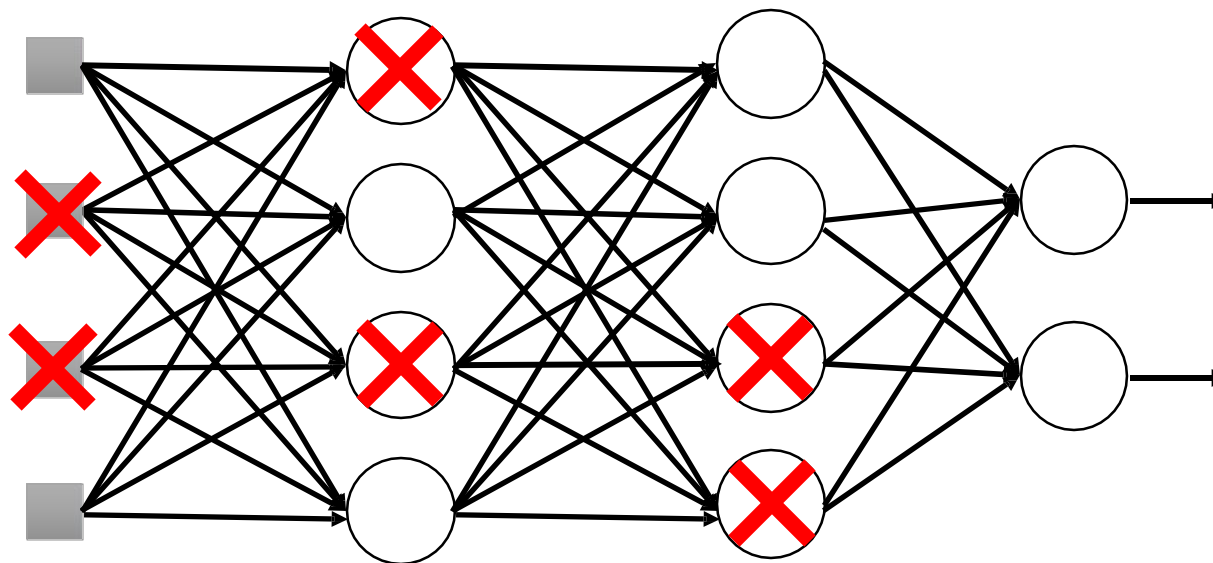
Early Stopping



Keras: <http://keras.io/getting-started/faq/#how-can-i-interrupt-training-when-the-validation-loss-isnt-decreasing-anymore>

Dropout

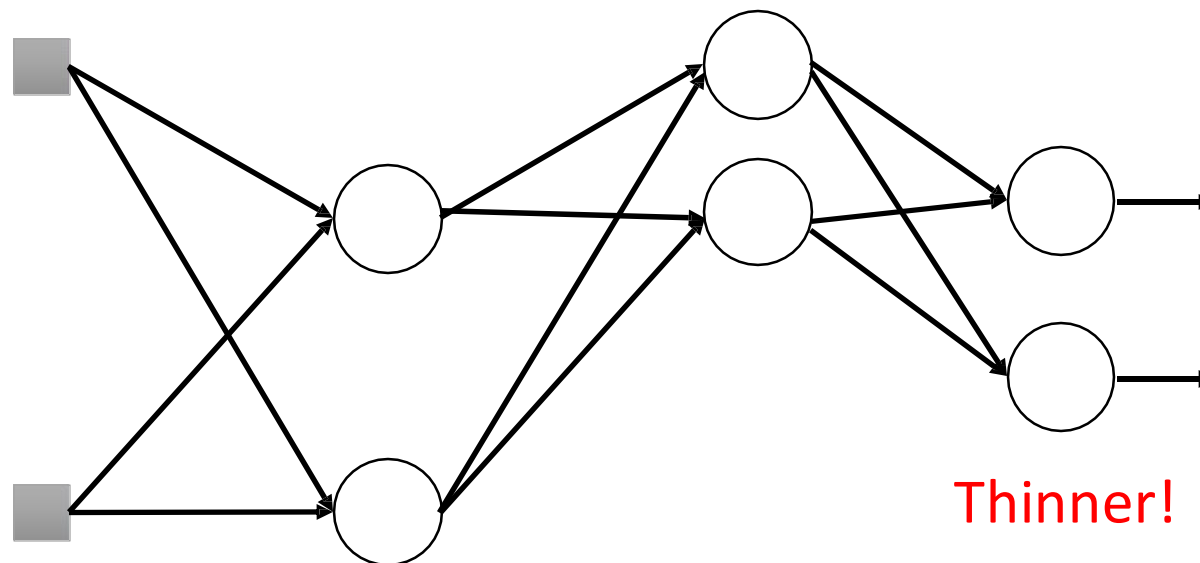
Training:



- Each time before updating the parameters
 - Each neuron has $p\%$ to dropout

Dropout

Training:



➤ Each time before updating the parameters

- Each neuron has $p\%$ to dropout



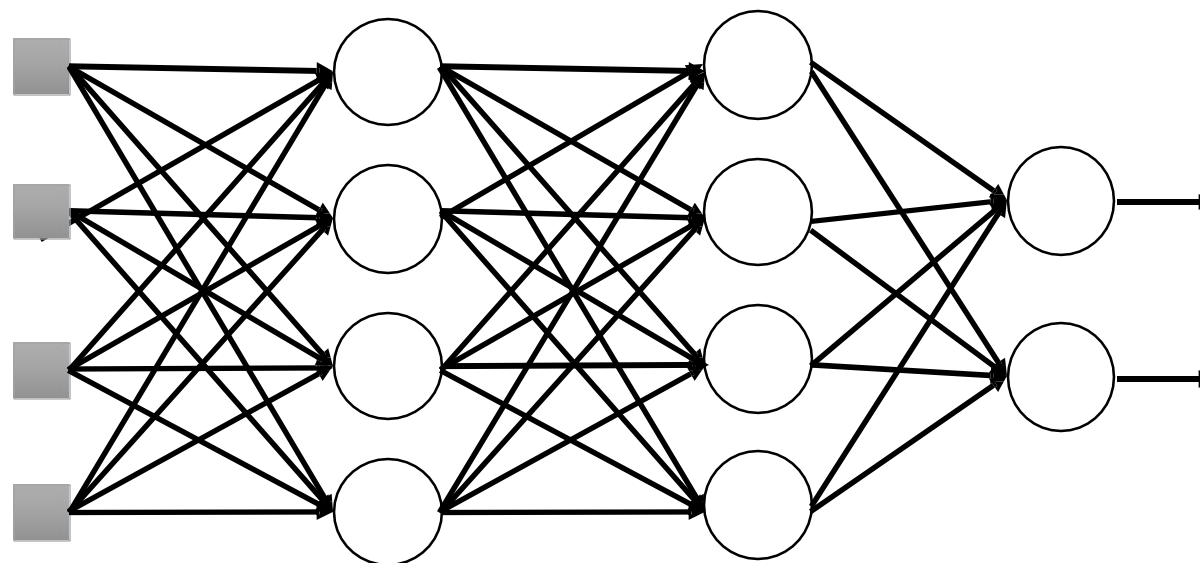
The structure of the network is changed.

- Using the new network for training

For each mini-batch, we resample the dropout neurons

Dropout

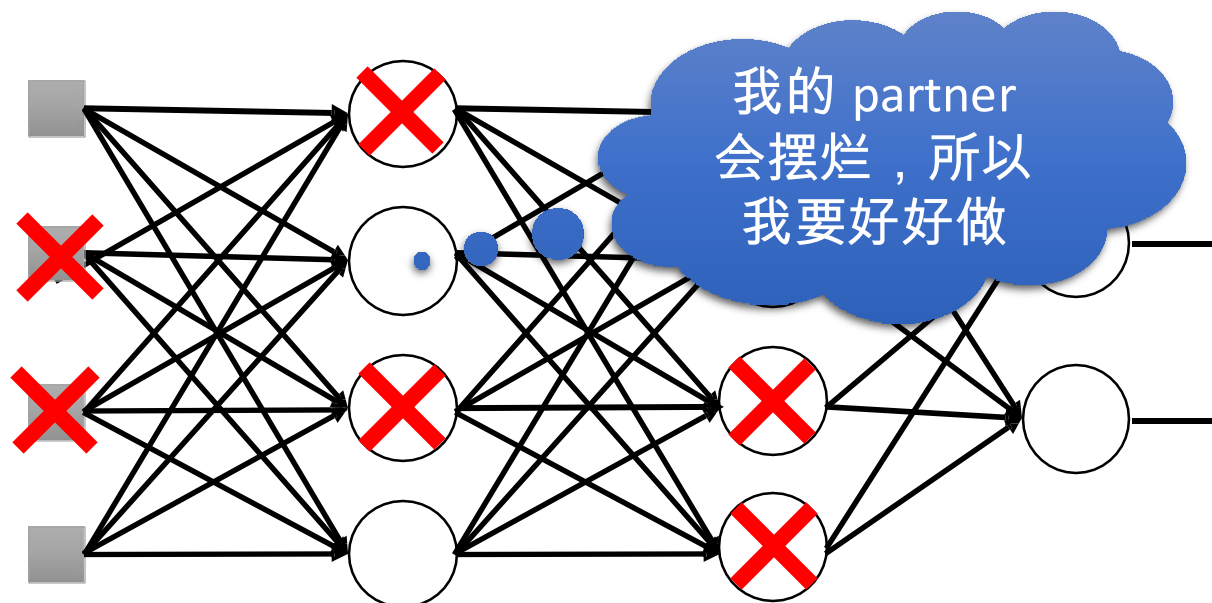
Testing:



➤ **No dropout**

- If the dropout rate at training is $p\%$, all the weights times $1-p\%$
- Assume that the dropout rate is 50%.
If a weight $w = 1$ by training, set $w = 0.5$ for testing.

Dropout - Intuitive Reason



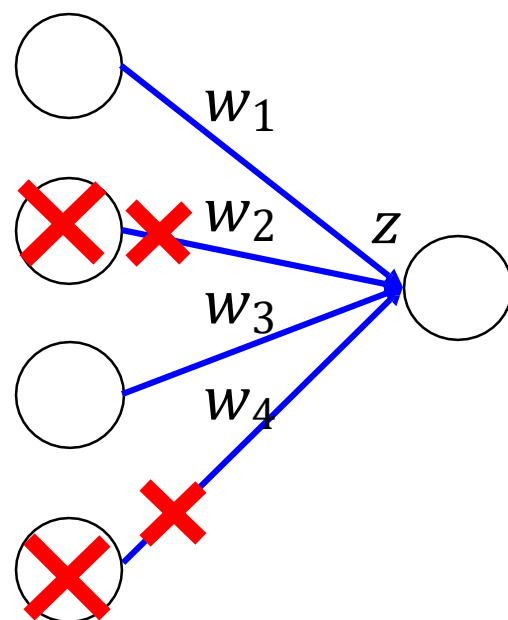
- When teams up, if everyone expect the partner will do the work, nothing will be done finally.
- However, if you know your partner will dropout, you will do better.
- When testing, no one dropout actually, so obtaining good results eventually.

Dropout - Intuitive Reason

- Why the weights should multiply $(1-p)\%$ (dropout rate) when testing?

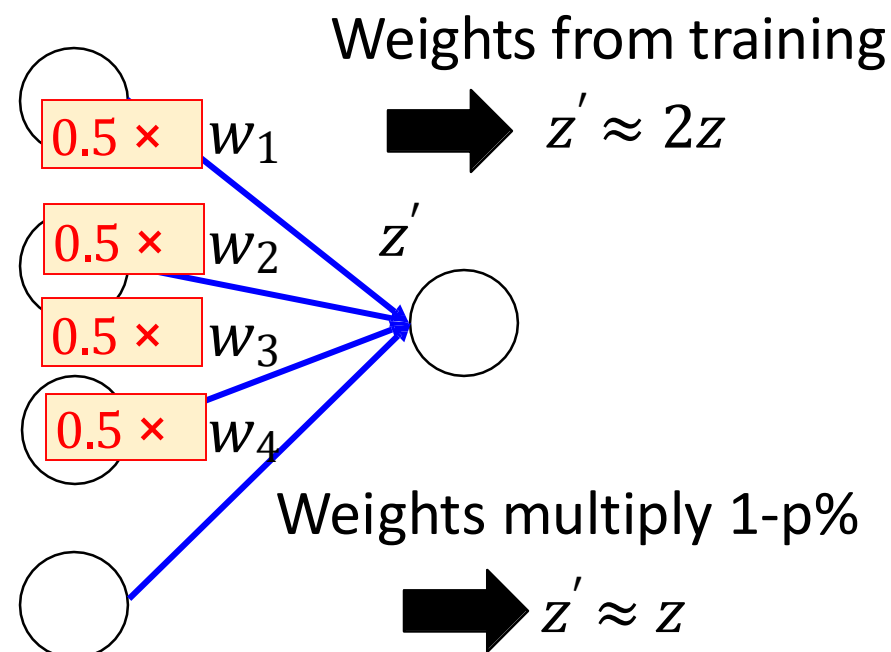
Training of Dropout

Assume dropout rate is 50%

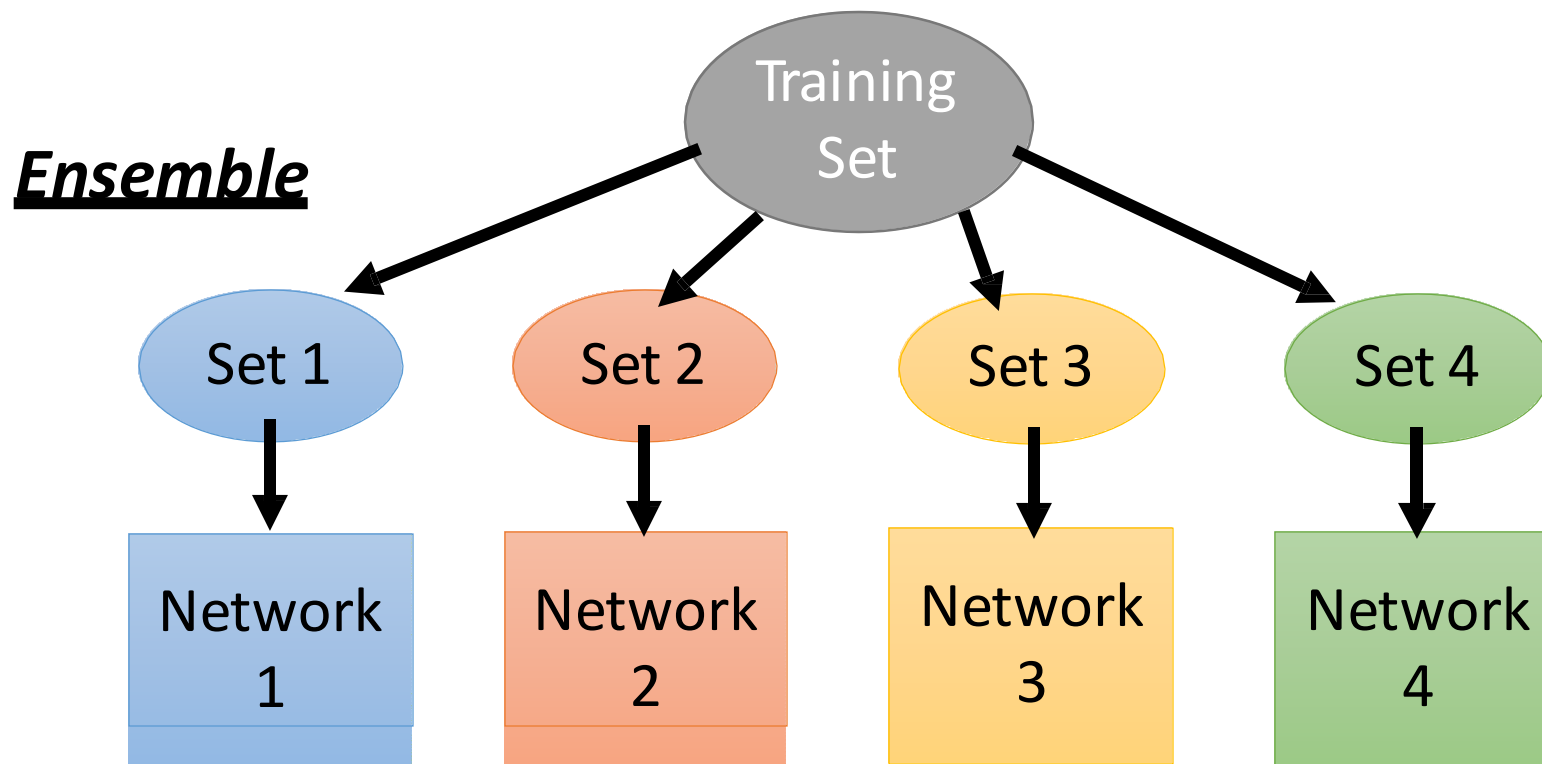


Testing of Dropout

No dropout



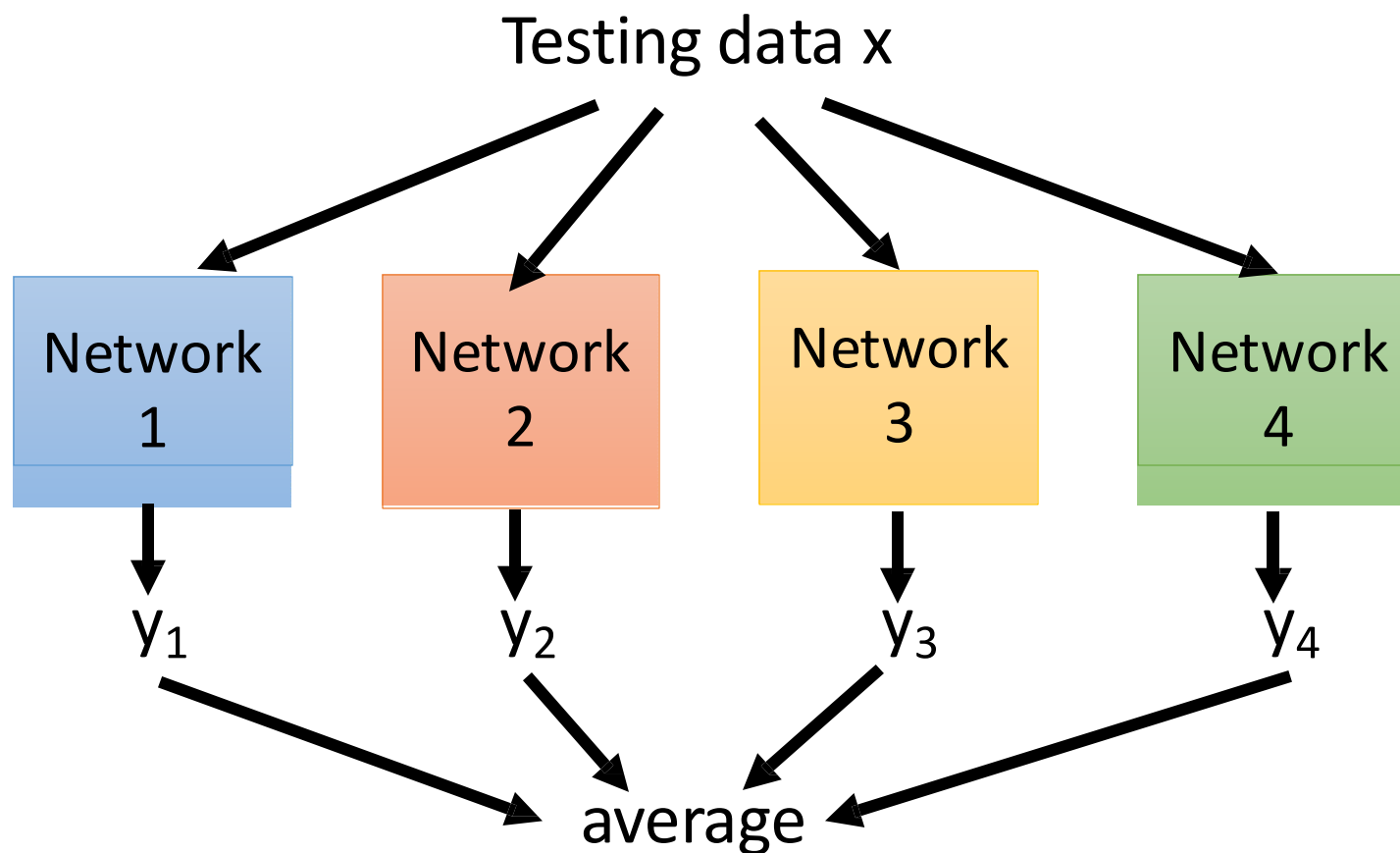
Dropout is a kind of ensemble.



Train a bunch of networks with different structures

Dropout is a kind of ensemble.

Ensemble



Dropout is a kind of ensemble.

